



UNIVERSITAS INDONESIA

**SAMPLE COMPLEXITY OF POLICY GRADIENT
REINFORCEMENT LEARNING IN ENVIRONMENTS WITH
TRANSIENT STATES**

UNDERGRADUATE THESIS

**ARDHITO NURHADYANSAH
2106750206**

**FACULTY OF COMPUTER SCIENCE
COMPUTER SCIENCE STUDY PROGRAM
DEPOK
JULY 2025**



UNIVERSITAS INDONESIA

**SAMPLE COMPLEXITY OF POLICY GRADIENT
REINFORCEMENT LEARNING IN ENVIRONMENTS WITH
TRANSIENT STATES**

UNDERGRADUATE THESIS

Submitted as one of the requirements to obtain the degree
Bachelor of Computer Science

**ARDHITO NURHADYANSAH
2106750206**

**FACULTY OF COMPUTER SCIENCE
COMPUTER SCIENCE STUDY PROGRAM
DEPOK
JULY 2025**

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Ardhito Nurhadyansah

NPM : 2106750206

Tanda Tangan : 

Tanggal : 17 Juni 2025

HALAMAN PENGESAHAN

Tugas Akhir ini diajukan oleh :
Nama : Ardhito Nurhadyansah
NPM : 2106750206
Program Studi : Sarjana Ilmu Komputer
Judul : Sample Complexity of Policy Gradient Reinforcement Learning in Environments With Transient States

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Ilmu Komputer pada Program Studi Sarjana Ilmu Komputer, Fakultas Ilmu Komputer, Universitas Indonesia

DEWAN PENGUJI

Pembimbing	: Prof. Dr.Eng. Wisnu Jatmiko, S.T., M.Kom.	(Nilai telah diberikan melalui SISIDANG pada 25-06-2025, 14:42:02) (Revisi telah disetujui melalui SISIDANG pada 04-07-2025, 19:02:27)
Pembimbing	: Vektor Dewanto, M.Eng., Ph.D.	(Nilai telah diberikan melalui SISIDANG pada 25-06-2025, 14:42:10) (Revisi telah disetujui melalui SISIDANG pada 06-07-2025, 14:59:45)
Penguji	: Prof. Heru Suhartanto, M.Sc., Ph.D.	(Nilai telah diberikan melalui SISIDANG pada 25-06-2025, 10:13:34) (Revisi telah disetujui melalui SISIDANG pada 04-07-2025, 13:12:38)
Penguji	: Meganingrum Arista Jiwanggi, S.Kom., M.Kom., M.Comp.Sc	(Nilai telah diberikan melalui SISIDANG pada 25-06-2025, 10:40:54) (Revisi telah disetujui melalui SISIDANG pada 07-07-2025, 20:38:42)

Ditetapkan di : Depok, Jawa Barat
Tanggal : 07 Juli 2025

PREFACE

All praise is due to Allah, who has bestowed His countless blessings and opportunities for me throughout the journey of completing this research titled "Sample Complexity of Policy Gradient Reinforcement Learning in Environments With Transient States". I would like to express my deepest gratitude to the following individuals and groups:

1. First and foremost, to my supervisors, Vektor Dewanto, S.T., M.Eng., Ph.D., and Prof. Dr. Eng. Wisnu Jatmiko, S.T., M.Kom., for their invaluable guidance, expertise, and encouragement throughout the research process.
2. My parents, my siblings, and my entire family, for their timeless presence, for being the first teachers of what it means to be a human, and for their countless blessings.
3. All of my friends, for the countless memorable experiences that have enriched my journey.
4. All of my teachers, whose expertise and dedication have deepened my understanding and inspired my pursuit of knowledge.
5. Everyone I have had the privilege to interact with, as each encounter has contributed a unique story and perspective to my life.
6. Every individual who strives to make this world a better and more convenient place to live.

I hope that this work may benefit all who read it and contribute positively to our shared quest for knowledge.

Depok, 17 June 2025



Ardhito Nurhadyansah

**HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI
TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS**

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Ardhito Nurhadyansah
NPM : 2106750206
Program Studi : Ilmu Komputer
Jenis Karya : Skripsi

demikian pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Sample Complexity of Policy Gradient Reinforcement Learning in Environments With
Transient States

berserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : 17 Juni 2025
Yang menyatakan



(Ardhito Nurhadyansah)

ABSTRAK

Nama : Ardhito Nurhadyansah
Program Studi : Ilmu Komputer
Judul : Kompleksitas Sampel *Policy Gradient Reinforcement Learning* di *Environments* dengan *Transient States*
Pembimbing : Prof. Dr. Eng. Wisnu Jatmiko, S.T., M.Kom.
Vektor Dewanto, S.T., M.Eng., Ph.D.

Reinforcement Learning (RL), yang berguna untuk masalah-masalah pengambilan keputusan secara sekuensial, seringkali memiliki tantangan dalam implementasinya karena kebutuhan akan sumber daya komputasi yang tinggi. Riset ini menekankan pentingnya kebutuhan akan algoritma RL yang *sample-efficient*, terutama untuk *environments* dengan *transient states*. Kami berfokus pada kondisi permasalahan dengan keberadaan *transient states* karena pada proses pengambilan keputusan yang riil, pilihan-pilihan yang diambil pada *states* terkait merupakan hal yang sangat krusial. Meskipun begitu, masih terdapat banyak implementasi praktis yang tidak menghiraukan keberadaan *transient states*, yang menyebabkan pemodelan permasalahan atau desain *environment* yang tidak optimal. Kami mensyaratkan algoritma untuk memenuhi kriteria optimalitas Blackwell, sebagai kriteria yang paling ideal pada kondisi permasalahan yang difokuskan, sebelum mengukur kompleksitas sampel yang dihasilkan. Hal ini didasarkan pada pertimbangan bahwa sebuah algoritma yang efisien akan sia-sia jika gagal menyelesaikan permasalahan utama. Namun, meskipun kriteria optimalitas Blackwell ideal secara teoretis, hal tersebut sulit untuk diaplikasikan, sehingga dibutuhkan dua kriteria lain sebagai pendekatan: *discounted reward* dan *discounting-free bias-optimality*. Algoritma-algoritma yang berbasis *discounted reward*, meskipun populer, seringkali masih menghasilkan ketidaksesuaian antara yang diprediksi dengan yang terjadi secara aktual. Selain itu, masih banyak implementasi yang menyimpang dari prinsip teoretis penggunaan sampel yang benar. Untuk menjembatani ketimpangan antara konsep teoretis dengan implementasi praktis, kami menyelidiki dua pertanyaan: (1) Bagaimana komparasi *sample-complexity* antara metode-metode *policy gradient* yang berbasis *discounting-free* dengan yang berbasis *discounted*? (2) Bagaimana komparasi performa antara metode pengambilan sampel secara *proper* (*geometrically distributed truncation*) dengan *popular* (*fixed-horizon*) dalam kerangka *discounted*? Hasil empiris menunjukkan bahwa metode *policy gradient* yang berbasis *discounting-free* secara rata-rata lebih *sample-efficient* dibandingkan yang berbasis *discounted*. Sementara itu, di dalam kerangka *discounted*, metode pengambilan sampel secara *proper* meningkatkan *sample-efficiency*, tetapi secara rata-rata, metode *popular* menghasilkan konvergensi yang lebih efektif.

Kata kunci:

Reinforcement learning, sample complexity, transient states, optimality criteria, policy gradient

ABSTRACT

Name : Ardhito Nurhadyansah
Study Program : Computer Science
Title : Sample Complexity of Policy Gradient Reinforcement Learning
in Environments With Transient States
Counselor : Prof. Dr. Eng. Wisnu Jatmiko, S.T., M.Kom.
Vektor Dewanto, S.T., M.Eng., Ph.D.

Reinforcement learning (RL), while useful for sequential decision-making problems, is often challenging to apply due to the high computational resources required. This research addresses the critical need for sample-efficient RL algorithms, particularly in environments with transient states. We focus on a problem setting with transient states because in real-world decision-making, choices made in these states are crucial. However, many implementations overlook these states, leading to suboptimal modeling or environment designs. We require algorithms to meet the Blackwell optimality criterion, which is the ideal criterion in our problem setting, as a prerequisite for assessing their sample complexity, since an efficient algorithm is not meaningful if it fails to solve the problem. However, while the Blackwell criterion is theoretically ideal, it is challenging to apply, necessitating approximations via two more tractable criteria: discounted reward and discounting-free bias-optimality. Discounted reward algorithms, though popular, often still produce discrepancies between predicted and actual outcomes, and many implementations deviate from the theoretical principles governing sample utilization. To bridge the gap between theoretical and applied RL, we investigate two questions: (1) How does the sample complexity of discounting-free policy gradient methods compare to that of discounted policy gradient methods? (2) How do "proper" (geometrically distributed truncation) versus "popular" (fixed-horizon) sampling method affect performance within the discounted framework? Empirical results show that discounting-free policy gradient methods are, on average, more sample-efficient than their discounted counterparts, while in the discounted framework, proper sampling enhances sample efficiency, but popular sampling yields better convergence on average.

Key words:

Reinforcement learning, sample complexity, transient states, optimality criteria, policy gradient

TABLE OF CONTENTS

TITLE PAGE	i
HALAMAN ORISINALITAS	ii
LEMBAR PENGESAHAN	iii
PREFACE	iv
LEMBAR PERSETUJUAN KARYA ILMIAH	v
ABSTRAK	vi
ABSTRACT	viii
TABLE OF CONTENTS	ix
LIST OF FIGURES	xi
LIST OF TABLES	xiv
1. INTRODUCTION	1
1.1 Background	1
1.1.1 Research Question	5
1.1.2 Research Scope	5
1.2 Outline	6
1.3 Research Goal	6
1.4 Research Benefits	7
2. PRELIMINARIES	8
2.1 Markov Chain	8
2.2 Markov Decision Processes (MDPs)	8
2.3 Problem-solving Setting	9
2.3.1 Dynamic Programming (DP)	9
2.3.2 Reinforcement Learning (RL)	9
2.4 Optimality Criteria	10
2.4.1 Total Reward	10
2.4.2 Discounted	11
2.4.2.1 Discounted Reward	11
2.4.2.2 Blackwell	12
2.4.3 Discounting-free	13
2.4.3.1 Average Reward	13
2.4.3.2 Bias	14
2.4.4 Bridging Optimality Criteria	15
2.4.5 Optimality Criteria For Environment with Transient States	17
2.5 Policy Iteration	17
2.6 Generalized Policy Iteration	18
2.7 Policy Gradient	21
2.8 Hyperparameter Tuning	25
2.9 Sample Complexity	26
3. RELATED WORKS	27
3.1 Optimality Criteria Comparison	27
3.2 Discounting-free Scheme	27
3.3 Discounted Scheme	28

3.4	Sample Complexity Measurements	29
4.	RESEARCH METHODOLOGY	30
4.1	Finding Bias-optimal Policies As Benchmark	30
4.2	Finding Effective Methods	31
4.3	Making Sure Each Bracket Has a Representative to Compare	32
4.4	Finding The Most Sample-efficient Methods From Each Bracket	33
4.5	Finding The Most Sample-efficient Scheme Between Each Bracket Per Environment	33
4.6	Finding The Most Sample-efficient Scheme Between Each Bracket On Average	34
5.	EXPERIMENT SETUP	35
5.1	Environment Design	35
5.2	Policy Parameterizations	37
5.3	Exact Policy Gradient	37
5.4	Policy Gradient	38
5.4.1	Discounting-free Policy Gradient	39
5.4.2	Discounted Policy Gradient	45
5.5	Algorithm Parameters	49
5.6	Hyperparameter Tuning	50
5.7	Value Landscape	51
6.	RESULTS AND ANALYSIS	55
6.1	Sample Complexity Analysis	55
6.1.1	Bias-optimal Policies	55
6.1.2	Effectiveness Measure	55
6.1.3	Sample Complexity Measure	58
6.1.3.1	Discounted vs Discounting-free Scheme	58
6.1.3.2	Popular vs Proper Sampling in Discounted Scheme	60
6.2	Optimization Behavior	63
6.2.1	Exact Policy Gradient	64
6.2.1.1	Vanilla Policy Gradient	65
6.2.1.2	Natural Policy Gradient	69
6.2.2	Policy Gradient	72
6.2.2.1	Vanilla Policy Gradient	72
6.2.2.2	Natural Policy Gradient	75
7.	CONCLUSIONS	77
7.1	Conclusions	77
7.2	Limitations	77
7.3	Future Works	77
	REFERENCES	79
	GLOSSARY	1

LIST OF FIGURES

Figure 1.1.	Amount of compute needed for OpenAI Five (Berner et al., 2019) . .	2
Figure 1.2.	An illustration that decisions in transient states are important: A disease progression Markov model (Briggs and Sculpher, 1998). . .	3
Figure 1.3.	Taxonomy of RL based on the research setting.	5
Figure 2.1.	Learning curves of Q-Learning with various discount factor (Dewanto and Gallagher, 2021a)	11
Figure 2.2.	Blackwell Optimality	12
Figure 2.3.	Average Reward Is Not Enough in Unichain MDPs with Transient States	14
Figure 2.4.	Veinott's Discounting-free Optimality (Dewanto and Gallagher, 2021b)	16
Figure 2.5.	Bridging Optimality Criteria	16
Figure 2.6.	Optimality Criteria for Environment with Transient States. Blue: ideal and theoretical optimality criteria. Green: practical optimality criteria. Red: Not considered as it is underselective for environments with transient states.	17
Figure 2.7.	Policy Iteration Process (Sutton and Barto, 2018)	17
Figure 3.1.	Related works and Our Research Position	27
Figure 4.1.	Phase 1 - Finding bias-optimal policies	30
Figure 4.2.	Phase 2 - Finding the effective methods	31
Figure 4.3.	Phase 3 - Making sure each bracket has a representative to compare .	32
Figure 4.4.	Phase 4 - Finding the most sample-efficient policy gradient from each bracket	33
Figure 4.5.	Phase 5 - Compare which one has the best sample complexity between each bracket	33
Figure 4.6.	Phase 6 - Compare which one has the best sample complexity (on average)	34
Figure 5.1.	Environments (first row): A1, A2. Environments (second row): B1, B2, B3. Figures are obtained from (Dewanto and Gallagher, 2021b), where environment A3 is not plotted because it is too complex and unintuitive to show. Each transition is represented by an arrow. In each of the arrow, there is tuple consisting of three elements, which consecutively corresponds to the type of the action, the immediate reward, and the probability of the transition.	35
Figure 5.2.	Policy gradient for discounting-free scheme	40
Figure 5.3.	Sampling process for discounting-free scheme	43
Figure 5.4.	Policy gradient for discounted scheme	46
Figure 5.5.	Sampling process for discounted scheme	48
Figure 5.6.	Hyperparameter tuning process for discounted scheme to find the best γ	50

Figure 5.7.	Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain landscape, bias landscape, discounted value landscape. For the discounted value landscape, the gamma chosen is the lowest tuned gamma across the methods, where for each environment (top to bottom) consecutively: 0.94,0.91,0.91,0.91,0.92,0.92 . . .	52
Figure 6.1.	Effectiveness measure for each policy gradient methods in environments with A variant. From top to bottom: A1, A2, A3	56
Figure 6.2.	Effectiveness measure for each policy gradient methods in environments with B variants. From top to bottom: B1, B2, B3	57
Figure 6.3.	Average and standard deviation for empirical samples needed for convergence across each policy gradient methods in each environment. First row: A1, A2, A3. Second row: B1, B2, B3.	58
Figure 6.4.	Summary - Number of samples needed to achieve convergence between the most sample-efficient method in discounted and discounting-free scheme in each environment	59
Figure 6.5.	Summary - Sample Efficiency and Effectiveness across all methods in all environments	60
Figure 6.6.	Summary - Effectiveness for discounted scheme between popular and proper sampling	61
Figure 6.7.	Summary - Best methods for discounted between popular and proper sampling from three different perspectives: gradient-type-agnostic, vanilla PG, and natural PG	62
Figure 6.8.	Trajectory for Discounting-free scheme. Environment (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: Vanilla PG's gain and bias trajectory, Natural PG's gain and bias trajectory.	63
Figure 6.9.	Trajectory for Discounted scheme. Environment (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: Vanilla PG's with popular sampling, Vanilla PG's with proper sampling, Natural PG's with popular sampling, Natural PG's with proper sampling.	64
Figure 6.10.	Trajectory for Vanilla Policy Gradients (Exact). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain trajectory, bias trajectory, discounted value trajectory (exact). The resulting γ sequentially from A1 to B3: 0.94,0.99,0.97,0.94,0.99,0.94	66
Figure 6.11.	Policy Value Progression for Vanilla Policy Gradient (Exact). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain progression, gain + bias progression, discounted value progression	67
Figure 6.12.	Trajectory for Natural Policy Gradients (Exact). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain trajectory, bias trajectory, discounted value trajectory (exact).	69
Figure 6.13.	Policy Value Progression for Natural Policy Gradient (Exact). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain progression, gain + bias progression, discounted value progression	71

Figure 6.14.	Trajectory for Vanilla Policy Gradient (Sampling). Environment (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain trajectory, bias trajectory, discounted value trajectory (popular), discounted value trajectory (proper). The resulting γ sequentially from A1 to B3: 0.99, 0.96, 0.97, 0.91, 0.99, 0.91, 0.94, 0.91, 0.94, 0.99, 0.94, 0.92 . . .	72
Figure 6.15.	Policy Value Progression for Vanilla Policy Gradient (Sampling). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain progression, gain + bias progression, discounted value progression for popular sampling, discounted value progression for proper sampling. If there is no resulting line plotted, it means that the policy converged directly at the first iteration.	74
Figure 6.16.	Trajectory for Natural Policy Gradient (Sampling). Environment (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain trajectory, bias trajectory, discounted value trajectory (popular), discounted value trajectory (proper). The resulting γ sequentially from A1 to B3: 0.97, 0.96, 0.94, 0.99, 0.96, 0.91, 0.99, 0.91, 0.92, 0.99, 0.99, 0.97 . . .	75
Figure 6.17.	Policy Value Progression for Natural Policy Gradient (Sampling). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain progression, gain + bias progression, discounted value progression for popular sampling, discounted value progression for proper sampling. If there is no resulting line plotted, it means that the policy converged directly at the first iteration.	76

LIST OF TABLES

Table 5.1.	Tuned gamma obtained after hyperparameter tuning	51
Table 6.1.	Bias-optimal policies in each environment and its value	55
Table 6.2.	Effective policy gradients in each environment	58
Table 6.3.	Samples needed for convergence in each environment	60

CHAPTER 1

INTRODUCTION

1.1 Background

Reinforcement Learning is useful for solving sequential decision-making problems. It tries to find an optimal policy for the agent to behave optimally with respect to some optimality criteria. Some success story of RL implementations happened in many domains. For example, in the field of board games like Go (Silver et al., 2016,1) and Chess (Silver et al., 2017a), video games like Atari (Mnih et al., 2013) and Dota 2 (Berner et al., 2019), robotics like dexterous in-hand manipulation (OpenAI et al., 2018), to language-reasoning model like DeepSeek R1 (DeepSeek-AI et al., 2025).

However, RL is computationally expensive. Unlike in Supervised Learning or Unsupervised Learning, by which the model is trained by a static data that's given, in RL the data is obtained by agent-environment interactions through simulations in the form of (current state, action, reward, next state). Moreover, in RL there are no labels for the data. The feedback given is evaluative (in the form of reward) instead of instructive (in the form of correct answer). Therefore, additional layers of approximations (to find the most appropriate label for training the model) is needed.

One example where RL training is expensive is OpenAI Five, which is a Dota 2 RL agents that successfully defeated the world champions (Berner et al., 2019). In total, they use 770 ± 50 PFlops/s-days for 10 months of compute with thousands of GPUs to reach that level. Another examples are the AlphaGo Zero (Silver et al., 2017b), which needs 72 hours of self-play to achieve superhuman Go agent, and (Zheng et al., 2021) which needs > 7 days using 32 CPU threads (Ponse et al., 2025). The amount of compute is actually related with sample complexity, i.e., a more sample-efficient algorithms leads to less compute needed.

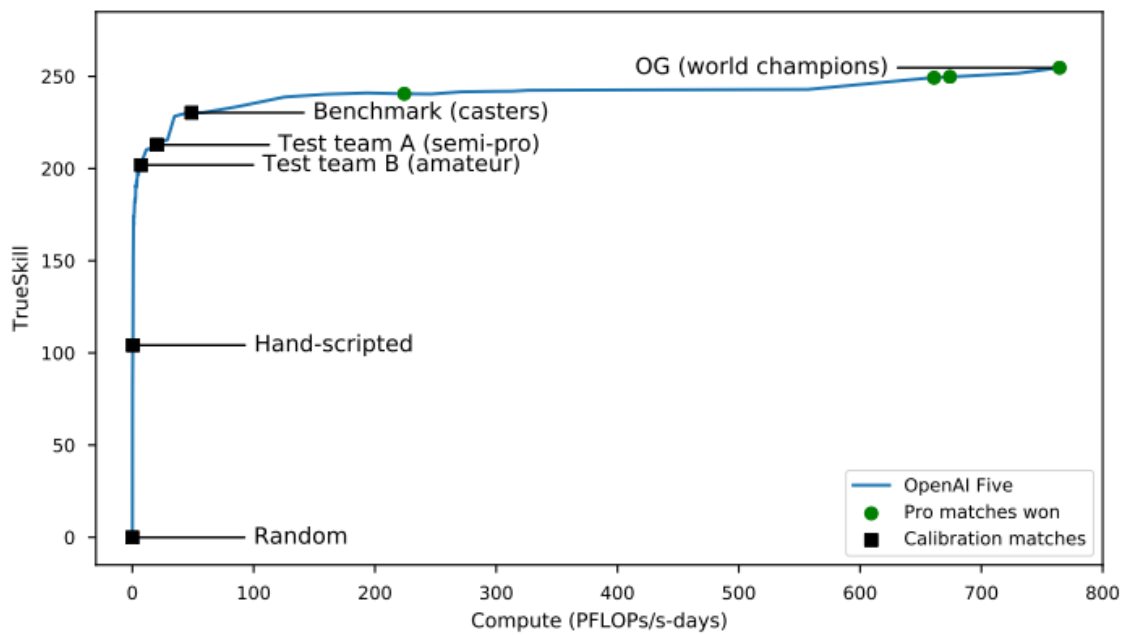


Figure 1.1: Amount of compute needed for OpenAI Five (Berner et al., 2019)

A sample-efficient algorithm needs an effectiveness with respect to some optimality criteria as a requirement. This is because faster and cheaper algorithms are not meaningful if the resulting policy is not optimal. In terms of choosing the optimality criteria, we also need to be careful because wrong selection of it leads to optimal decisions based on misunderstanding of the problem. In our case, we want to find the most suitable and advantageous one for cases with transient states.

We consider environment with transient states because it is important to consider decisions in those states. One example is the disease progression Markov model (Briggs and Sculpher, 1998). In the model 1.2, there are two transient states: asymptotic disease and progressive disease, and one recurrent state: mortality, where we want to prevent to go to the latter, which probability depends upon decisions in previous states. Other examples that illustrate decisions in transient states is important include stroke care (McClean, 2021) and myocardial infarction care (Sonnenberg and Beck, 1993).

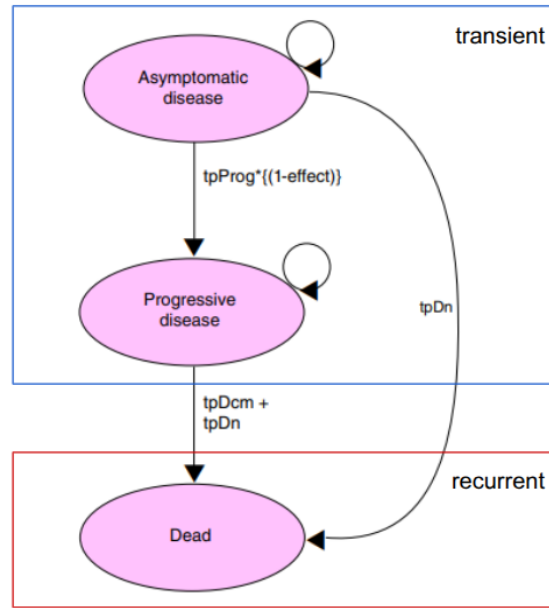


Figure 1.2: An illustration that decisions in transient states are important: A disease progression Markov model (Briggs and Sculpher, 1998).

Abbreviations: effect = effectiveness of drug in reducing disease progression; tpDcm = transition probability to death caused by the disease; tpDn = transition probability to natural death; tpProg = transition probability to progression.

For environment with transient states, we cannot use average reward as the optimality criteria, as it is underselective (Dewanto and Gallagher, 2021b). In this case, the most proper optimality criteria is Blackwell. However, it is hard to obtain in practice, because in RL, the agent doesn't know the model of the environment, which needed to be able to compute the upper bound of the Blackwell discount factor (Grand-Clement and Petrik, 2023). Therefore, we need to approach it approximately using another optimality criteria, i.e., discounted and discounting-free. For discounted optimality criteria, we need to tune the discount factor γ to be greater than the Blackwell discount factor γ_{BW} . Meanwhile for discounting-free optimality criteria, we will utilize the structure from Veinott's n-discount optimality criteria (Veinott, 1969), and assumed that bias-optimality, which corresponds to $(n=0)$ -discount optimality, to be equivalent to Blackwell-optimality. Optimality criteria in RL also affects the amount of samples needed for the agent to get into its optimal policy. For instance, under discounted reward optimality criteria, as there is additional hyperparameter, i.e., the discount factor γ , we need to take into account the samples needed to do hyperparameter tuning.

For the algorithm, we will use policy gradient methods, which is a class of RL algorithms that directly optimize the policy. It is chosen because policy gradient concepts are the foundation for state-of-the-art RL algorithms, e.g., PPO (Schulman et al., 2017), SAC (Mnih et al., 2016), and DDPG (Lillicrap et al., 2015). However, there are still mismatch between predicted and empirical behavior of policy gradients, especially in Deep RL (Ilyas et al., 2018). Not only that, in discounted scheme, some policy gradient methods implements improper sampling method which leads to biased estimation of the policy value gradient (Nota and Thomas, 2019), i.e., dropping the discount factor γ and using all trajectory data to compute the gradient instead of sampling it from the proper distribution $Geo(1 - \gamma)$. In this research, we will refer the non-proper sampling based implementation as popular sampling method, whilst the other side as proper sampling.

There are several reasons of why this direction of research is important. First, Current RL algorithms tends to rely on ergodicity assumption (Agarwal et al., 2019; Bai et al., 2024; Moldovan and Abbeel, 2012), which might not true in realistic scenarios. Ergodicity assumption implies recurrent chain assumption, which doesnt consider transient states (Puterman, 1994). Second, in unichain MDPs, many still use average reward as the optimality criteria, e.g., (Ganesh and Aggarwal, 2025), whilst if there exist transient states, we need to expand from average reward to bias-optimality (Dewanto and Gallagher, 2021b) or deeper to attain Blackwell criteria (Puterman, 1994). Most of the algorithms that based on discounting-free scheme is also only successfully developed until average reward criteria, which is underselective in environment with transient states. Moreover, while it is possible, few that consider unichain MDPs to rely on discounted optimality criteria, where they actually need to tune $\gamma > \gamma_{BW}$ (it is also hard in practice because knowing the true model is often infeasible, making it difficult to obtain the γ_{BW}). Third, in applied RL, sometimes they dont even consider MDP classification (which implies no transient states consideration) and use discounted reward even for environment without inherent discount factor, e.g., semiconductor production scheduling with Deep Q Network (DQN) (Waschneck et al., 2018). Forth, in terms of discounted scheme policy gradient methods, many of the implementations use popular sampling method, e.g., A3C (Mnih et al., 2016), and ACKTR (Wu et al., 2017).

In summary, based on the setting of the research, we consider the problem based on the taxonomy given below.

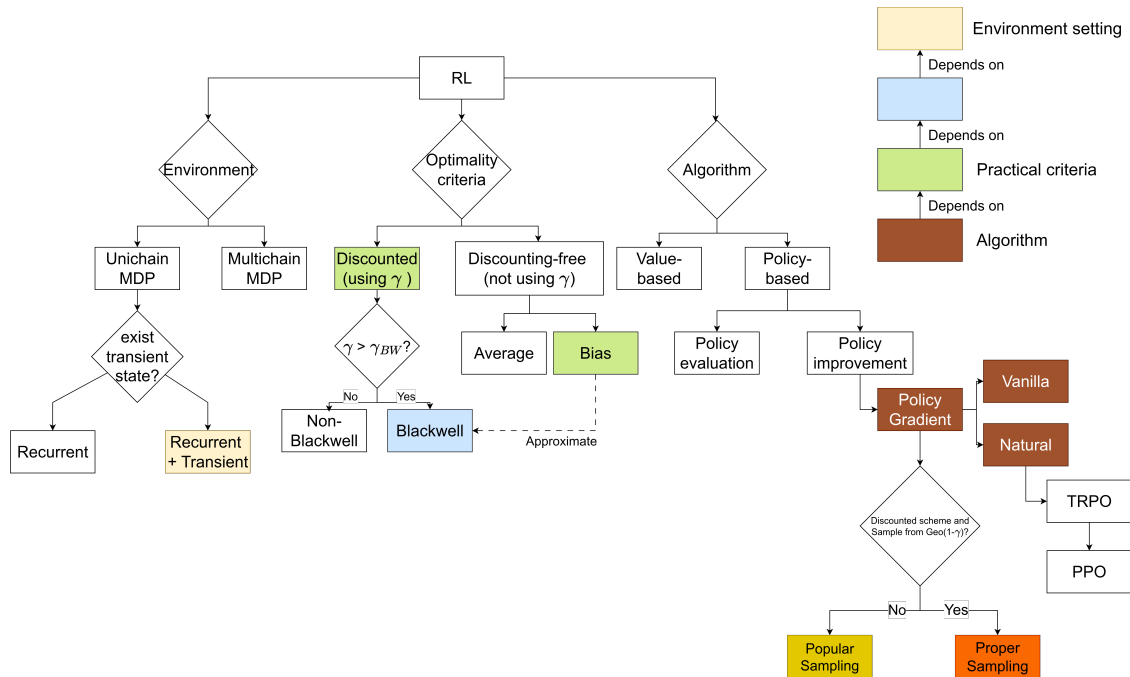


Figure 1.3: Taxonomy of RL based on the research setting.

1.1.1 Research Question

1. How do discounting-free policy gradient methods compare to discounted policy gradient methods in terms of sample complexity?
2. How do proper sampling compare to popular sampling in discounted policy gradient methods in terms of performance?

1.1.2 Research Scope

1. Finite MDP, where the state set and action set are finite
2. Infinite horizon MDP, where the time set or decision epoch is infinite (no upper bound)
3. The environments considered are unichain MDPs with transient states
4. Policy gradient involves exact policy evaluations
5. The learning rate (α) is obtained through backtracking line search ascent
6. We assume that Blackwell-optimality is equivalent to bias-optimality for discounting-free scheme.

1.2 Outline

- Chapter 1 INTRODUCTION

This chapter is intended to give background of what and why we do the research.

- Chapter 2 PRELIMINARIES

This chapter is filled with preliminary concepts that's used or correlated with the research.

- Chapter 3 RELATED WORKS

This chapter depicts the related works and our position in the field.

- Chapter 4 RESEARCH METHODOLOGY

This chapter describes the research methodology to answer the research questions.

- Chapter 5 EXPERIMENT SETUP

This chapter explains the setup for the experiments, from environment design, policy parameterizations, procedure for the algorithms, algorithm parameters, hyperparameter tuning, and the initial observations on the value landscape for the policy optimization process.

- Chapter 6 RESULTS AND ANALYSIS

This chapter elaborates the main results and analysis of the experiments. We analyze the effectiveness, sample-complexity, and additional analysis on the policy optimization behavior.

- Chapter 7 CONCLUSIONS

This chapter gives conclusions, limitations, and future works recommendations for the research.

1.3 Research Goal

The main goal of this research are:

1. Compare the sample complexity of discounting-free policy gradient methods vs its discounted reward counterpart
2. Compare the performance of different sampling method, i.e., popular and proper, for policy gradients with discounted reward optimality criteria

In addition, we also want to observe the behavior of policy gradient methods based on three different perspectives: the optimality criteria (discounted vs discounting-free), the

sampling type (proper vs popular), and the gradient type (vanilla vs natural).

1.4 Research Benefits

We hope that this research could bring benefits such as:

1. Empirical comparison of sample-complexity between 2 schemes of optimality criteria (discounting-free vs discounted)
2. Empirical comparison of performance between 2 schemes of sampling in discounted-based optimality criteria (proper vs popular)
3. Better understanding of policy gradient methods behavior with three different perspectives: the optimality criteria (discounted vs discounting-free), the sampling type (proper vs popular), and the gradient type (vanilla vs natural).

CHAPTER 2

PRELIMINARIES

2.1 Markov Chain

Puterman (1994) defines Markov Chain as a sequence of random variables $\{X_0, X_1, X_2, \dots, X_n\}$, where each of the elements corresponds to a discrete value in state space S and for $n \geq 1$ and $j_k \in S, 0 \leq k \leq n$:

$$P(X_n = j_n | X_{n-1} = j_{n-1}, X_{n-2} = j_{n-2}, \dots, X_0 = j_0) = P(X_n = j_n | X_{n-1} = j_{n-1}) \quad (2.1)$$

The above equation tells one of the key property of Markov Chain, i.e., Markov Property, where future outcomes only depends on current state.

A state i is called:

- Recurrent if, starting from i , the chain returns to i with probability 1; otherwise it is transient.
- Periodic with period $d > 1$ if returns to i are only possible at multiples of d . If $d = 1$, i is aperiodic.
- Ergodic if recurrent and aperiodic.

2.2 Markov Decision Processes (MDPs)

Formally, we can define a Markov Decision Process (MDP) as a tuple $(S, \mathcal{A}, \mathcal{T}, \overset{\circ}{p}, p, r)$ where:

- S is the set of states, representing all possible situations the agent can be in
- $\mathcal{A}$ is the set of actions, representing all possible actions the agent can take
- $\mathcal{T}$ is the time set, which represents the discrete time steps at which the agent interacts with the environment
- $\overset{\circ}{p} : S \rightarrow [0, 1]$ is the initial state distribution, which defines the probability of starting in each state

- $p : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the state transition probability function, where $p(s'|s, a)$ is the probability of transitioning to state s' from state s after taking action a
- $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, where $r(s, a)$ is the expected reward received after taking action a in state s

With the above formalization, we could have a well-defined problem to solve. The target is to find the most optimal sequential decisions throughout the horizon for the agent. Decision-making mechanism for the agent here could be formalized to something called a policy, i.e., a mapping from state to action probability distribution $f : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$, usually denoted with $\pi(a|s)$ where $s \in \mathcal{S}, a \in \mathcal{A}$. In unichain infinite-horizon, finite MDPs, it is assured that exist an optimal policy that is stationary and deterministic, i.e., $\pi^* \in \Pi_{SD}$. Therefore, in that case, the policy degenerates to a deterministic mapping from state to action $f : \mathcal{S} \rightarrow \mathcal{A}$. The correlation between MDP and Markov Chain lies where the resulting trajectory from the stationary policy in an infinite-horizon finite MDP induces a Markov Chain.

2.3 Problem-solving Setting

To solve the MDP, there are generally two setting that one can use, i.e., Dynamic Programming (DP) or Reinforcement Learning (RL). The appropriate setting to be used is based on the situation of what we have as the problem-solver or the inherent characteristics of the problem. Each of the which will be explained in more detailed below.

2.3.1 Dynamic Programming (DP)

DP setting requires the model of the environment to be known exactly by us, the programmer. Then, it will be given to the agent, specifically the state transition probability distribution p and the reward function r . Those two components are useful for the agent to do exact policy evaluations and exact policy improvements (more on this later at 2.5).

2.3.2 Reinforcement Learning (RL)

In RL setting, the agent doesn't have the model of the environment. This setting is needed when we, the problem-solver, doesn't know the model of the environment dynamics, i.e., the transition probability p and the reward function r . The other condition is when those are known, but it is not practical as of limitations in resources and time, because the state

space S or action space A is too big to be represented exactly. In that regard, we need to use approximation to be able to generalize the whole state or action space. This will be explained in more detailed in section 2.6.

2.4 Optimality Criteria

Optimality criteria refer to the measurement of the objective that agent will try to achieve. In MDPs, the one that can give signal on what's good or bad is the reward that's given from the environment to the agent after it selects and execute an action. In RL, we have a phrase called "reward is enough" (Silver et al., 2021; Sutton and Barto, 2018). It is a hypothesis that the objective of the problem could be measured solely based on the expected cumulative reward that the agent obtained throughout the horizon.

2.4.1 Total Reward

The most straightforward optimality criteria is to measure the expected total reward, i.e.:

$$v_{tot}(\pi, s) = \lim_{t_{max} \rightarrow \infty} \mathbb{E}_{S_{t+1} \sim p(\cdot | s_t, a_t), A_t \sim \pi(\cdot | s)} \left[\sum_{t=0}^{t_{max}-1} r(S_t, A_t) \mid S_0 = s, \pi \right] \quad (2.2)$$

But in practice, it is problematic because as time goes to infinity, the total reward, assuming that the recurrent chain produces a non-zero reward, will lead to a value of ∞ or $-\infty$. Therefore, if we have two policies, and both yields the value of one of those values, we cannot really compare them on which one is better. As a remedy, we could use discounted reward or discounting-free optimality criteria. In addition, there is another theoretical optimality criteria that bridge the correlation between the two, i.e., the Blackwell optimality criteria. A comprehensive comparison of those optimality criteria has been reviewed comprehensively in (Dewanto and Gallagher, 2021a).

2.4.2 Discounted

2.4.2.1 Discounted Reward

Discounted reward is by far the most popular optimality criteria in RL. Formally, it can be expressed as follows:

$$v_\gamma(\pi, s) = \lim_{t_{\max} \rightarrow \infty} \mathbb{E}_{S_{t+1} \sim p(\cdot | s_t, a_t), A_t \sim \pi(\cdot | s)} \left[\sum_{t=0}^{t_{\max}-1} \gamma^t r(S_t, A_t) \mid S_0 = s, \pi \right] \quad (2.3)$$

In some environments, there is an inherent notion of discounting, but some is not. In environments without inherent notion of discounting, there are several reasons that it is make sense to be used. Unlike in discounting-free settings, it is independent regardless of the chain classification. In addition, it is also contractive which helps convergence and the discount factor γ could reduce the variance of policy gradients or treated as a regularizer as reviewed in (Amit et al., 2020), exchanging it with bias.

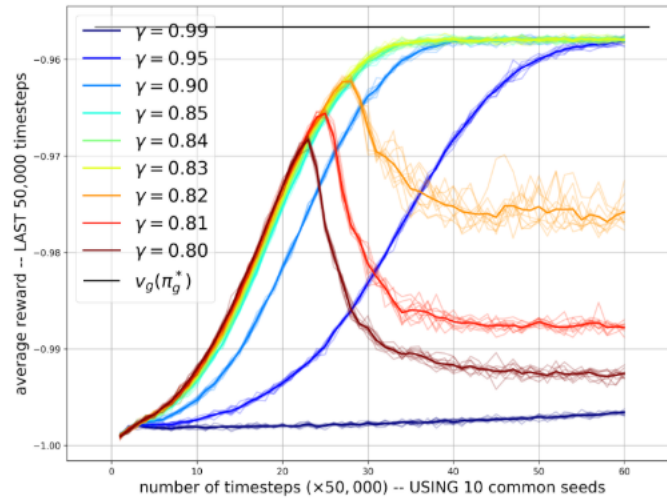


Figure 2.1: Learning curves of Q-Learning with various discount factor (Dewanto and Gallagher, 2021a)

However, discounted reward optimality has some problems. As we can infer from the figure above, the discount factor is a sensitive hyperparameter. It is also determining the objective of the problem, as it represents the discount optimality criteria. Lower value induces faster convergence but a more shortsighted horizon, whilst higher value produces the opposite.

In addition, there are some setting that makes discounted RL is not an optimization problem (Naik et al., 2019), which related to the validity of the problem solution. Optimization problem needs at least a set of feasible solutions and an objective function that produces a scalar value that represents each of the solution's value. In continuing task with RL setting, we need to use function approximation. With that, using discounted reward as the optimality criteria is not producing an optimization problem, because there's no clear objective function, since policy's value depends on initial state, hence different initial state could produce different optimal policies. We could make the objective as a single number by making possible initial states as weights based on initial state distribution, but such approach might not be appropriate as the initial state might not represent the whole states that will be visited by the agent throughout the horizon. Such objective could be made more sense through several ways, where it eventually leads to average reward optimality criteria.

2.4.2.2 Blackwell

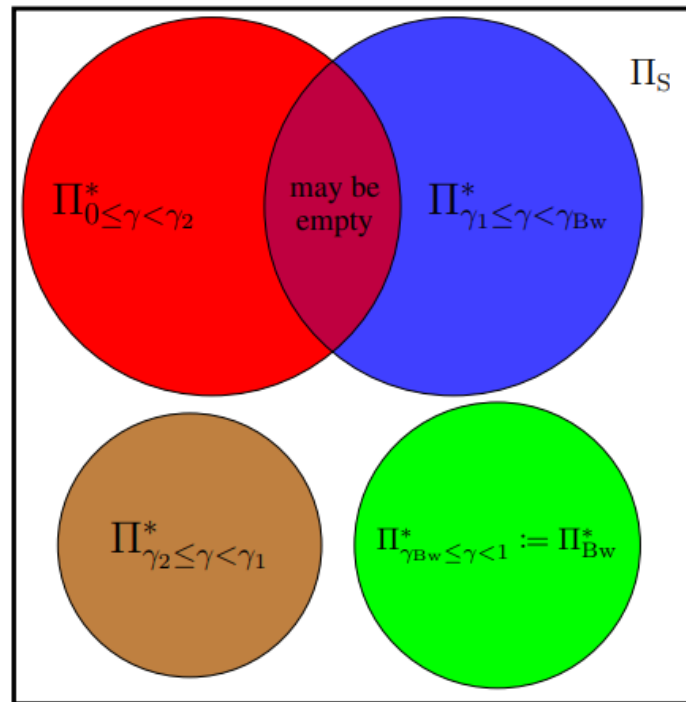


Figure 2.2: Blackwell Optimality

Blackwell optimality criteria is a special case of discounted reward optimality criteria, As can be seen in Figure 2.2, different discount factor γ produces different set of optimal policies. Interestingly, for any $\gamma > \gamma_{Bw}$, the set of optimal policies remains the same.

Blackwell optimality criteria refers to discounted criteria with the additional requirement that the $\gamma > \gamma_{BW}$. Formally, a Blackwell optimal policy π_{BW}^* must fulfill this requirement:

$$\forall \pi \in \Pi_S, \forall s \in \mathcal{S}, \quad v_\gamma(\pi_{BW}^*, s) \geq v_\gamma(\pi, s), \quad 0 \leq \gamma_{BW} \leq \gamma \leq 1 \quad (2.4)$$

Conceptually, Blackwell is regarded as the most selective optimality criteria in unichain MDPs (Puterman, 1994). However, in practice it is hard to find the value of γ_{BW} , as it depends on the MDP characteristics (Grand-Clement and Petrik, 2023). In RL, the agent doesn't know the MDP model exactly. Thus, hyperparameter tuning for γ is needed to find the best γ that ideally is the smallest value that is greater than γ_{BW} . But, even hyperparameter tuning (in this case, in practice usually we use average reward as the evaluation metric) doesn't guarantee that the agent will find the best $\gamma > \gamma_{BW}$. It is because average reward is just one component to reach the Blackwell optimality criteria, i.e., the first term of Laurent series expansion (Veinott, 1969).

2.4.3 Discounting-free

2.4.3.1 Average Reward

Average reward is more suitable for real world applications, as it considers long-term behavior of the agent throughout the horizon. Formally, it could be defined as:

$$v_g(\pi, s) = \lim_{t_{max} \rightarrow \infty} \frac{1}{t_{max}} \mathbb{E}_{S_{t+1} \sim P(\cdot | s_t, a_t), A_t \sim \pi(\cdot | s)} \left[\sum_{t=0}^{t_{max}-1} r(S_t, A_t) \mid S_0 = s, \pi \right] \quad (2.5)$$

Choosing average reward as the optimality criteria has several benefits. First, in recurrent MDPs, average reward is unconditionally the most selective criteria. It is because there is no transient states, and the average reward is constant in unichain MDPs, which is the superset of recurrent MDPs. Second, as average reward is constant across all states in unichain MDPs, it makes the optimization independent of the initial state or its distributions. This is in contrast with discounted reward optimality criteria, which depends on the initial state s_0 to measure $v_\gamma(\pi, s_0)$. Third, average reward optimality criteria is potentially more sample efficient, or has higher convergence rate compared to discounted. The reason is because of the efficiency gained by the independence of initial state, as

mentioned in the second reason of the benefits. In addition, unlike in discounted reward, average reward doesn't involve discount factor. Therefore, there is no problem for tuning the corresponding hyperparameter. Moreover, average reward optimizes just the first term of the Laurent series expansion of the Blackwell optimality criteria, i.e., v_{-1} compared to discounted reward which indirectly optimizes the whole $v_{-1} + v_0 + \dots + v_{|S|-2}$ at the same time. Forth, there might be less discrepancy between the learning objective and the performance metric compared to discounted optimality criteria. It is because the final learning performance metric in RL usually measured with the average return over several last experiment-episodes. Average-reward based algorithms has been reviewed extensively in (Dewanto et al., 2020).

However, average reward optimality criteria has some drawbacks, especially in unichain MDPs with transient states, as can be seen in the illustration below.

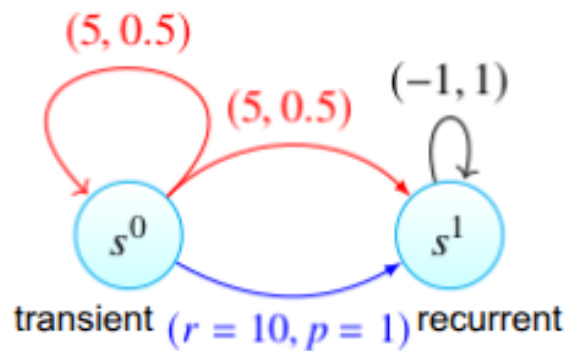


Figure 2.3: Average Reward Is Not Enough in Unichain MDPs with Transient States

In the above scheme, there are 2 stationary-deterministic policies:

- Policy 1: choose red at s^0 , and black at s^1 .
- Policy 2: choose blue at s^0 , and black at s^1 .

Both of the policies yield the same average reward, i.e., -1 . Therefore, the average reward is not sufficient to distinguish the quality between these two policies.

2.4.3.2 Bias

To get rid of the previous drawback, we can use bias as the additional term to average reward optimality criteria. Bias is a measure of the difference between the expected total

reward and the average reward. Therefore, bias is a measure of the quality of the policy, i.e., how much better the value of the policy is compared to the average reward. With that scheme, we can also gauge the transient behavior of the policy, beside the steady-state behavior. Formally, bias could be defined as:

$$v_b(\pi, s) = \lim_{t_{max} \rightarrow \infty} \mathbb{E}_{S_{t+1} \sim p(\cdot | s_t, a_t), A_t \sim \pi(\cdot | s)} \left[\sum_{t=0}^{t_{max}-1} r(S_t, A_t) - v_g(\pi) \mid S_0 = s, \pi \right] \quad (2.6)$$

Bias-optimal means that the policy is optimal with respect to the average reward and bias. Therefore, the optimal policy π^* must fulfill this requirement:

$$v_b(\pi_b^*, s) \geq v_b(\pi_g^*, s), \forall s \in S, \forall \pi_g^* \in \Pi_g^*, \text{ where } \Pi_g^* := \arg \max_{\pi \in \Pi_S} v_g(\pi) \quad (2.7)$$

With bias consideration, in the previous case 2.3, the agent could distinguish value, thus quality between policies, where the Policy 1 has bigger bias than Policy 2 (because it can achieve higher rewards in transient states by accumulating more rewards), therefore it is considered to be better than Policy 2.

2.4.4 Bridging Optimality Criteria

Interestingly, we can express Blackwell (which is the most selective criteria in unichain MDPs) in the form of discounting-free by utilizing the Laurent Series Expansion (Veinott, 1969) as follows:

$$v_\gamma(\pi) = \frac{1}{\gamma} v_{-1}(\pi) + v_0(\pi) + e(\pi, \gamma) \quad (2.8)$$

where $e(\pi, \gamma)$ is the error term, which converge to $\mathbf{0}$ as γ approaches 1.

A more detailed structural correlation between the discounted and discounting-free optimality criteria can be seen in Figure 2.4 below.

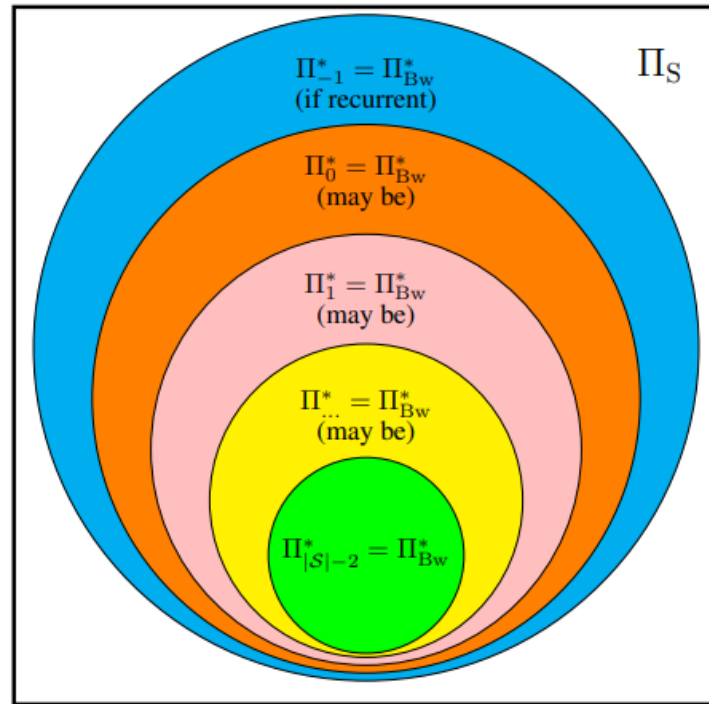


Figure 2.4: Veinott's Discounting-free Optimality (Dewanto and Gallagher, 2021b)

From the above figure, we can see that in recurrent MDPs (no transient states), Blackwell is equal to average criteria or $(n=-1)$ -optimality. In environment with transient states, Blackwell might not be achieved until reaching level $(n = |S| - 2)$ -optimality. Optimality in deeper level implies optimality in shallower level, i.e., $(n = |S| - 2)$ -optimality implies $(n = |S| - 1)$ -optimality, and so on. This means that the deeper level of optimality criteria is more selective than the shallower level.

We conclude that the discounted and discounting-free optimality criteria could be bridged with the scheme below.

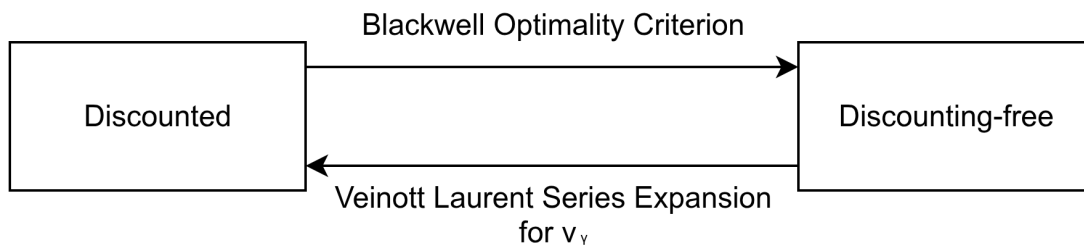


Figure 2.5: Bridging Optimality Criteria

2.4.5 Optimality Criteria For Environment with Transient States

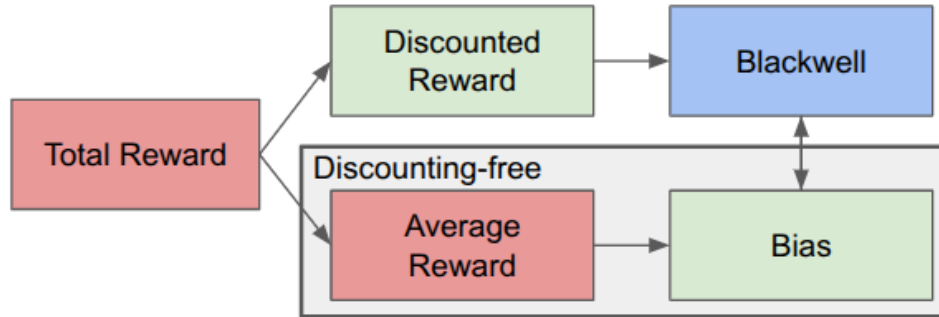


Figure 2.6: Optimality Criteria for Environment with Transient States. Blue: ideal and theoretical optimality criteria. Green: practical optimality criteria. Red: Not considered as it is underselective for environments with transient states.

In summary, in unichain MDPs with transient states, the ideal optimality criteria to use is Blackwell optimality criteria. However, it is hard to find the value of γ_{BW} , as it depends on the MDP characteristics. Therefore, in practice, it will be approached with:

- Discounted scheme: by trying to tune the discount factor γ to be greater than γ_{BW} .
- Discounting-free scheme: by optimizing the average reward and bias, i.e., $(n = -1)$ -optimality and $(n = 0)$ -optimality, respectively, with the assumption that $(n = 0)$ -optimality (bias-optimality) is enough to attain Blackwell optimality.

2.5 Policy Iteration

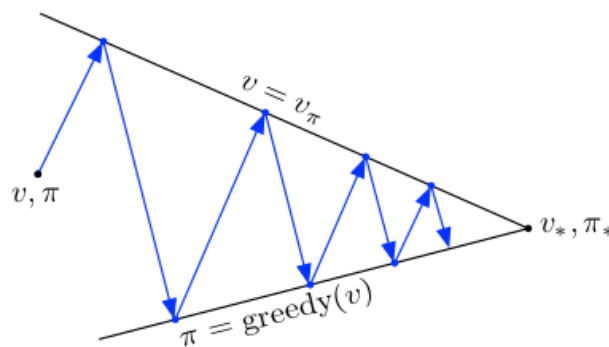


Figure 2.7: Policy Iteration Process (Sutton and Barto, 2018)

Policy iteration is one of the methods to find the optimal policy in an MDP. It consists of two main steps: policy evaluation and policy improvement. Policy evaluation deals with

finding the value of a policy, i.e., given a policy π , what is $v_x(\pi)$? x here refers to the optimality criteria, e.g., total reward, average reward, discounted reward, etc. Meanwhile, policy improvement deals with improving the policy with respect to the policy's value. For example, to select actions that maximize the state-action value on each state, i.e.:

$$\pi'(s) = \arg \max_{a \in \mathbb{A}} q_{\pi}(s, a) \quad (2.9)$$

For an illustration, we will use average reward as the optimality criteria. Policy evaluation can be done by utilizing Bellman Evaluation Equation (BEE) (Bellman, 1957) in the form of:

$$v_g(\pi) + v_b(\pi, s) = \mathbb{E}_{A \sim \pi(\cdot|s)}[r(s, A) + \sum_{s' \in \mathbb{S}} p(s' | s, A) v_b(s')] \quad (2.10)$$

On the other hand, policy improvement can be done by utilizing Bellman Optimality Equation (BOE) in the form of:

$$v_g(\pi^*) + v_b(\pi^*, s) = \max_{a \in \mathbb{A}} [r(s, a) + \sum_{s' \in \mathbb{S}} p(s' | s, a) v_b(\pi^*, s')] \quad (2.11)$$

Both of the process can be done iteratively until the policy converges to the optimal policy π^* .

2.6 Generalized Policy Iteration

In RL setting, the agent do not know the state transition probability p and the reward function r as explained in section 2.3.2. Therefore, for doing policy evaluations or policy improvements, they cannot do it exactly like in 2.5. To get rid of that, the agent need to do approximations, and that's where Generalized Policy Iteration comes in. This enables us to move from DP setting to RL setting, where the agent can learn from the environment by interacting with it.

As the name suggests, Generalized Policy Iteration refers to the generalized version of the Policy Iteration where the computations are done using approximations. For doing inexact policy evaluations, we can define a function approximator, where we can choose several loss functions to be minimized, e.g., :

- Mean squared error e_{MS} , which measures the average squared difference between the approximated value function and the true value function:

$$e_{MS}(\mathbf{w}) = \sum_{s \in \mathcal{S}} p^*(s) (v(s) - \hat{v}(s; \mathbf{w}))^2 \quad (2.12)$$

where $p^*(s)$ is the stationary distribution of the Markov Chain induced by the policy.

- Projected Bellman error e_{PB} or sometimes referred to Least Square Temporal Difference (LSTD) method, which measures the difference between the approximated value function and the Bellman operator applied to it:

$$e_{PB}(\mathbf{w}) = ||(\hat{v}(\mathbf{w}) - \mathbb{P}\hat{v}(\mathbf{w}))||^2 \quad (2.13)$$

The function approximator could also be classified into parametric or non-parametric. Parametric ones have a fixed and finite number of parameters that keep being adjusted during the learning process to approximate the true value function. It could further be classified into linear and non-linear. Linear parametric function approximator is represented as a linear combination of features, e.g.,

$$\hat{v}(s; \mathbf{w}) = \mathbf{w}^\top \mathbf{f}(s) \approx v(s) \quad (2.14)$$

where $\mathbf{f}(s)$ is the feature vector for state s and $\mathbf{w}$ is the weight vector that we want to optimize.

For inexact policy improvement, the agent can update the policy based on the approximated value function such that the new policy π^{k+1} is better than the previous one, i.e., $v(\pi^{k+1}) > v(\pi^k)$.

In RL, this can be done with policy gradient 2.7, specifically utilizing the concept of gradient ascent method to keep adjusting the policy to maximize its value. Remember that gradient is having role to guide below components so that the maximization of the objective function can be reached as soon as possible:

- Direction - where should the parameter update go
- Magnitude - Not only the step size α , the gradient is also having the role to determine how big the step that should be taken for the parameter update. The bigger the gradient

norm, the more sensitive the objective function w.r.t the change of the gradient. In other words, the bigger the norm of the gradient, the bigger the step, and vice versa.

Concretely, to do the inexact policy improvement, we could:

1. Find the gradient of the policy's value w.r.t the policy, i.e., $\nabla_{\pi} v_x(\pi^k) = \frac{\partial}{\partial \pi} v_x(\pi^k)$.
2. Use the gradient ascent method to update the policy:

$$\pi^{k+1} \leftarrow \pi^k + \alpha \nabla_{\pi} v_x(\pi^k), \quad (2.15)$$

However, at the above method, we use direct policy parameterization, where the gradient is computed directly w.r.t the policy. This method is not practical to be used in RL setting, because when the state and action space is large, then the parameter needed to represent the policy is also large (each of the state-action pair needs to be represented by a separate parameter). This makes the computation infeasible, and thus the optimization hard to be done.

As a remedy, we can use explicit policy parameterization, where we define a parameter vector $\theta \in \Theta = \mathbb{R}^{dim(\theta)}$, so that it becomes the parameter for the policy:

$$\pi(\theta) = \pi(a \mid s; \theta) \quad (2.16)$$

This brings several benefits:

- We can use high-dimensional representation for the policy, e.g., using neural networks or other flexible approximators for $\pi(\theta)$ allows us to represent complex policies that can generalize well across large state spaces.
- Explicit policy parameterizations can naturally incorporate an exploration mechanism (similar in spirit to ϵ -greedy).

Many of the proven empirical success, State-of-The-Art (SOTA), and best practices in RL comes from this explicit policy parameterization method, e.g., PPO (Schulman et al., 2017). The policy gradient that we will discuss in the next section is also based on this explicit policy parameterization.

One example for explicit policy parameterization with discrete action space is the softmax

policy, where the probability of taking action a in state s is given by:

$$\pi(a | s; \theta) = \frac{\exp(\theta^\top \phi(s, a))}{\sum_{a' \in \mathcal{A}} \exp(\theta^\top \phi(s, a'))}, \quad \forall (s, a) \in \mathcal{S} \times \mathcal{A} \quad (2.17)$$

where $\phi(s, a)$ is the feature vector for state-action pair (s, a) , and θ is the parameter vector that we want to optimize. The softmax function ensures that the probabilities for each action is positive and will sum to 1, making it a valid probability distribution over actions given a state.

2.7 Policy Gradient

Policy gradient is one of the derivation of Generalized Policy Iteration. There are two variants of policy gradient based on its update. One involves the maximization of gain difference $\max(v_g - v_g^k)$, where v_g corresponds to a gain from other policy whilst v_g^k corresponds to the gain from current policy. This variant leads to TRPO (Schulman et al., 2015), which then developed further into state-of-the-art RL algorithm, which is PPO (Schulman et al., 2017). The other variant of policy gradient, which we will be our focus in this research, is the one which uses gradient of the policy's value $\nabla_{v_g}(\pi, s; \theta)$ for its update, i.e.:

$$\theta^{k+1} \leftarrow \theta^k + \alpha C^{-1} \nabla_{\theta} v_x(\theta) \big|_{\theta=\theta^k} \quad (2.18)$$

where:

- k is the iteration index
- θ^k is the parameter vector at iteration k .
- α is the (positive) step size or learning rate.
- $\nabla_{\theta} v_x(\theta)$ is the gradient of the value function with respect to the parameter vector θ under some optimality criterion x (e.g., discounted reward, average reward, total reward)
- θ^{k+1} is the updated parameter vector after applying the gradient ascent step
- C^{-1} is the preconditioning matrix, which is used to scale the gradient. Based on the chosen C , policy gradients could be classified into vanilla policy gradient and natural policy gradient. Vanilla policy gradient uses identity matrix (I), whilst natural policy gradient uses Fisher x-action information matrix (Φ_{xa}), where x refers to gain (g), bias (b), or gamma (γ).

The above scheme makes policy gradient method an optimization process, where it seeks to maximize the expected return by iteratively adjusting the policy parameters in the direction of the gradient of the policy's value.

To make the above scheme work, we need to be able to find the $\nabla_{\theta} v_g(\theta)$. First, let us use the average reward or gain to be the optimality criteria x . Then, we can write the value of the policy as follows (Puterman, 1994):

$$v_g(\theta) = \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p_{\pi(\theta)}^*(s) \cdot \pi(a | s; \theta) \cdot r(s, a) \quad (2.19)$$

where $p_{\pi(\theta)}^*(s)$ is the steady state distribution of the Markov Chain induced by the policy $\pi(\theta)$.

Introducing the gradient operator ∇_{θ} at both sides:

$$\nabla_{\theta} v_g(\theta) = \nabla_{\theta} \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p_{\pi(\theta)}^*(s) \cdot \pi(a | s; \theta) \cdot r(s, a) \quad (2.20)$$

Exchanging the order of sum and gradient, applying gradient to θ -dependent terms:

$$\nabla_{\theta} v_g(\theta) = \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} r(s, a) \cdot \nabla_{\theta} \left(p_{\pi(\theta)}^*(s) \cdot \pi(a | s; \theta) \right) \quad (2.21)$$

Using the product rule:

$$\nabla_{\theta} v_g(\theta) = \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} r(s, a) \cdot \left(\nabla_{\theta} p_{\pi(\theta)}^*(s) \cdot \pi(a | s; \theta) + p_{\pi(\theta)}^*(s) \cdot \nabla_{\theta} \pi(a | s; \theta) \right) \quad (2.22)$$

The above equation is not sampling-friendly, because it involves the gradient of the steady state distribution $p_{\pi(\theta)}^*(s)$, which usually unknown in RL setting because it requires knowledge of the MDP.

Now, we can utilize the score function identity, i.e.:

$$\nabla_{\theta} \log \pi(a | s; \theta) = \frac{\nabla_{\theta} \pi(a | s; \theta)}{\pi(a | s; \theta)} \quad (2.23)$$

Using the above identity, we can rewrite the gradient of the policy as:

$$\nabla_{\theta} \pi(a | s; \theta) = \pi(a | s; \theta) \cdot \nabla_{\theta} \log \pi(a | s; \theta) \quad (2.24)$$

Utilizing the score function identity, we can rewrite the gradient as:

$$\nabla_{\theta} v_g(\theta) = \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} r(s, a) \cdot \left(p_{\pi(\theta)}^*(s) \cdot \nabla_{\theta} \log p_{\pi(\theta)}^*(s) \cdot \pi(a | s; \theta) + p_{\pi(\theta)}^*(s) \cdot \pi(a | s; \theta) \cdot \nabla_{\theta} \log \pi(a | s; \theta) \right) \quad (2.25)$$

$$\nabla_{\theta} v_g(\theta) = \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p_{\pi(\theta)}^*(s) \cdot \pi(a | s; \theta) \cdot r(s, a) \cdot \left(\nabla_{\theta} \log p_{\pi(\theta)}^*(s) + \nabla_{\theta} \log \pi(a | s; \theta) \right) \quad (2.26)$$

Note that the $p_{\pi(\theta)}^*(s)$ that appears as a multiplicative term is not problematic as it can be handled under expectation. While this form is sampling-friendly, however, in RL setting the $\nabla_{\theta} \log p_{\pi(\theta)}^*(s)$ is unknown to the agent and difficult to compute.

To get rid of the above problem, Sutton et al. (1999) proposed that we can encode the $\nabla_{\theta} \log p_{\pi(\theta)}^*(s)$ and $r(s, a)$, which are unknown to the agent, into something that can be known, i.e., the state-action value function $q_x^{\pi}(s, a)$, where it is in a way combines the state transition probability p and value function v_x , thus giving benefits in terms of computation as we don't need to estimate them separately:

$$\nabla_{\theta} v_g(\theta) = \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} p_{\pi(\theta)}^*(s) \cdot \pi(a | s; \theta) \cdot q_b^{\pi}(s, a) \cdot \nabla_{\theta} \log \pi(a | s; \theta) \quad (2.27)$$

$$= \mathbb{E}_{S \sim p_{\pi(\theta)}^*(\cdot), A \sim \pi(\cdot | S; \theta)} [q_b^{\pi}(S, A) \cdot \nabla_{\theta} \log \pi(A | S; \theta)] \quad (2.28)$$

Recall that expectation is equivalent to mean μ , thus we can get an unbiased approximation of the gradient by using sample mean $\hat{\mu}$ (which is also a random variable) and the expectation of it is equal to the mean.

That $p_{\pi(\theta)}^*$ is unknown but appears under expectation. Hence, in practice, we simply

sample states from it. But it is only available when $t \geq t_{mix}$. Before that, we sample from $p_{\pi(\theta)}^t$, which result in a biased approximation and thus called Semi Stochastic Gradient Ascent. When the sampling is obtained from the true distribution $p_{\pi(\theta)}^*$, the approximation is unbiased, thus we can call it as Stochastic Gradient Ascent.

Theoretically, this method is guaranteed to converge to a local optimum under standard stochastic approximation conditions (Sutton et al., 1999), e.g., the policy is differentiable and the gradient is unbiased, the step size α must satisfy the Robbins-Monro conditions (Robbins and Monro, 1951), i.e., $\sum_{k=1}^{\infty} \alpha_k = \infty$ and $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$, etc.

However, in practice it is not always working (converge to global optimum). The reason is that it has high variance, thus affecting the policy update towards local optima that is not the optimal region. Second, the step size, which usually becomes a hyperparameter, must be appropriately chosen because if it's too small, the convergence will be slow, and if it's too big, it will overshoot the optimal region and diverge. There are other reasons such as the value function might be non-convex, limited samples, the choice of exploration strategy, policy parameterizations, etc. that can affect the optimization process.

To reduce variance, the well-known way to do so is to introduce a baseline (Weaver and Tao, 2013; Williams, 1992). The baseline is a function that is independent of the action taken. For example, in discounting-free policy gradient, the baseline could be the state bias value starting from initial state $v_b(s_0)$, while in discounted policy gradient, the baseline could be the discounted state value $v_Y(s)$. However, there is also recent findings that shows in Natural Policy Gradient, the main role of baseline is to control the aggressiveness of the policy update rather than bounding the variance (Mei et al., 2023).

As different optimality criteria induces different formula for its value function, it also induces different formula for its policy gradients. For average reward policy gradient, the state is sampled from the stationary state distribution p_{π}^* . Meanwhile, for discounted policy gradient, the state must be sampled from normalized discounted state visitation distribution $\tilde{p}_{\pi}^Y$. This can be seen in the following derivation (Sutton et al., 1999):

$$\nabla_{\theta} v_Y(\pi, s_0) = \sum_{s \in \mathcal{S}} \underbrace{\mathbf{P}_{\pi}^Y(s | s_0)}_{\text{not a distribution}} \sum_{a \in \mathcal{A}} q_Y^{\pi}(s, a) \nabla_{\theta} \log \pi(a | s; \theta). \quad (2.29)$$

$\mathbf{P}_\pi^\gamma(s | s_0)$ is not a proper distribution over states, because it does not sum to one and thus not sampling-friendly:

$$\sum_{s \in \mathcal{S}} \mathbf{P}_\pi^\gamma(s | s_0) = \sum_{s \in \mathcal{S}} \sum_{t=0}^{\infty} \gamma^t P_\pi^t(s | s_0) = \sum_{t=0}^{\infty} \gamma^t \underbrace{\sum_{s \in \mathcal{S}} P_\pi^t(s | s_0)}_{=1} = \sum_{t=0}^{\infty} \gamma^t = \frac{1}{1-\gamma} \neq 1. \quad (2.30)$$

We therefore multiply both sides by $(1 - \gamma)$, yielding

$$(1 - \gamma) \nabla_{\theta} v_\gamma(\pi, s_0) = \sum_{s \in \mathcal{S}} \underbrace{(1 - \gamma) \mathbf{P}_\pi^\gamma(s | s_0)}_{\tilde{p}_\pi^\gamma(s | s_0)} \sum_{a \in \mathcal{A}} q_\gamma^\pi(s, a) \nabla_{\theta} \log \pi(a | s; \theta). \quad (2.31)$$

By definition,

$$\tilde{p}_\pi^\gamma(s | s_0) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t P_\pi^t(s | s_0), \quad \sum_s \tilde{p}_\pi^\gamma(s | s_0) = 1, \quad (2.32)$$

so $\tilde{p}_\pi^\gamma(s | s_0)$ is a proper distribution over states.

The consequence of the above is that for discounted policy gradient, the maximum time step for each experiment episode should also follow geometric distribution, i.e., $T_{max} \sim \text{Geo}(1 - \gamma)$, and properly we should sample only from the last time step of each experiment episode.

2.8 Hyperparameter Tuning

For using discounted reward optimality criteria, one of the hyperparameter is the discount factor (γ). But many research papers did not really tune it, and use a fixed gamma value instead. For example, the tax optimization with RL (Zheng et al., 2021), in-hand robot dexterous learning (OpenAI et al., 2018), and others related to theoretical policy gradients (Lehmann, 2024; Mnih et al., 2016). Tuning in RL is challenging due to the stochasticity of the environment and the high computational cost of evaluating policies. Several strategies for tuning include:

- Grid search, where we test a predefined range of values for each hyperparameter.
- Random search, where we sample randomly from a defined distribution.

- Population-Based Training (PBT), where we simultaneously train a population of agents with different hyperparameters and dynamically adjust them based on performance (Jaderberg et al., 2017).

2.9 Sample Complexity

In realistic settings, getting samples might be costly, and the more samples we use, the more expensive the computation will be. Therefore, ideally we want to use as few samples as possible to get the optimal policy. In the context of RL, sample complexity measures how many samples needed for the agent to be able to get into some level of performance. The more samples needed, the higher the sample complexity, and vice versa. Theoretically, we can define sample complexity as the number of samples needed to achieve an ϵ -optimal policy with high probability (e.g., $1 - \delta$) (Kakade, 2003). Formally, for a policy π , the goal is to find π such that:

$$v_x(\pi, s) \geq v_x(\pi^*, s) - \epsilon, \quad \forall s \in \mathcal{S},$$

where v_x is the value function under optimality criterion x (e.g., discounted or average reward), and π^* is the optimal policy. The sample complexity depends on:

- State and Action Space Size: In finite MDPs, sample complexity scales with $|\mathcal{S}|$ and $|\mathcal{A}|$. For large or continuous spaces, function approximation reduces sample complexity by generalizing across states.
- Horizon and Discount Factor: In discounted settings, the effective horizon $\frac{1}{1-\gamma}$ influences sample complexity (Dewanto and Gallagher, 2021a). A smaller γ reduces the horizon, lowering sample requirements but potentially sacrificing long-term optimality.
- Algorithm Efficiency: Model-free algorithms (e.g., Q-learning, PPO) typically have higher sample complexity than model-based algorithms, which learn a model of the environment to simulate transitions (Sutton and Barto, 2018).

CHAPTER 3

RELATED WORKS

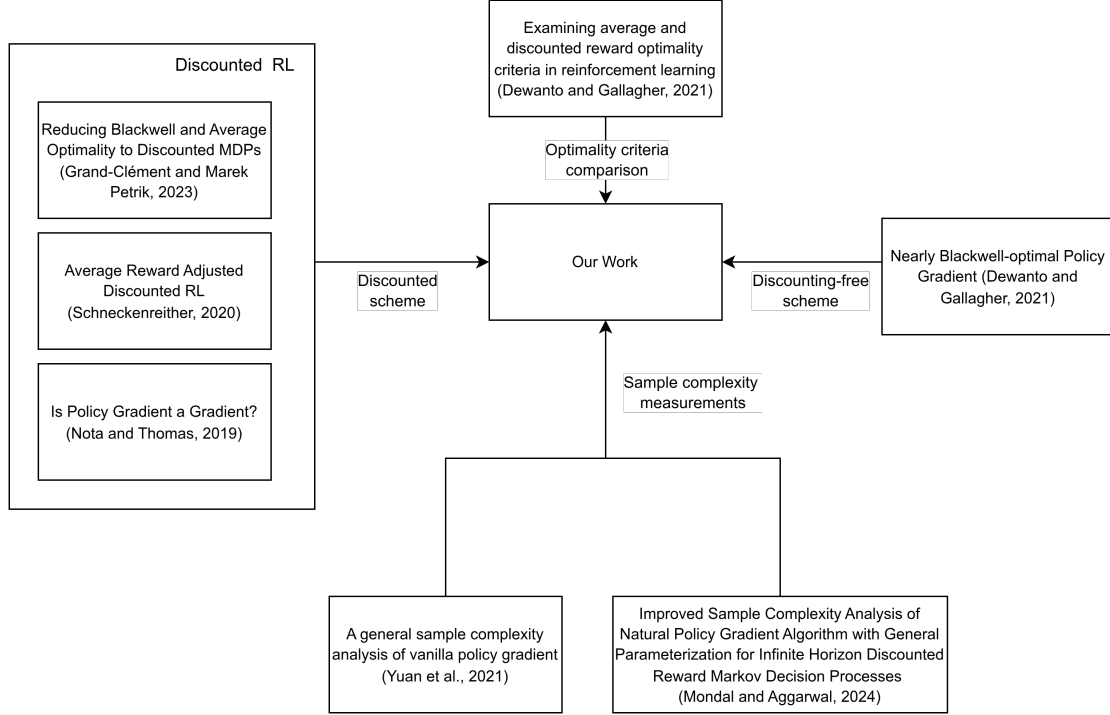


Figure 3.1: Related works and Our Research Position

3.1 Optimality Criteria Comparison

We are taking advantage of the work from (Dewanto and Gallagher, 2021a) that give insights on the comparison between optimality criteria, mainly discounted and average reward. The optimality criteria will be the objective basis for the RL algorithms schemes that will be compared in the research. From the work, we conclude that discounting-free RL might result in a more sample efficient approach to attain Blackwell optimality criteria through Veinott’s Laurent series expansion (Veinott, 1969) as it eliminates the complications coming from artificial discounting that present in discounted schemes. More insights on optimality criteria has been reviewed in 2.4.

3.2 Discounting-free Scheme

(Dewanto and Gallagher, 2021b) is one of the research that ours was built on top of. They propose one of the first nearly-Blackwell optimal (bias-optimal) discounting-free policy

gradient. Their approach addresses a limitation of standard policy gradient algorithms, which typically maximize solely the average reward objective, without considering bias value or transient performance. The method optimizes bi-level objectives, i.e., gain then bias.

Key contributions of this work include deriving the gradient estimation formulas for the bias objective and designing an algorithm that solve the bi-level optimization problem. Second, they introduce a logarithmic barrier term that keeps the policy updates within the gain-optimal region, whenever it is in the phase of optimizing the bias. The resulting algorithm is a form of natural policy gradient ascent that runs the average-reward policy gradient updates, then activate the bias gradient updates with the barrier constraint.

3.3 Discounted Scheme

(Schneckenreither, 2020) proposed an algorithm called Average Reward Adjusted Discounted Reinforcement Learning. It aims to achieve nearly Blackwell-optimal policies by adjusting the standard discounted RL framework. However, it has a drawback because of the need of two discount factors as hyperparameters, i.e., (γ_1, γ_2) . Their work is relevant to our research, as they approach the Blackwell optimality criteria by considering the discounting-free behavior while still leveraging the convergence guarantees by the discounted framework.

Key contributions of the paper include separating the average reward from the value estimates and uses a gain-adjusted value function. Concretely, the method continually estimates the average reward per step v_g and then centers the rewards by subtracting the corresponding estimate. This yields adjusted rewards $r'(s, a) = r(s, a) - v_g$, which have an expected value of zero in steady-state. A discounted value function computed with these adjusted rewards will converge to a bias value, i.e., $v_b(\pi, s)$ rather than blowing up as γ approaching 1. The Laurent series expansion could provide the theoretical justification, i.e., by subtracting the current average reward estimate, the leading $\frac{1}{(1-\gamma)}$ term is removed, and the algorithm will focus on the bias term. This is actually related to the concept of reward centering (Naik et al., 2024). (Wan et al., 2025) studied the effectiveness of the concept with TD-based RL algorithm in continuing tasks.

Whilst the previous work propose a new algorithm to tackle Blackwell optimality criterion,

(Grand-Clément and Petrik, 2023) take a more theoretical approach. Their work gives insight that emphasize potential inefficiency of naive approaches, i.e., if the chosen γ is set slightly below the threshold, the resulting optimal policy could be suboptimal in the long run. Conversely, setting γ close to 1 makes the learning process slower. The condition gives dilemma in terms of sample complexity in discounted framework.

Key contributions of the paper includes the introduction of the Blackwell discount factor γ_{BW} , which is the smallest discount factor such that any discounted optimal policy for $\gamma > \gamma_{BW}$ is also Blackwell-optimal. This is a shift from the classical definition of Blackwell optimality, which defines a threshold $\gamma(\pi)$ but it actually not guarantees that all discounted optimal policies for $\gamma > \gamma(\pi)$ are Blackwell-optimal. The paper introduce a counterexample under certain value function schemes. Additionally, the work introduce an approach to compute the upper bound of γ_{BW} based on the MDP model. That will be useful for getting a value of $\gamma = \gamma_{BW} + \delta$ for a small δ , which could give a guarantee of Blackwell optimality criteria by solving the MDP with discounted framework.

3.4 Sample Complexity Measurements

In terms of sample complexity measurements, our work is related with (Yuan et al., 2021) which analyze the general sample complexity for vanilla policy gradients, and (Mondal and Aggarwal, 2024) for the natural policy gradient. Those two papers give insights on how to compute the sample complexity analytically. In this work, we only consider the empirical sample complexity, but additional experiments could lead to some hypothesis on the analytical form. However, their work consider only discounted scheme, while we also consider its discounting-free counterparts.

CHAPTER 4

RESEARCH METHODOLOGY

To answer the primary research questions, we'll make two brackets, each of which corresponds to the effective policy gradients according to the optimality criteria (first is for discounted, second is for discounting-free). From those brackets, we'll extract the best method from each bracket and compare between the two. From all environment, we'll see in average, which scheme has the better sample complexity. The one which have the most wins will be considered as more sample-efficient. In more detail, the procedure is as follows.

4.1 Finding Bias-optimal Policies As Benchmark

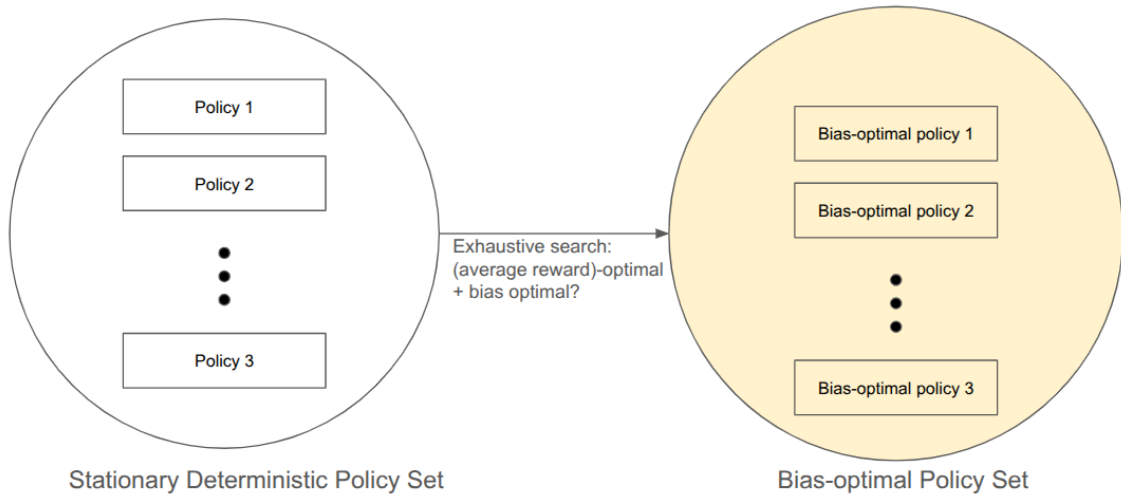


Figure 4.1: Phase 1 - Finding bias-optimal policies

In this phase, we find the bias-optimal policies by doing exhaustive search on the stationary deterministic policy set Π_{SD} . Because the environment used will be synthetic and relatively small. The maximum number of actions per state is two, and the biggest environment only has 7 states, which in that case there will be $2^7 = 128$ stationary deterministic policies. The bias-optimal policies that obtained will be a reference or benchmark against the resulting policy from each method in phase 2.

4.2 Finding Effective Methods

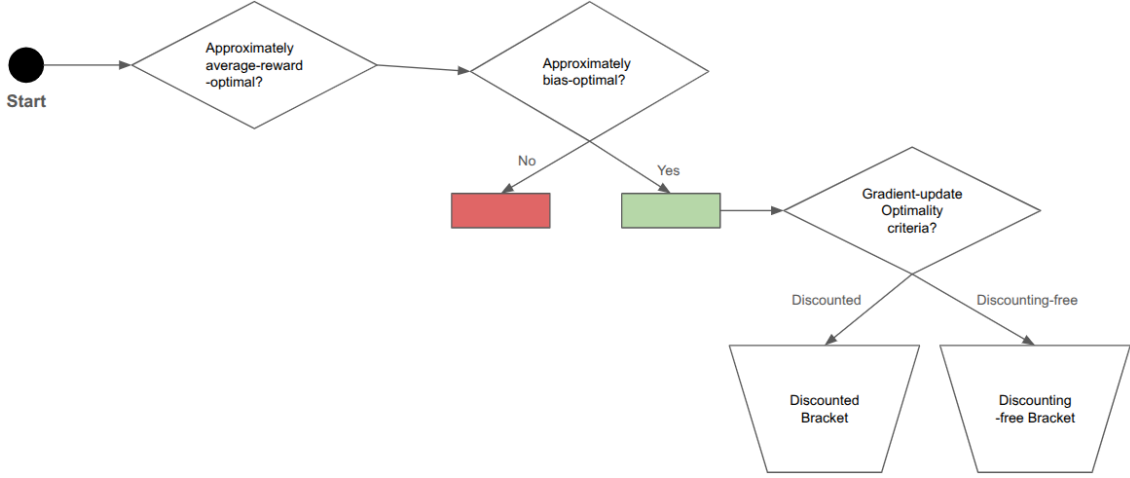


Figure 4.2: Phase 2 - Finding the effective methods

In this phase, in each of the six environments, we find the effective algorithms. Effective here refer to the condition where the converged policy is approximately bias-optimal, which in our case, assumed to be Blackwell-optimal. The condition of convergence here refers to the condition where the gradient of the policy's value is less than a threshold, i.e., ϵ . To check it, we do two layers of checking. First, we check that whether the converged policy is in the gain-optimal region, as it is a requirement for bias-optimal policy. It will be done by checking the policy's final average reward, that at least it is less than 1 from the bias-optimal policy's average reward. Second, if the converged policy is indeed still in the gain-optimal region, then we check its bias w.r.t the bias-optimal policy obtained from the first phase. For bias, we consider a policy to be approximately bias-optimal if it is at least having a bias valued 50% from the reference policy. Concretely, we formulate the effectiveness value as follows:

$$bias_{eff} = 1 - \frac{v_b^* - v_b}{|v_b^*|} \quad (4.1)$$

where v_b^* is the bias-optimal policy's bias, and v_b is the converged policy's bias. Bias-effective methods will have $bias_{eff} \geq 0.5$, where those will be flagged by green color and put into the corresponding bracket based on its optimality criteria, whilst ineffective methods will be flagged by red color. We consider that effectiveness should be the prerequisite before sample complexity considerations, as the sample complexity is only meaningful if the method is effective.

4.3 Making Sure Each Bracket Has a Representative to Compare

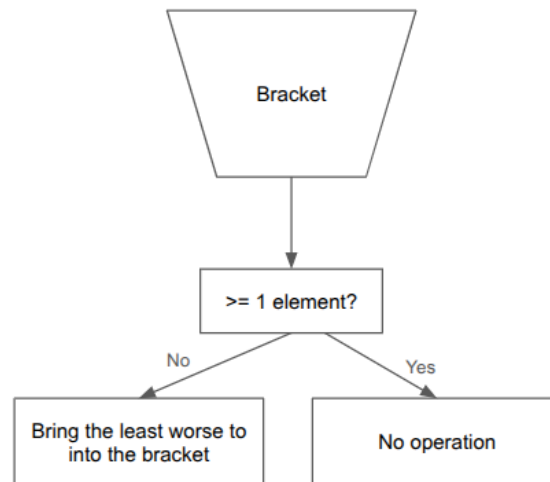


Figure 4.3: Phase 3 - Making sure each bracket has a representative to compare

As previously explained, we prioritize the effectiveness of the methods before sample complexity. Therefore, if in a particular bracket there is no effective methods whilst the other bracket has effective ones, then we will consider the latter to be more sample-efficient. Nevertheless, we will still compare it with the least bad method in the no-effective methods bracket just to observe the concrete number of samples needed to converge between the two brackets. If both brackets does not have effective methods, then we will compare the least bad method in each bracket.

4.4 Finding The Most Sample-efficient Methods From Each Bracket

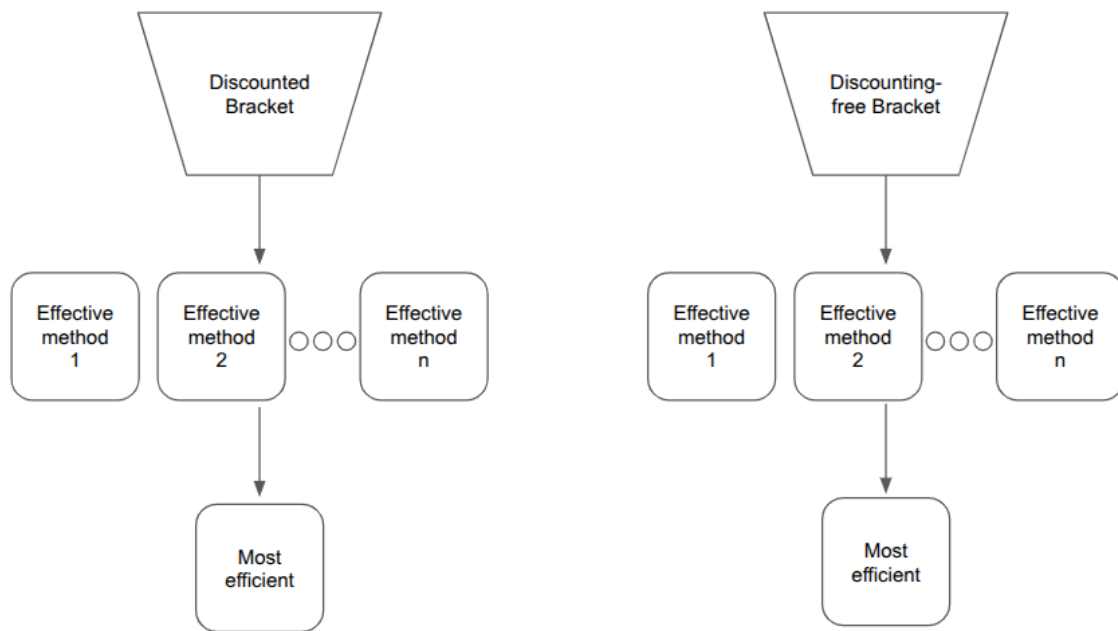


Figure 4.4: Phase 4 - Finding the most sample-efficient policy gradient from each bracket

Each of the brackets might have more than one method that's effective. We'll find the best policy gradient based on its complexity to be the bracket's final representative.

4.5 Finding The Most Sample-efficient Scheme Between Each Bracket Per Environment

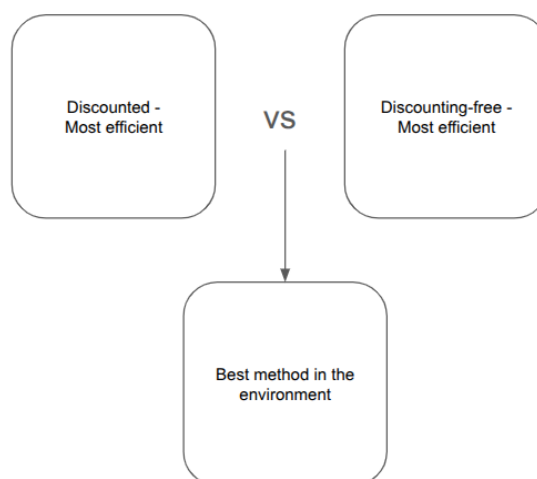


Figure 4.5: Phase 5 - Compare which one has the best sample complexity between each bracket

In this phase, we'll compare which one between discounted and discounting-free scheme yields the most efficient result in terms of sample complexity in each environment.

4.6 Finding The Most Sample-efficient Scheme Between Each Bracket On Average

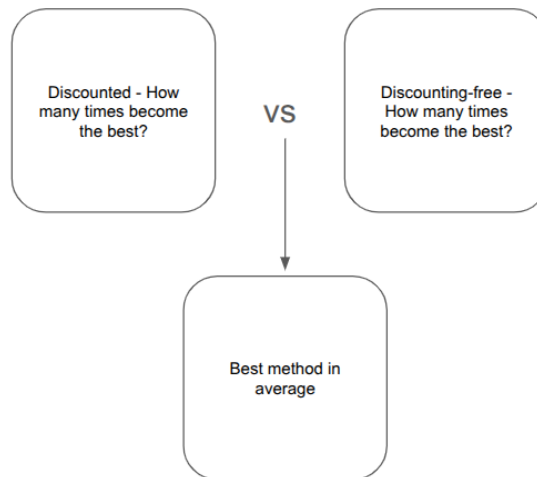


Figure 4.6: Phase 6 - Compare which one has the best sample complexity (on average)

Finally, each scheme wins from the previous phase will be counted and compared. The one with the most frequent wins will be considered as more sample efficient approach.

CHAPTER 5

EXPERIMENT SETUP

In this chapter, we first introduce the environments that will be used, describe our sigmoid-based policy parameterization, then move to the algorithms and the hyperparameter optimization. Finally, we visualize each environment's policy value landscape to provide initial insights into the corresponding optimization objectives. The full code implementations for every environment, algorithm, and visualization are available at <https://github.com/ArdhitoN/NBWPG-SampleComplexity>.

5.1 Environment Design

The environment that will be used for the experiments is unichain MDPs with one or more transient states. There are two major variants of the environments, i.e., with constant average reward (variant A) and various average reward (variant B). For each, there will be three different environments used. Environments are coming from (Dewanto and Gallagher, 2021b), where some of them are also adapted from (Hordijk et al., 1985) for its Taxi Loop in environment B2 and (Strens, 2000) for its Chain loop in environment B3. The figure of each of the environments can be seen below.

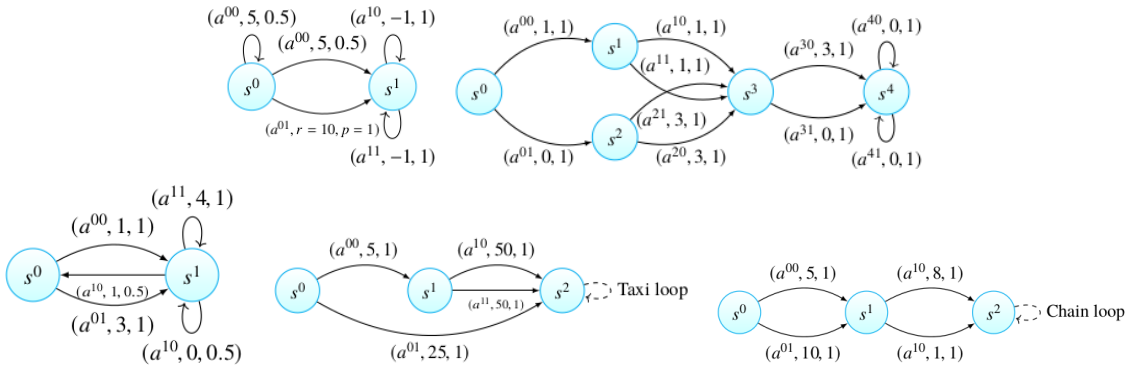


Figure 5.1: Environments (first row): A1, A2. Environments (second row): B1, B2, B3. Figures are obtained from (Dewanto and Gallagher, 2021b), where environment A3 is not plotted because it is too complex and unintuitive to show. Each transition is represented by an arrow. In each of the arrow, there is tuple consisting of three elements, which consecutively corresponds to the type of the action, the immediate reward, and the probability of the transition.

For environment A3, it is a 2-by-2 grid-world, so there are 4 cells, where each cell represents a state. The start state s_0 is at the bottom-left, s_1 is at the bottom-right, s_2 is at

the top-left, and the final recurrent state (or goal state) s_3 is at the top-right. In each state, there are 2 actions available. Concretely, at s_0 , the agent can move to s_1 with action a^{s_0} , or to s_2 with action a^{s_1} . At s_1 , the agent can move to s_0 with action a^{s_0} , or to s_3 with action a^{s_1} . At s_2 , the agent can move to s_3 with action a^{s_0} , or to s_0 with action a^{s_1} . Finally, at the final recurrent state s_3 , the agent will stay there with either of the actions, i.e., a^{s_0} or a^{s_1} . Any movement effort to adjacent grid cells will have a slip probability of 0.1 going to the other direction and incurs a cost of -1.0 reward. In final recurrent state, however, any of the 2 actions there will not incur transition to other states neither cost (the reward will be 0.0).

In environment with variant A, there is only one recurrent state. Therefore, in stationary condition, the agent will end up in that state. The case is different for environment with variant B. In B1, there are possibly one or more recurrent states. Therefore, the agent can move between those recurrent states. With that difference, environments with variant A will have constant average reward across all policies, while environments with variant B will have varying average rewards, as we can see later in more detailed in 5.7.

The similarity for all environments is that there are the same possible number of actions possible in each state, i.e., 2. Therefore, the number of stationary deterministic policy $|\Pi_{SD}|$ is $2^{|S|}$. By design, we choose this setting because we want to be able to plot the policy's value landscape in a 2D space, where the x-axis is the first action's parameter and the y-axis is the second action's parameter. Also, this makes the number of states is small enough to be able to do exhaustive search on the stationary deterministic policy set Π_{SD} to get the bias-optimal or nearly-Blackwell optimal policies.

The order in each of the variant is based on the complexity of the environment. In variant B, the bigger the serial number, more states are introduced, leading to a more complex state space. In variant A, whilst A3 has fewer states than A2, the transitions are more complex, leading to a more complex environment. Concretely, in A2, all transitions are deterministic, i.e., the probability of each transition is 1.0. Meanwhile, in A3, there is always a slip probability of 0.1 for each transition except at the final recurrent state.

5.2 Policy Parameterizations

The policy is parameterized with sigmoid function, following the specification given at (Dewanto and Gallagher, 2021b). Sigmoid-based parameterization is chosen because at each of the states in each of the environments, there are only two possible actions, i.e., $\{a^{s0}, a^{s1}\}$. Recall that sigmoid function is a function that maps a real number to the range of $[0, 1]$. Therefore, it can be used to represent the probability of taking an action a^{s0} in state s . Meanwhile, the complementary probability $1 - \sigma(x)$ represents the probability of taking the other action.

Concretely, with the policy parameterization and state feature function as input, the policy with sigmoid parameterization is expressed as:

$$\pi(a^{s0} | s; \theta) = \sigma(f(s)\theta_0 + \theta_1), \pi(a^{s1} | s; \theta) = 1 - \pi(a^{s0} | s; \theta), \forall s \in \mathbb{S}, \forall \theta \in \Theta \quad (5.1)$$

where:

- $\sigma(x) = \frac{1}{1 + \exp(-x)}$ is the sigmoid function
- $f(s) \in 0, 1, \dots, |S| - 1$ is the state feature function which return the index of a state s

5.3 Exact Policy Gradient

Exact policy gradients will be employed where the policy evaluations and policy improvements are done exactly. In the exact setting, we give the agent full knowledge of the MDP's transition probabilities and reward function. This allows for both the average-reward and discounted-reward objectives exact computations, as well as their gradients without any sampling noise. Doing this can give a clear baseline for the convergence behavior, where we observe how each method moves through the true value landscape, without variance that exist if there are approximations. The exact variant that will be employed will be the same as the sampling-based variant (except we do not consider the sampling scheme), where based on its gradient type, we'll consider vanilla policy gradient and natural policy gradient. With respect to the optimality criteria, we will consider the discounting-free and discounted based policy gradients.

5.4 Policy Gradient

After previous exact setting, we will do policy gradient with inexact way by sampling for the policy improvements, i.e., to get approximations on the gradients (and the Fisher information if natural policy gradient is used). Meanwhile, the policy evaluations will still be done exactly. The variants of the policy gradient based on the optimality criteria and the type of the gradient will be the same as in the exact methods, except that in discounted policy gradient setting, there will be additional type based on sampling:

1. Proper sampling: where the gradient update will be based on the proper distribution of the problem, i.e., only last samples from each of the experiment episodes, where the length of each experiment episodes will be based on geometric distribution with parameter $1 - \gamma$, i.e., $T_{max} \sim Geo(1 - \gamma)$. This comes from the formulation of gradient for discounted policy gradient:

$$\nabla J_{\gamma}(\theta) = \mathbb{E}_{A_t \sim \pi(\cdot|s), S_t \sim p(\cdot|s_t, a_t)} \left[\sum_{t=0}^{\infty} \gamma^t \nabla \log \pi(S_t, A_t) a_{\gamma}^{\theta}(S_t, A_t) \mid \theta \right] \quad (5.2)$$

2. Popular sampling: where the samples used for the gradient update will come from the whole trajectory experienced by the agent over the experiment episodes, and the length of the experiment episodes will be a hyperparameter and kept fixed. This comes from the formulation of popular discounted policy gradient that drops the discount factor γ :

$$\nabla J_{\gamma}(\theta) = \mathbb{E}_{A_t \sim \pi(\cdot|s), S_t \sim p(\cdot|s_t, a_t)} \left[\sum_{t=0}^{\infty} \nabla \log \pi(S_t, A_t) a_{\gamma}^{\theta}(S_t, A_t) \mid \theta \right] \quad (5.3)$$

The popular sampling leads to a biased gradient, as the discount factor γ is not taken into account (Nota and Thomas, 2019).

5.4.1 Discounting-free Policy Gradient

Algorithm 1 Discounting-Free Policy Gradient

```

1 Input:
  • env: Environment
  • a policy parameter  $\theta$ 
  • convergence thresholds for average reward  $\epsilon_g$ , and bias  $\epsilon_b$ 
  • The barrier parameter  $\beta$ , and its divisor  $\beta_{\text{div}}$ 
  • variant  $\in \{\text{vanilla}, \text{natural}\}$ 
  • sampling : use sampling (True) or exact (False) scheme
  • outer_iters : maximum number of outer optimization iteration
  • inner_outers : maximum number of inner optimization iteration
  •  $n_{\text{ep}}$  : number of experiment episodes for sampling-based policy improvement
  •  $t_{\text{max}}$  : maximum time step for each experiment episodes

2 Init:
  • avg_reward_conv  $\leftarrow$  False
  •  $\delta \leftarrow 1$ 
  • max_avg_reward  $\leftarrow -\infty$ 
  •  $\tilde{\beta} \leftarrow 0$ 
  • samples  $\leftarrow 0$ 

3 for  $i = 0$  to  $\text{outer\_iters} - 1$  do
4   for  $j = 0$  to  $\text{inner\_iters} - 1$  do
5      $v_g \leftarrow \text{compute\_avg\_reward}(\theta)$ 
6      $Q_b \leftarrow \text{compute\_Qb}(\theta)$ 
7      $V_b \leftarrow \text{compute\_bias}(\theta)$ 
8      $s_0 \leftarrow \text{env.initial\_state}$ 
9     if sampling then
10       $P_\pi, r_\pi \leftarrow \text{compute\_transitions\_and\_reward}()$ 
11       $P_\pi^* \leftarrow \text{pstar}(P_\pi)$ 
12       $t_{\text{max}} \leftarrow \text{mixing\_time}(P_\pi, P_\pi^*) + 5$ 
13       $(\nabla_g, \nabla_b, \Phi_{ga}, \Phi_{ba}) \leftarrow \text{est\_grads\_fishers\_avg}(Q, V_b, n_{\text{ep}}, t_{\text{max}})$  ▷ Alg. 2
14      samples  $\leftarrow$  samples +  $n_{\text{ep}} \times t_{\text{max}}$ 
15    else
16       $\nabla_g \leftarrow \text{grad\_avg\_reward}(\theta), \nabla_b \leftarrow \text{grad\_bias}(\theta, s_0)$ 
17       $\Phi_{ga}, \Phi_{ba} \leftarrow \text{variant} = \text{natural} ? (\text{fisher\_avg\_reward}(), \text{fisher\_bias}(s_0)) : \text{None}$ 
18    end if
19    if avg_reward_conv then
20       $\tilde{\beta} \leftarrow \beta / (v_g - \text{max\_avg\_reward} + \delta)$ 
21       $\nabla_v \leftarrow \nabla_b + \tilde{\beta} \cdot \nabla_g$ 
22       $C \leftarrow \text{variant} = \text{natural} ? (\Phi_{ba} + \tilde{\beta} \cdot \Phi_{ga} + (\tilde{\beta}^2 / \beta) \cdot \nabla_g \nabla_g^T + 10^{-6} \cdot I) : I$ 
23    else
24       $\nabla_v \leftarrow \nabla_g, C \leftarrow \text{variant} = \text{natural} ? (\Phi_{ga} + 10^{-6} \cdot I) : I$ 
25    end if
26    norm  $\leftarrow \|\nabla_v\|_2$ 
27    if not avg_reward_conv and norm  $< \epsilon_g$  then
28      max_avg_reward  $\leftarrow v_g$ 
29      avg_reward_conv  $\leftarrow$  True
30      avg_reward_samples  $\leftarrow$  samples
31      continue
32    else if avg_reward_conv and norm  $< \epsilon_b$  then
33      total_samples  $\leftarrow$  samples, break
34    end if
35     $\Delta\theta \leftarrow C^{-1} \cdot \nabla_v$  ▷ Pseudo-inverse if singular
36    Compute  $\alpha$  with backtracking linesearch ascent
37     $\theta \leftarrow \theta + \alpha \cdot \Delta\theta$ , update_policy( $\theta$ )
38  end for
39  if avg_reward_conv then
40     $\beta \leftarrow \beta / \beta_{\text{div}}$ 
41  end if
42  if total_samples is not None then
43    break
44  end if
45 end for
46 Return:  $\pi(\theta)$ , total_samples, avg_reward_samples

```

Adapted from (Dewanto and Gallagher, 2021b), where we add modifications to use baseline to reduce the variance, hence using advantage value $a_x^\pi(s, a) = q_x^\pi(s, a) - v_x^\pi(s)$ rather than just state-action value Q , the process can be summarized as follows:

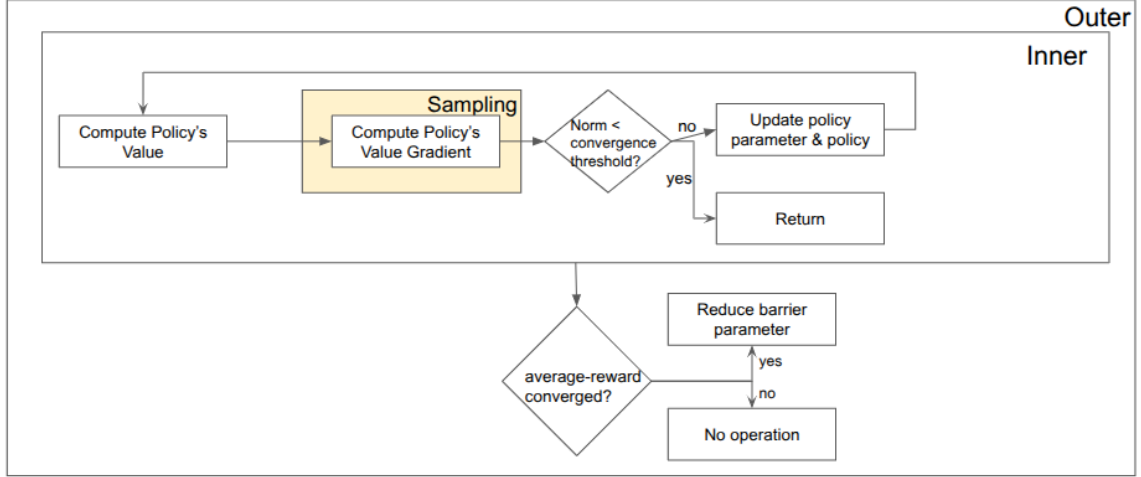


Figure 5.2: Policy gradient for discounting-free scheme

(a) **Inner Iterations**

- (1) Compute the oracles q_b and v_b for the current policy, to get $a_b = q_b(s, a) - v_b(s)$.
- (2) Compute the components for policy update:
 - (•) Gradients of average reward ($\nabla_g = \nabla_\theta v_g$) and bias ($\nabla_b = \nabla_\theta v_b$). In exact form, these are:

$$\nabla_\theta v_g = \mathbb{E}_{S \sim p_{\pi(\theta)}^*(\cdot), A \sim \pi(\cdot | S; \theta)} [a_b^\pi(S, A) \cdot \nabla_\theta \log \pi(A | S; \theta)] \quad (5.4)$$

$$\begin{aligned} \nabla v_b(\theta, s_0) &= \sum_{t=0}^{t_{mix}^\pi - 1} \mathbb{E}_{S_t \sim p_\pi^t(\cdot | s_0), A_t \sim \pi(\cdot | S_t; \theta)} [a_b^\pi(S_t, A_t) \nabla \log \pi(A_t | S_t; \theta)] - \nabla v_g(\theta) \\ &\quad + \mathbb{E}_{S_{mix}^\pi \sim p_\pi^*(\cdot), A_{mix}^\pi \sim \pi(\cdot | S_{mix}^\pi; \theta)} [\nabla a_b^\pi(S_{mix}^\pi, A_{mix}^\pi)] + \nabla v_g(\theta). \end{aligned} \quad (5.5)$$

- (•) Fisher information matrices for gain-action Φ_{ga} and bias-action Φ_{ba} (if using the natural variant):

$$\Phi_a(\theta, s) = \sum_{a \in \mathbb{A}} \pi(a | s; \theta) \nabla_\theta \log \pi(a | s; \theta) \nabla_\theta \log \pi(a | s; \theta)^T \quad (5.6)$$

$$\Phi_{ga} = \sum_{s \in \mathbb{S}} p_{\pi(\theta)}^*(s) \Phi_a(\theta, s) \quad (5.7)$$

$$\tilde{\Phi}_{ba}^{t_{abs:min}^{\pi, s_0}}(\theta, s_0) = \sum_{t=0}^{t_{abs:min}^{\pi, s_0}-1} \sum_{s \in \mathbb{S}} p_{\pi}^t(s | s_0) \Phi_a(\theta, s) + t_{abs:min}^{\pi, s_0} \cdot \sum_{s \in \mathbb{S}} p_{\pi}^*(s) \Phi_a(\theta, s) \quad (5.8)$$

- (3) Form the combined policy's value gradient ∇_v and preconditioning matrix C , where ∇_g will be weighted by the barrier parameter. Formally, we define the combined policy's value gradient as:

$$\nabla_v = \nabla_{v_b}(\theta, s_0) + \tilde{\beta} \nabla_{v_g}(\theta) \quad (5.9)$$

where:

$$\tilde{\beta} = \frac{\beta}{v_g - v_g^* + \delta} \quad (5.10)$$

- (4) Check convergence: if $\|\nabla_v\|$ is below the threshold (ϵ_g or ϵ_b), skip the update and proceed to next phase (bias phase if the current is average reward phase, and stop if current is bias phase).
- (5) Otherwise, update policy parameters:
- (•) Compute update direction $\Delta\theta = C^{-1} \nabla_v$
 - (•) Find step size α via backtracking line search.
 - (•) Apply update: $\theta \leftarrow \theta + \alpha \Delta\theta$.

(b) Additional Step in Bias Optimization Phase

- (1) For computing the combined policy's value gradient, compare current average reward v_g to the best so far:
- (•) If within threshold of average reward optimality, downweight ∇_g to focus on bias.
 - (•) Otherwise, upweight ∇_g to continue average reward improvement.

(c) Outer Iterations

- (1) Run the inner iteration loop.
- (2) If average reward already converged, reduce the barrier parameter to make the bias optimization more aggressive: $\beta \leftarrow \beta / \beta_{\text{div}}$.
- (3) Otherwise, keep β unchanged.

Algorithm 2 Estimate Gradients and Fishers for Average Reward

```

1: Input:
    •  $Q_b$ : State-action bias value
    •  $V_b$ : Bias value
    •  $n_{\text{ep}}$ : number of experiment episodes for sampling-based policy improvement
    •  $t_{\text{max}}$ : maximum time step for each experiment episodes
2: Init:
    •  $\text{dim}_{\theta} \leftarrow |\theta|$ 
    •  $t_{\text{abs.min}} \leftarrow 2$ 
    • lists:  $\text{grad}_g, \text{grad}_b, \Phi_g, \Phi_b$ 
3: for  $e = 0$  to  $n_{\text{ep}} - 1$  do
4:    $s \leftarrow \text{env.reset}(), \text{traj} \leftarrow \emptyset$ 
5:    $\nabla_g, \nabla_b \leftarrow \mathbf{0}_{\text{dim}_{\theta}}$ 
6:    $\Phi_{ga}, \Phi_{ba} \leftarrow \mathbf{0}_{\text{dim}_{\theta} \times \text{dim}_{\theta}}$ 
7:   for  $t = 0$  to  $t_{\text{max}} - 1$  do
8:      $a \leftarrow \text{select\_action}(s)$ 
9:      $(s', r) \leftarrow \text{env.step}(a)$ 
10:    Append  $(s, a)$  to traj
11:     $s \leftarrow s'$ 
12:   end for
13:    $\nabla \log \pi \leftarrow \text{compute\_grad\_log\_pi}(\theta)$ 
14:   for  $(s, a)$  at time  $t$  in traj do
15:      $A \leftarrow Q_b[s, a] - V_b[s]$ 
16:      $z \leftarrow A \cdot \nabla \log \pi[s, a]$ 
17:      $X \leftarrow \nabla \log \pi[s, a] \nabla \log \pi[s, a]^T$ 
18:     if  $t = t_{\text{max}} - 1$  then
19:        $\nabla_g \leftarrow z$ 
20:        $\nabla_b \leftarrow z - (t_{\text{max}} - 1) \cdot \nabla_g$ 
21:        $\Phi_{ga} \leftarrow X$ 
22:        $\Phi_{ba} \leftarrow t_{\text{abs.min}} \cdot X$ 
23:     else
24:        $\nabla_b \leftarrow \nabla_b + z$ 
25:       if  $t < t_{\text{abs.min}}$  then
26:          $\Phi_{ba} \leftarrow \Phi_{ba} + X$ 
27:       end if
28:     end if
29:   end for
30:   Append  $\nabla_g, \nabla_b, \Phi_{ga}, \Phi_{ba}$  to lists
31: end for
32: Return:  $\text{mean}(\text{grad}_g), \text{mean}(\text{grad}_b), \text{mean}(\Phi_g), \text{mean}(\Phi_b)$ 

```

The sampling process can be summarized as follows:

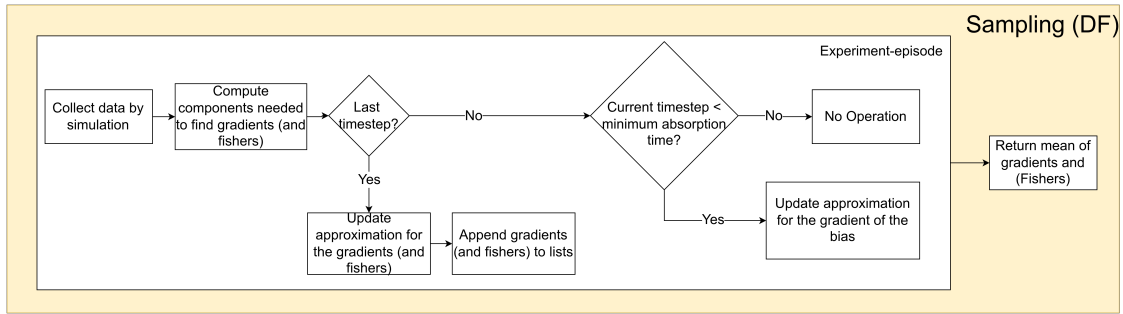


Figure 5.3: Sampling process for discounting-free scheme

• Generalized Procedure

- Perform n_{ep} experiment episodes to approximate gradients and Fisher information, where the length of each episode is determined by the maximum time step t_{max} which obtained from mixing time calculations in the outer policy gradient algorithm 1. In each experiment episodes:
 - **Data collection:** Simulate agent-environment interactions under current policy to gather samples.
 - **Compute components:**
 - Log-policy gradient $\nabla_{\theta} \log \pi(a|s; \theta)$.
 - Advantage values using oracles Q_b and V_b .
 - Sample gradient of average reward (z): advantage $\times$ log-policy gradient.
 - Sample for action-agnostic Fisher information (X).
 - **Per-time step updates:**
 - If not last time step: Accumulate bias-gradient approximation by adding current average-reward gradient. If $t < t_{\text{min:abs}}$ (absorption threshold), accumulate bias-action Fisher information.
 - If last time step: Finalize approximations for average-reward gradient, bias gradient, Fisher of average-reward, and Fisher of bias. Append each approximation to its list.
- Return the mean estimates of:
 - Gradient of average reward,
 - Gradient of bias,
 - Fisher information for average reward,
 - Fisher information for bias.

In discounting-free scheme, for each sampling run, the number of samples will be the number of experiment episodes multiplied by the maximum time step, which comes from mixing time computations.

5.4.2 Discounted Policy Gradient

Algorithm 3 Discounted Policy Gradient

1: **Input:**

- env: Environment
- a policy parameter θ
- Discount factor γ
- convergence threshold ϵ_γ
- variant $\in \{\text{vanilla}, \text{natural}\}$
- sampling : use sampling (True) or exact (False) scheme
- iters : maximum number of optimization iteration
- method $\in \{\text{proper}, \text{popular}\}$
- n_{ep} : number of experiment episodes for sampling-based policy improvement
- t_{max} : maximum time step for each experiment episodes

2: **Init:**

- samples $\leftarrow 0$

3: **for** $i = 0$ **to** $\text{iters} - 1$ **do**

4: $Q_\gamma \leftarrow \text{compute_Q}(\theta)$

5: $V_\gamma \leftarrow \text{compute_full_discounted_value}(\theta)$

6: **if** sampling **then**

7: $(\nabla_v^\gamma, \Phi_\gamma, n) \leftarrow \text{est_grads_fishers_disc}(Q_\gamma, V_\gamma, n_{\text{ep}}, \text{method} = \text{popular} ? t_{\text{max}} : \text{None}, \text{method}) \triangleright \text{Alg. 4}$

8: samples $\leftarrow \text{samples} + n$

9: **else**

10: $\nabla_v^\gamma \leftarrow \text{grad_v_gamma}(\theta)$

11: $\Phi_\gamma \leftarrow \text{variant} = \text{natural} ? \text{fisher_gamma}() : \text{None}$

12: **end if**

13: $v_\gamma \leftarrow \text{discounted_value}(\theta)$

14: norm $\leftarrow \|\nabla_v^\gamma\|_2$

15: **if** norm $< \epsilon_\gamma$ **then**

16: total_samples $\leftarrow \text{samples}$

17: **return** $\pi(\theta)$, total_samples

18: **end if**

19: $C \leftarrow \text{variant} = \text{natural} ? (\Phi_\gamma + 10^{-6} \cdot I) : I$

20: $\Delta\theta \leftarrow C^{-1} \cdot \nabla_v^\gamma$

21: Compute α with backtracking linesearch ascent

22: $\theta \leftarrow \theta + \alpha \cdot \Delta\theta$, update_policy(θ)

23: **end for**

The process can be summarized as follows:

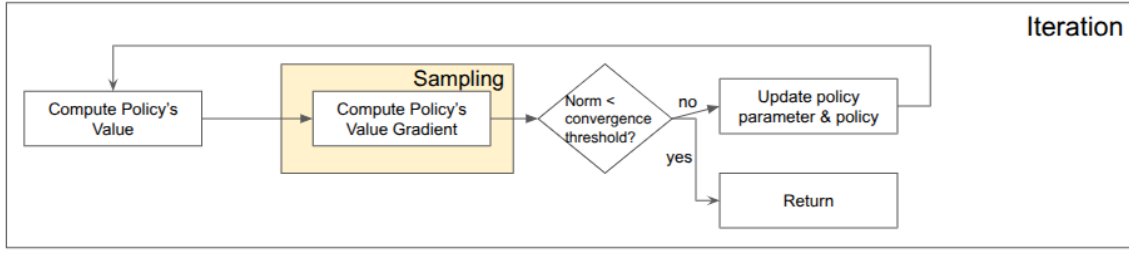


Figure 5.4: Policy gradient for discounted scheme

- **Generalized Procedure**

- Compute the discounted-reward oracles q_γ and v_γ for the current policy, then find $a_\gamma(s, a) = q_\gamma(s, a) - v_\gamma(s)$.
- Compute the update components:
 - Gradient of the discounted value with respect to the policy parameters, adapted from 2.7:

$$\nabla_{\theta} v_\gamma(\pi, s_0) = \frac{1}{(1-\gamma)} \sum_{s \in \mathcal{S}} \underbrace{(1-\gamma) \mathbf{P}_\pi^\gamma(s | s_0)}_{\tilde{p}_\pi^\gamma(s | s_0)} \sum_{a \in \mathcal{A}} a_\gamma^\pi(s, a) \nabla_{\theta} \log \pi(a | s; \theta). \quad (5.11)$$

- Fisher information matrix (if using the natural variant):

$$\Phi_\gamma(\theta, s) = \sum_s \tilde{p}_\pi^\gamma(s | s_0) \Phi_a(\theta, s) \quad (5.12)$$

- Check convergence: if $\|\nabla v_\gamma\|$ is below the threshold, terminate without updating.
- Otherwise, update policy parameters:
 - Compute update direction $\Delta\theta = C^{-1} \nabla v_\gamma$
 - Determine step size α via backtracking line-search.
 - Apply update: $\theta \leftarrow \theta + \alpha \Delta\theta$.

Algorithm 4 Estimate Gradients and Fishers for Discounted Reward

1: **Input:**

- Q_γ : State-action (discounted) value
- V_γ : Discounted value
- n_{ep} : number of experiment episodes for sampling-based policy improvement
- t_{max} : maximum time step for each experiment episodes
- method: Sampling method, either popular or proper

2: **Init:**

- $\text{dim}_\theta \leftarrow |\theta|$
- lists: $\text{grad}_\gamma, \Phi_\gamma$
- $\text{samples} \leftarrow 0$

3: **for** $e = 0$ **to** $n_{\text{ep}} - 1$ **do**

4: $s \leftarrow \text{env.reset}()$

5: $\text{traj} \leftarrow \emptyset$

6: $\nabla_\gamma \leftarrow \mathbf{0}_{\text{dim}_\theta}$

7: $\Phi_\gamma \leftarrow \mathbf{0}_{\text{dim}_\theta \times \text{dim}_\theta}$

8: **if** method = proper **then**

9: $T \leftarrow \text{geometric}(1 - \gamma)$

10: **else**

11: $T \leftarrow t_{\text{max}}$

12: **end if**

13: **for** $t = 0$ **to** $T - 1$ **do**

14: $a \leftarrow \text{select_action}(s)$

15: $(s', r) \leftarrow \text{env.step}(a)$

16: Append (s, a, t) to traj

17: $s \leftarrow s'$

18: **end for**

19: $\text{samples} \leftarrow \text{samples} + |\text{traj}|$

20: $\nabla \log \pi \leftarrow \text{compute_grad_log_pi}(\theta)$

21: **if** method = proper **then**

22: $(s, a, t) \leftarrow \text{last}(\text{traj})$

23: $A \leftarrow Q_\gamma[s, a] - V_\gamma[s]$

24: $\nabla_\gamma \leftarrow A \cdot \nabla \log \pi[s, a]$

25: $\Phi_\gamma \leftarrow \nabla \log \pi[s, a] \nabla \log \pi[s, a]^T$

26: **else**

27: **for** (s, a, t) in traj **do**

28: $A \leftarrow Q_\gamma[s, a] - V_\gamma[s]$

29: $z \leftarrow A \cdot \nabla \log \pi[s, a]$

30: $\nabla_\gamma \leftarrow \nabla_\gamma + z$

31: $\Phi_\gamma \leftarrow \Phi_\gamma + \nabla \log \pi[s, a] \nabla \log \pi[s, a]^T$

32: **end for**

33: **end if**

34: Append $\nabla_\gamma, \Phi_\gamma$ to lists

35: **end for**

36: **Return:** $\text{mean}(\text{grad}_\gamma), \text{mean}(\Phi_\gamma), \text{samples}$

The sampling process can be summarized as follows:

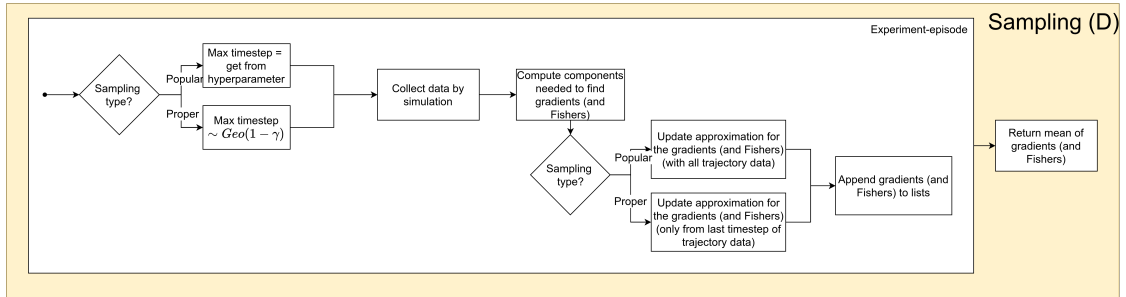


Figure 5.5: Sampling process for discounted scheme

- **Experiment Episodes (n_{ep}):**
 - Determine max time step:
 - Popular: fixed hyperparameter.
 - Proper: sample from $\text{Geo}(1 - \gamma)$.
 - Collect samples by simulating agent-environment interactions under the current policy up to the max time step.
 - Compute approximations: log-policy gradient $\nabla_{\theta} \log \pi(a|s)$; advantage via oracles Q_{γ} , V_{γ} ; sample gradient (advantage $\times$ log-policy gradient); and sample for action-agnostic Fisher information.
 - Select data for estimating gradients and Fisher information:
 - Popular: use all trajectory data.
 - Proper: use only data from the final time step.
 - Append each approximation to its corresponding list.
- Return mean estimates of:
 - Gradient of discounted value,
 - Fisher information for discounted value.

For discounted scheme, for each sampling runs, the total samples will be the the accumulation of each maximum time step at each experiment episodes (which the maximum time step will be constant under popular sampling, but varies under proper sampling, as it is sampled from $\text{Geo}(1 - \gamma)$).

5.5 Algorithm Parameters

For each method, we run the experiment with 100 different seeds (42 - 141) with the initial policy parameterized by a pair of $(\theta_0 = 0, \theta_1 = 0)$, and the tolerance for optimization convergence (ϵ) is set to 10^{-2} . The learning rate or step size α is computed using the backtracking line search ascent method (Boyd and Vandenberghe, 2004), with initial value of 1 and maximum number of iterations of 100. Based on its optimality criteria, some parameters used for the policy gradient methods are different. For discounting-free policy gradients, the parameters are:

- Maximum number of optimization iterations:
 - Outer: 1
 - Inner: 1000
- Initial barrier parameters:
 - β_0 : 0.1 for environments with A variant, and β_0 : 100 for environments with B variant
 - β_{div} : 10
- Number of gradient and fisher samples used, which indicated with number of experiment episode n_{exp} , as per one experiment episode, we get one sample for the corresponding components: 16
- Maximum time step in an experiment episode ($\hat{t}_{\text{max}}^{\text{exp}}$): $t_{\text{mix}} + 5$
- Minimum absorption time $\hat{t}_{\text{abs:min}}$: 2

For discounted policy gradients, the parameters are:

- Maximum number of optimization iterations: 1000
- Number of gradient and fisher samples used, which indicated with number of experiment episode n_{exp} , as per one experiment episode, we get one sample for the corresponding components: 16
- Maximum time step in an experiment episode $\hat{t}_{\text{max}}^{\text{exp}} \sim \text{Geo}(1 - \gamma)$ for proper sampling, and 100 for popular sampling

5.6 Hyperparameter Tuning

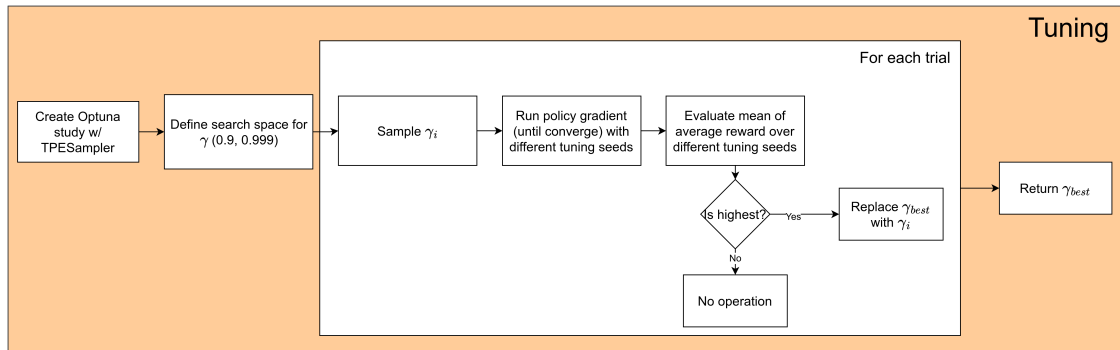


Figure 5.6: Hyperparameter tuning process for discounted scheme to find the best γ

Hyperparameter tuning will be done to find the best discount factor gamma (γ) that gives the highest reward for an evaluation interval over several last episodes. This will be done using one of state-of-the-art tools, i.e., Optuna (Akiba et al., 2019). Optuna is chosen because of its robustness across different functionalities, ease of use (with its define-by-run principle), and efficiency. Initially, we define the search space for the gamma, in our case it will be in the interval $[0.900, 0.999]$. Then, for each trial (there will be 8 trials, means 8 different γ values), we will run the policy gradient algorithm until converged. At each iteration, we will evaluate the policy using average reward as evaluation metric and save it as evaluation history. However, for each tuning seeds (in this experiment we use 10 out of 100 total seeds as tuning seeds), we will only consider the last 10 evaluation history and compute its mean, which will be the one "per-seed" performance score. Then, we will compute the mean of all performance scores across all tuning seeds, which will be the final performance score for that trial. The trial with the highest performance score will be the best trial, and the γ value from that trial will be the best γ for the policy gradient algorithm.

We consider average reward as the evaluation metric because it is the dominant term in discounting-free scenarios. We cant directly add up average reward + bias as there is no guarantee that were already in the (average reward)-optimal landscape. It will be computed approximately in environments with A variant as in those environments, the average reward is constant across the parameter space. However, in environments with variant B, that doesnt happen, so we can reduce the computation by computing it exactly. Samples used in environments with variant A for evaluations is not considered for computing the sample

complexity.

The resulting γ can be observed below.

Environment	D-vanilla-proper	D-vanilla-popular	D-natural-proper	D-natural-popular	D-vanilla-exact	D-natural-exact
A1	0.96	0.99	0.96	0.97	0.94	0.94
A2	0.91	0.97	0.99	0.94	0.99	0.99
A3	0.91	0.99	0.91	0.96	0.97	0.99
B1	0.91	0.94	0.91	0.99	0.99	0.99
B2	0.99	0.94	0.99	0.92	0.94	0.99
B3	0.92	0.94	0.97	0.99	0.99	0.99

Table 5.1: Tuned gamma obtained after hyperparameter tuning

5.7 Value Landscape

Before we jump to the main experiments, we provide a visual exploration of the policy's value landscape for each environment. These landscapes are plotted w.r.t the two policy parameters, where both axes range from -10 to 10, discretized into 41 points. This visualization offers initial insights into the characteristics of the optimization objectives for both discounting-free (average reward then bias value) and discounted value (using the tuned γ) optimality criteria.

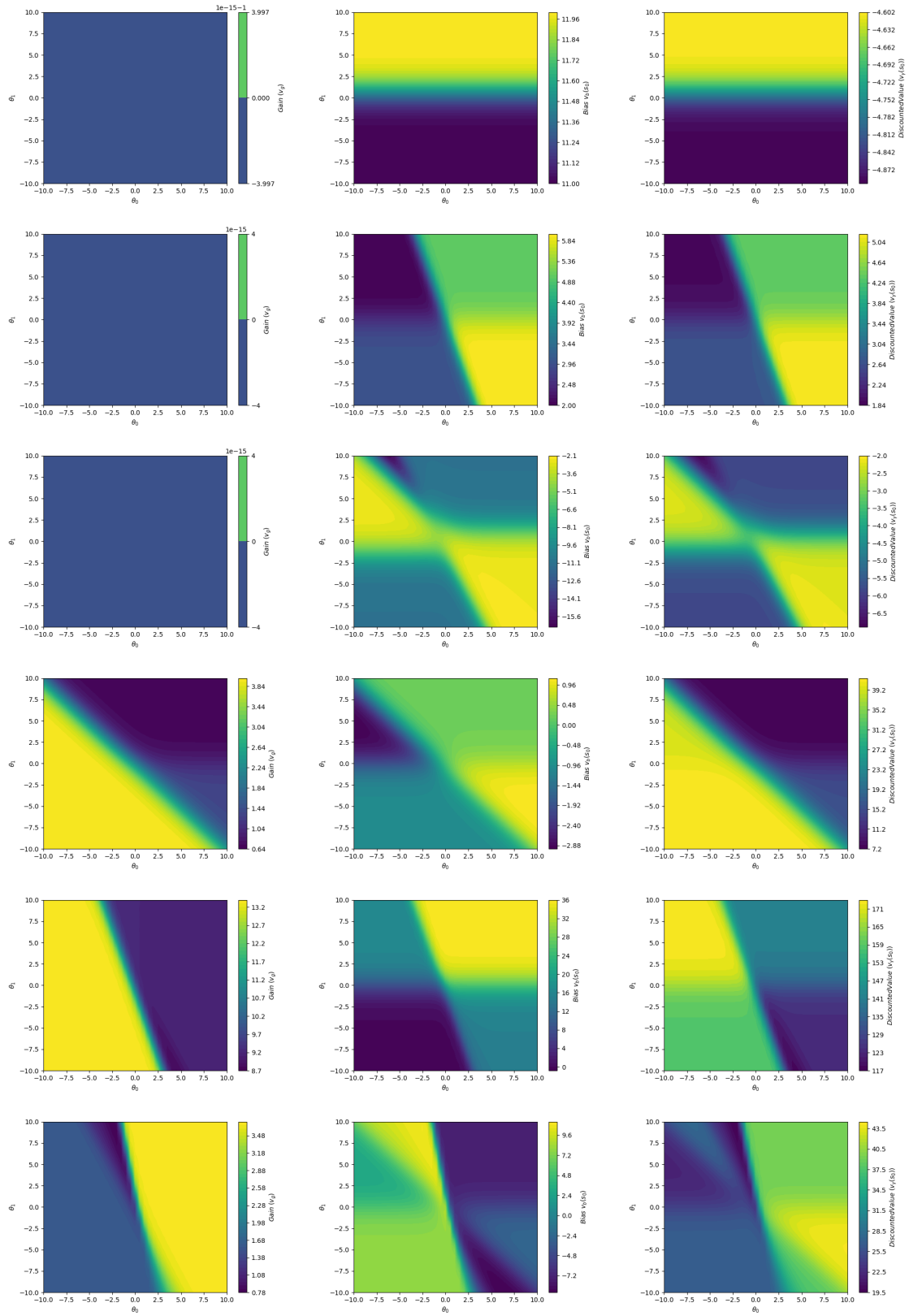


Figure 5.7: Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain landscape, bias landscape, discounted value landscape. For the discounted value landscape, the gamma chosen is the lowest tuned gamma across the methods, where for each environment (top to bottom) consecutively: 0.94, 0.91, 0.91, 0.91, 0.92, 0.92

The above figures depict the policy's value landscape for each of the environments w.r.t each of the optimality criteria. Gain and bias landscape are for discounting-free, while discounted value is for discounted scheme. As the policy parameterizations is using a vector of two components, the value landscape is plotted w.r.t those parameters, where each axis corresponds to one of the parameter vector's component. The resulting color depicts the policy's value w.r.t the parameter values, where brighter color signaling bigger values and vice versa. The numeric value of the policy could be seen at the right of the bar.

From the figures, we can observe that the gain for environments A1, A2, and A3 are constant across the parameter space. It is expected as there is one absorbing terminal state at those environments. At environment A1, it is at state s_1 , where the possible actions $\{a_{10}, a_{11}\}$ leads to the same state with reward -1. At environment A2 and A3, there is zero-reward absorbing terminal state, i.e., s_4 and s_3 respectively.

For environment B1, B2, and B3, the gain landscape filled with different colors at different regions across the parameter space. In environment B1, there is possibly one transient state, i.e., s_0 . In environment B2, there are 3 recurrent states, i.e., the 3-state Taxicab from Hordijk et al. (1985). In environment B3, there are 5 recurrent states, i.e., the 5-state Chain from Strens (2000). As there are more than one alternative at the recurrent states, the gain is not constant across the parameter space. For example, in environment B2, at the recurrent chain, the agent could alternate between states $s_2, s_3, \text{ and } s_4$. Two possible actions (0 and 1) respectively at each state produces different rewards, i.e., in s_3 , it is 15 and 16 compared to 8 and 2.75 at s_2 and 7 and 4 at s_4 . Thus, the gain for the policy that support the agent to go to s_3 more frequently would lead to a higher gain.

We could also observe that the resulting discounted landscape produces a similar landscape to its discounting-free counterpart. For environment with variant A, it resembles the bias value landscape. This happened because the gain is constant, thus the effect of it is also constant across the parameter space at the discounted value landscape. Meanwhile, for environment with variant B, it resembles the combination of the gain and bias landscape, but put the weights more into the gain landscape. The reason of that is because of the bi-level optimization to achieve Blackwell optimality criteria. First thing first, the agent need to go to the gain-optimal region, then optimize the bias value. In other words, to

achieve bias optimality, gain optimality is a requirement.

CHAPTER 6

RESULTS AND ANALYSIS

6.1 Sample Complexity Analysis

To answer the primary and the secondary research question, we follow the methodology given at chapter 4. In addition, we analyze additional insights related to how each algorithm behave.

6.1.1 Bias-optimal Policies

Environment	Average Reward	Bias
A1	-1	12
A2	0	6
A3	0	-2.22
B1	4	-1
B2	13.345	16.367
B3	3.677	-2.461

Table 6.1: Bias-optimal policies in each environment and its value

For each six environments, the bias-optimal policies average reward and bias value can be seen at the table 6.1 above, each of which will be the reference to compare the resulting converged policy from each of policy gradient methods to measure its effectiveness in the next phase.

6.1.2 Effectiveness Measure

For each policy gradient methods (also for discounted-based scheme), we will compute its final average reward and bias, and compare it to the bias-optimal policies. As explained in chapter 4, a method is considered average-reward-effective if its final average-reward is at least having a value less than 1 from the bias-optimal policies. For bias-effectiveness, it is considered effective if its final bias value is at least having a value 50% of the bias-optimal policies.

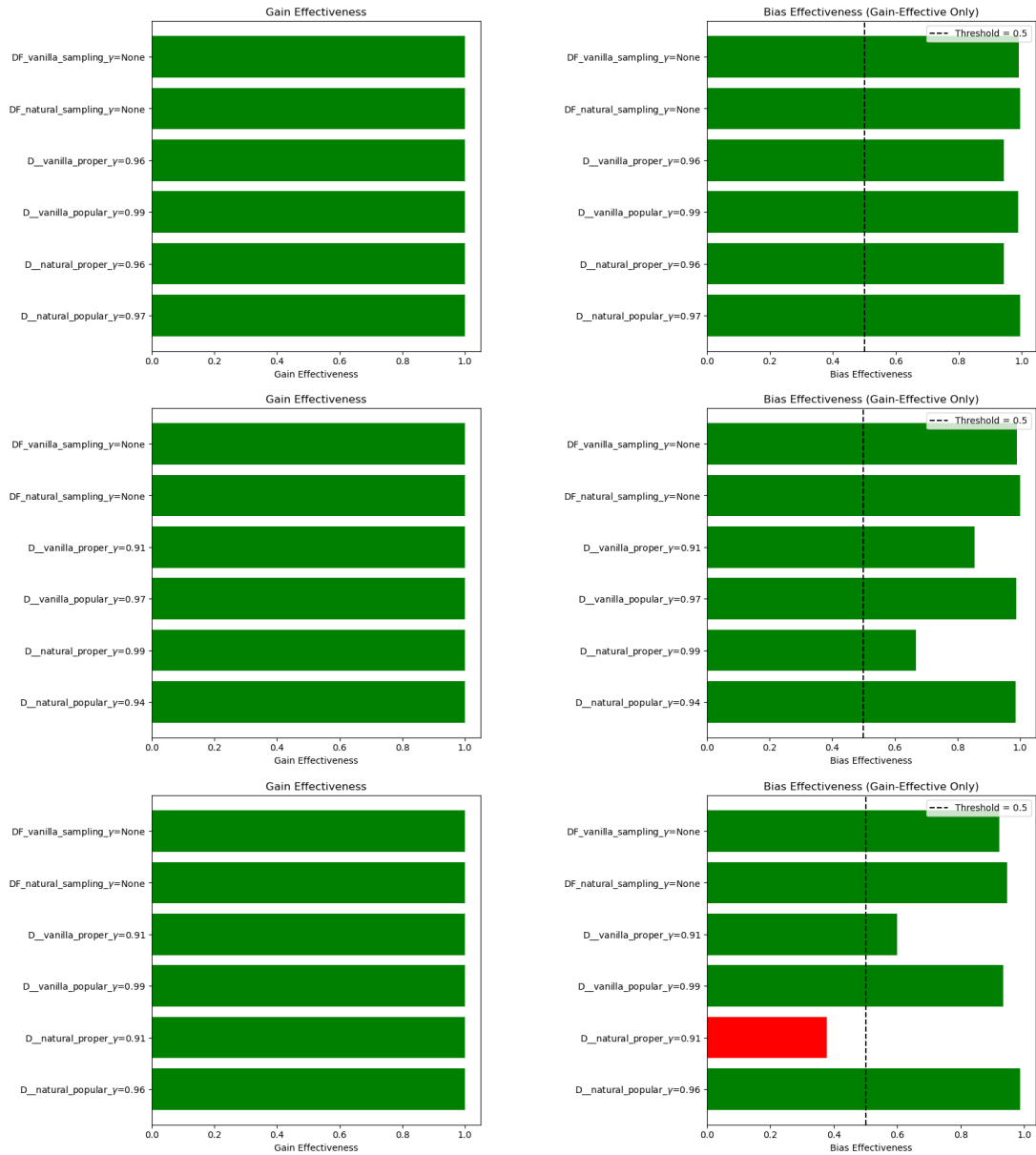


Figure 6.1: Effectiveness measure for each policy gradient methods in environments with A variant. From top to bottom: A1, A2, A3

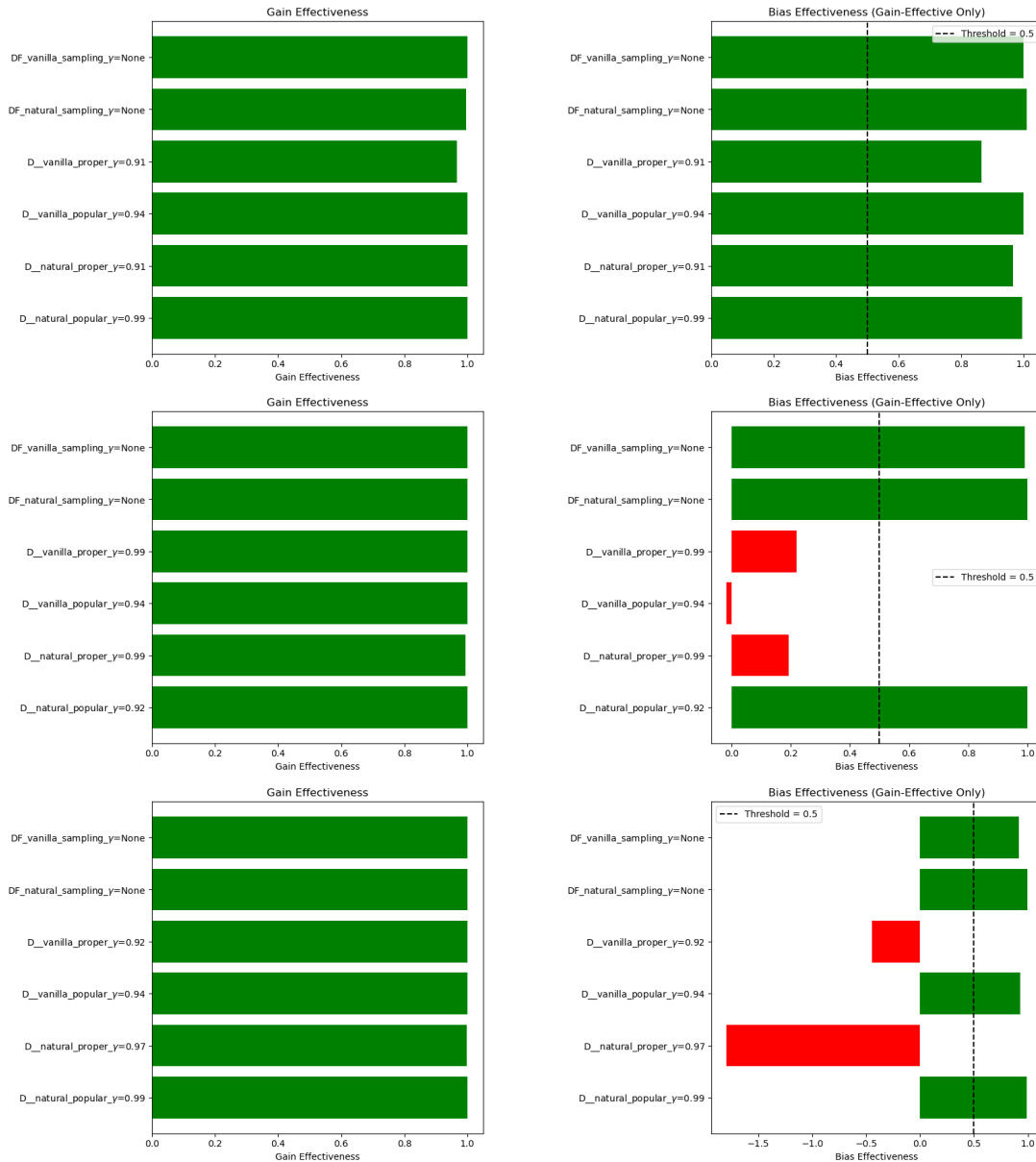


Figure 6.2: Effectiveness measure for each policy gradient methods in environments with B variants.
From top to bottom: B1, B2, B3

As can be seen in the effectiveness measure above, all methods is effective w.r.t average reward (by having average reward difference with the gain-optimal policies of at most 1). However, some discounted policy gradients failed to approximately achieve bias-optimality. There might be several reasons for this. First, the tuned gamma γ is less than γ_{BW} , which makes it suboptimal w.r.t bias-optimality. Thus, the converged policy will only be optimal w.r.t the tuned gamma, but not the most selective criterion, i.e., bias-optimality. Second, the algorithm might fail because of the high variance, as the optimization process

for discounted policy gradient involve the gradient of the discounted value, by which it depends on the v_γ . By the Laurent series expansion, v_γ combines the average reward and bias at the same time, which makes the optimization process more complex compared to the discounting-free, which only focus on a specific term in the Laurent series expansion in each iteration.

The list of effective policy gradients in each environment can be seen in the table below.

Environment	Discounting-free	Discounted
A1	Vanilla, Natural	Vanilla-Popular, Vanilla-Propor, Natural-Popular, Natural-Propor
A2	Vanilla, Natural	Vanilla-Popular, Vanilla-Propor, Natural-Popular, Natural-Propor
A3	Vanilla, Natural	Vanilla-Popular, Vanilla-Propor, Natural-Propor
B1	Vanilla, Natural	Vanilla-Popular, Vanilla-Propor, Natural-Popular, Natural-Propor
B2	Vanilla, Natural	Natural-Popular
B3	Vanilla, Natural	Vanilla-Popular, Natural-Popular

Table 6.2: Effective policy gradients in each environment

As can be seen from the table above, all discounting-free methods are all effective. In contrast, some discounted-based methods are not effective. Therefore, based on the empirical result from six synthetic environments that we are experimenting, discounting-free scheme is more effective than discounted scheme.

6.1.3 Sample Complexity Measure

6.1.3.1 Discounted vs Discounting-free Scheme

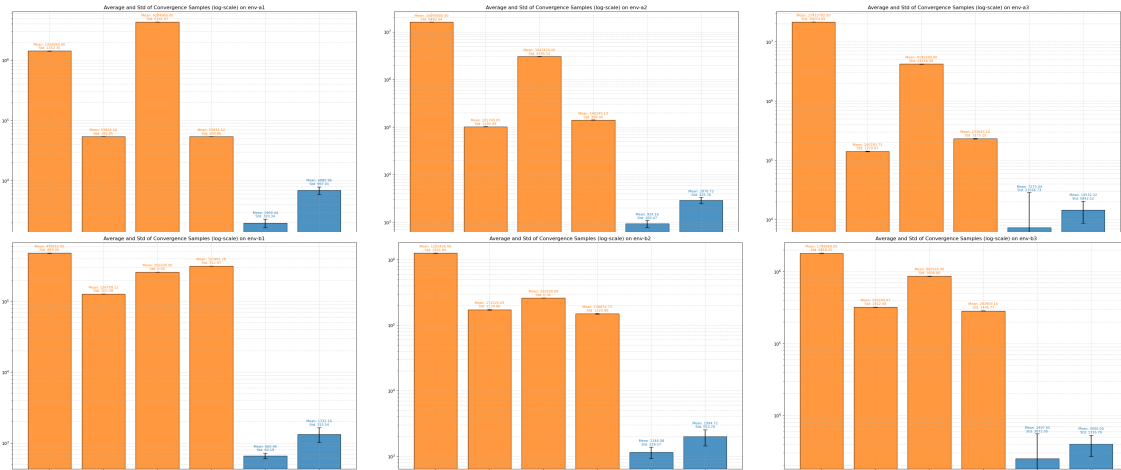


Figure 6.3: Average and standard deviation for empirical samples needed for convergence across each policy gradient methods in each environment. First row: A1, A2, A3. Second row: B1, B2, B3.

Sample complexity here refers to the empirical samples needed for the policy gradient to converge (having the norm of the policy's value gradient less than the convergence threshold). In discounted scheme, the samples used for tuning is also taken into account as explained in chapter 5. As can be observed from the figure 6.3, in total, the discounting-free scheme needs fewer samples than in the discounted scheme across all six environments. This is likely because the need of tuning the discount factor γ in the discounted scheme with configurations explained in 5.6, which is not needed in the discounting-free scheme.

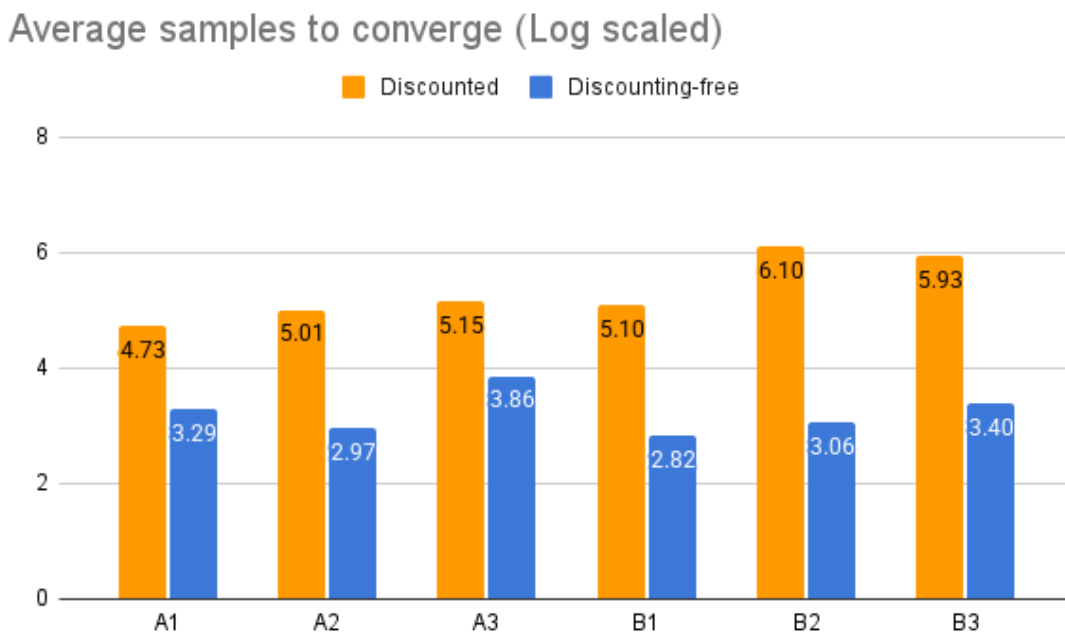


Figure 6.4: Summary - Number of samples needed to achieve convergence between the most sample-efficient method in discounted and discounting-free scheme in each environment

From the figure above, we can see that the gap between the most sample-efficient method from discounted scheme and the most sample-efficient from discounting-free is significant across all environments. The biggest gap for the average samples needed between the two (log-scaled) is at environment B2, i.e., 3.04. Meanwhile, the smallest gap happened in environment A3, i.e., 1.29, which is still significant. This enhancing the belief that discounting-free scheme is not just more effective, but much more efficient. In addition, we can also see that the more complex the environment is (in each variant, the higher the serial number, the more complex it is as explained in 5.1), the more samples needed to achieve convergence.

PG \ Environment	A1	A2	A3	B1	B2	B3
DF-Vanilla	effective	effective	effective	effective	effective	effective
DF-Natural	Most sample-efficient	Most sample-efficient	Most sample-efficient	Most sample-efficient	Most sample-efficient	Most sample-efficient
D-Vanilla-Popular	effective	effective	effective	effective	not effective	effective
D-Vanilla-Proper	effective	effective	effective	effective	not effective	not effective
D-Natural-Popular	effective	effective	not effective	effective	effective	effective
D-Natural-Proper	effective	effective	effective	effective	not effective	not effective

Color and indications:
 Most sample-efficient
 effective
 not effective

Figure 6.5: Summary - Sample Efficiency and Effectiveness across all methods in all environments

In all environments, the most sample efficient method is the discounting-free with natural gradient. It is expected, as natural gradient utilize the Fisher information matrix to find the information related to the curvature of the parameter space. Thus, the gradient update could be better by looking at the policy distribution w.r.t the parameters. A more detailed mapping of average samples needed with the standard deviation can be observed via the table 6.3 below.

Environment	DF-vanilla	DF-natural	D-vanilla-proper	D-vanilla-popular	D-natural-proper	D-natural-popular
A1	6881 $\pm$ 997	1969 $\pm$ 320	53832 $\pm$ 209	4284960 $\pm$ 5241	53824 $\pm$ 193	1418944 $\pm$ 1352
A2	2879 $\pm$ 436	924 $\pm$ 160	140245 $\pm$ 996	3047424 $\pm$ 5596	101749 $\pm$ 1194	16098800 $\pm$ 8483
A3	14532 $\pm$ 5843	7273 $\pm$ 21567	231643 $\pm$ 3175	4195280 $\pm$ 24756	140784 $\pm$ 1280	21415792 $\pm$ 38055
B1	1332 $\pm$ 316	661 $\pm$ 60	315481 $\pm$ 313	259200 $\pm$ 0	126709 $\pm$ 103	479552 $\pm$ 849
B2	1995 $\pm$ 554	1146 $\pm$ 229	148877 $\pm$ 1321	259200 $\pm$ 0	171526 $\pm$ 2140	1255456 $\pm$ 2602
B3	4000 $\pm$ 1317	2498 $\pm$ 3071	283903 $\pm$ 1447	860560 $\pm$ 3609	320204 $\pm$ 1612	1799968 $\pm$ 6818

Table 6.3: Samples needed for convergence in each environment

6.1.3.2 Popular vs Proper Sampling in Discounted Scheme

To compare between popular and proper sampling in discounted scheme, we can utilize the results obtained from 6.1, 6.2, 6.5 for effectiveness, and 6.3 for sample complexity.

PG \ Environment	A1	A2	A3	B1	B2	B3
D-Vanilla-Popular	effective	effective	effective	effective	not effective	Most sample-efficient
D-Vanilla-Proper	effective	effective	effective	effective	not effective	not effective
D-Natural-Popular	effective	effective	not effective	effective	Most sample-efficient	effective
D-Natural-Proper	Most sample-efficient	Most sample-efficient	Most sample-efficient	Most sample-efficient	not effective	not effective

Color and indications:

- Most sample-efficient
- effective
- not effective

Figure 6.6: Summary - Effectiveness for discounted scheme between popular and proper sampling

In the figure 6.6 above, we can infer that in terms of sample complexity, from gradient-type-agnostic perspective, proper sampling is better, as it become the representative for discounted bracket in 4/6 environments. However, in terms of effectiveness, popular sampling is better, where it has representative in all environments. From vanilla-based PG perspective, popular sampling is better in terms of effectiveness, as it is effective in all environments except at B2, whilst proper sampling has two environments where it is not effective, i.e., B2 and B3. From natural-based PG perspective, popular sampling is also better in terms of effectiveness, as it is effective in all environments except at A3, whilst proper sampling has two environments where it is not effective, i.e., B2 and B3. However, in terms of samples needed to converge as can be observed in figure 6.3 and table 6.3, proper sampling is better than popular sampling across all three perspective. In summary, based on three different perspectives, i.e., gradient-type-agnostic, vanilla PG, and natural PG, with sample-complexity as the final objective, proper sampling is better than popular sampling as can be seen below.

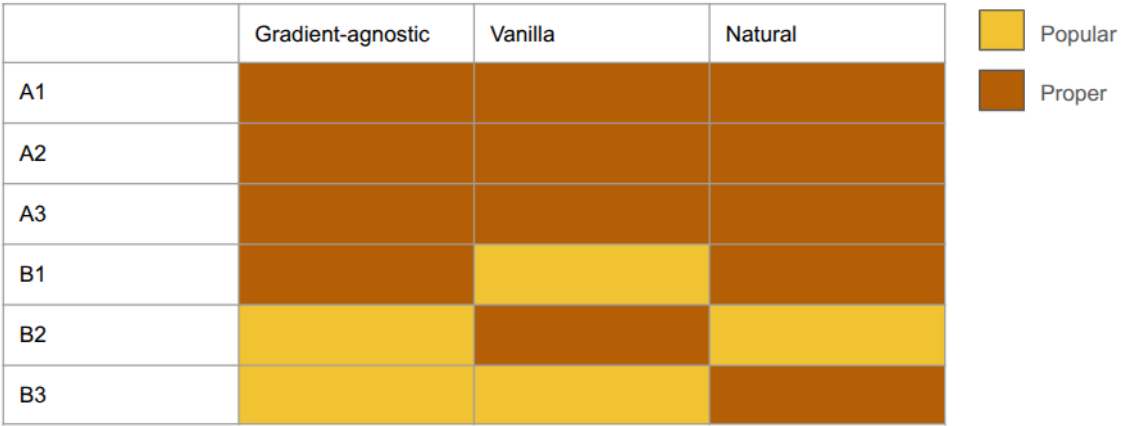


Figure 6.7: Summary - Best methods for discounted between popular and proper sampling from three different perspectives: gradient-type-agnostic, vanilla PG, and natural PG

6.2 Optimization Behavior

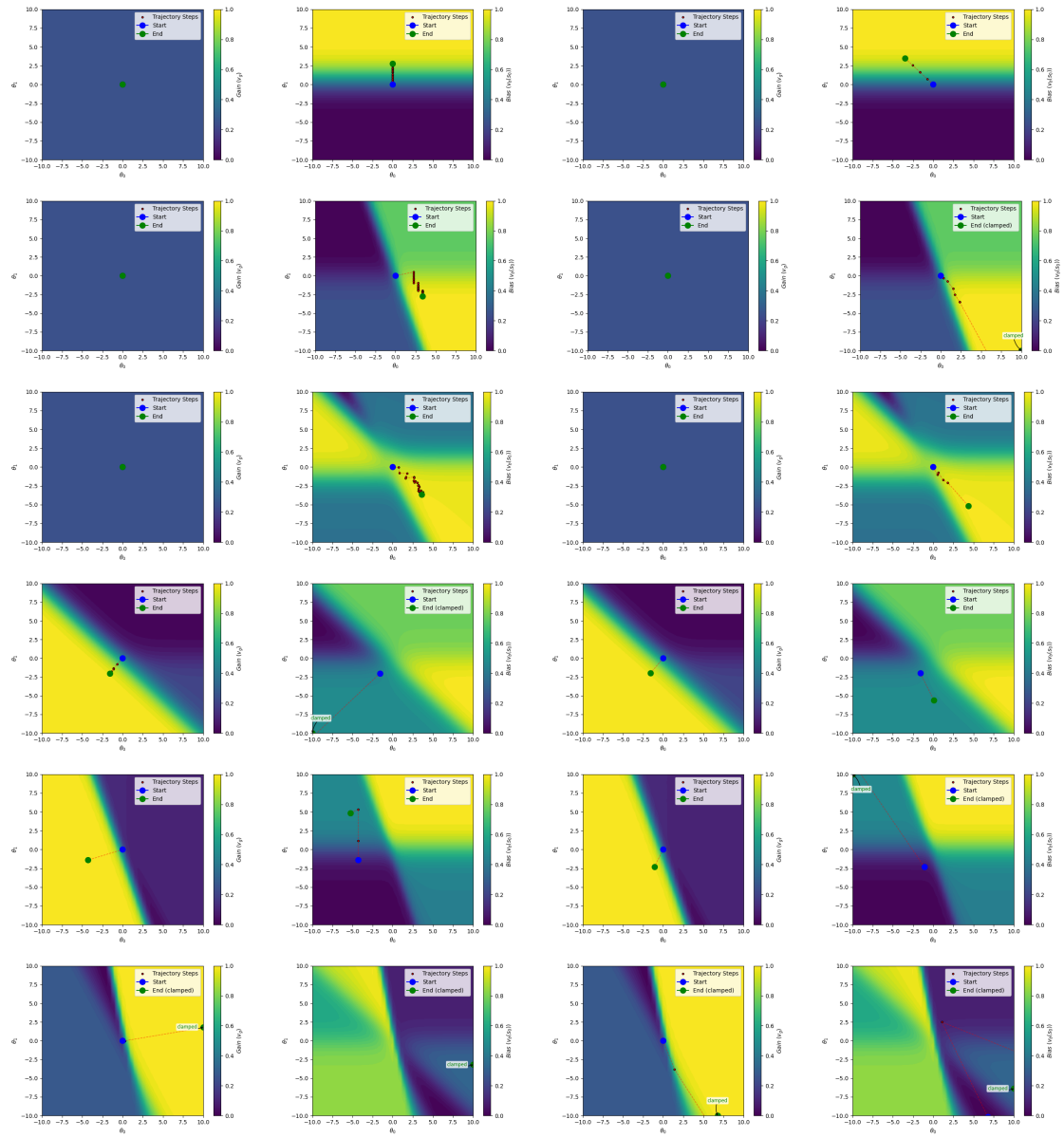


Figure 6.8: Trajectory for Discounting-free scheme. Environment (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: Vanilla PG's gain and bias trajectory, Natural PG's gain and bias trajectory.

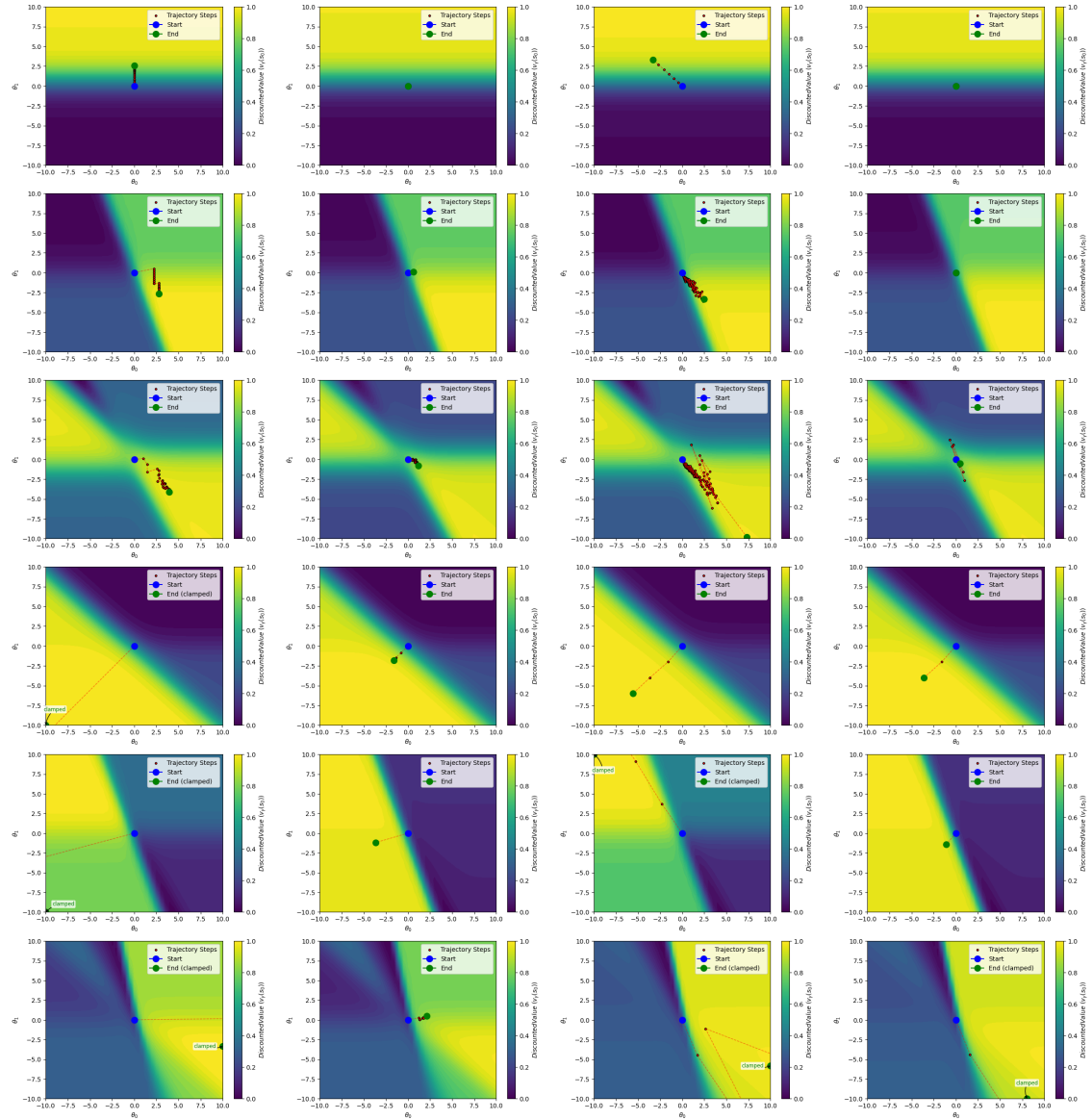


Figure 6.9: Trajectory for Discounted scheme. Environment (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: Vanilla PG's with popular sampling, Vanilla PG's with proper sampling, Natural PG's with popular sampling, Natural PG's with proper sampling.

6.2.1 Exact Policy Gradient

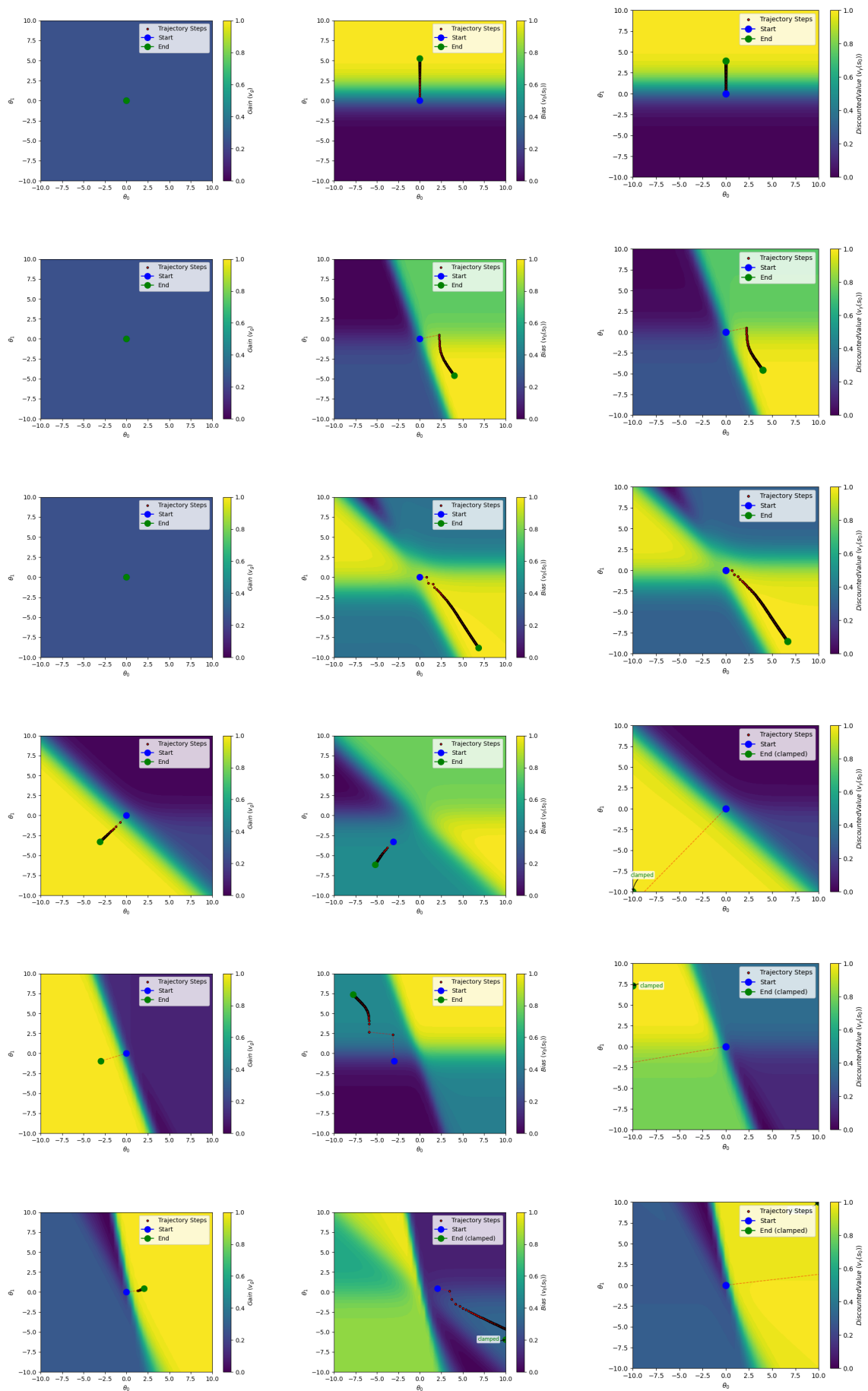
To better understand the intrinsic behavior of policy gradient methods, we first analyze them in the exact setting. By computing gradients and value functions exactly, i.e., using complete knowledge of the environment, we remove sampling noise and approximation errors. With this scheme, we can obtain a clearer baseline to observe convergence patterns, step sizes, and any inherent biases in both the gain and bias-optimal objectives. Insights from the exact case could guide our interpretation when comparing against its sampling-

based variants.

6.2.1.1 Vanilla Policy Gradient

The result of exact vanilla policy gradient seems effective for both the discounting-free and discounted policy gradient. For the discounting-free method, it successfully goes to the gain-optimal region then towards the bias-optimal region. For discounted method, all goes towards the yellow regions directly except for environment B2, where initially it goes to a green region, then clamped as the trajectory goes off of the visualized parameter space. In environment with A variant, both methods yields similar trajectory, where the steps seem small but consistent towards the optimal region. For environment with B variant, the discounted scheme is having much bigger trajectory step, thus it goes off of the visualized parameter space faster.

To verify the value of the policy, especially for the clamped ones, the policy's value is plotted across iterations at figure 6.11 below. For the gain + bias visualization, in some environments initially it has a high value, but after some iterations it goes down. There are more than one possible reasons for that, e.g., because of the gain optimal region has low bias value, or the policy converge to a local optima that is not the optimal region. For environment B3, seems that the reason is the first one, because at the iteration 0, the gain value is relatively low while the gain + bias value is relatively high. However, at iteration 1, the gain value goes up and the gain + bias value goes down.



Universitas Indonesia

Figure 6.10: Trajectory for Vanilla Policy Gradients (Exact). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain trajectory, bias trajectory, discounted value trajectory (exact). The resulting γ sequentially from A1 to B3: 0.94, 0.99, 0.97, 0.94, 0.99, 0.94

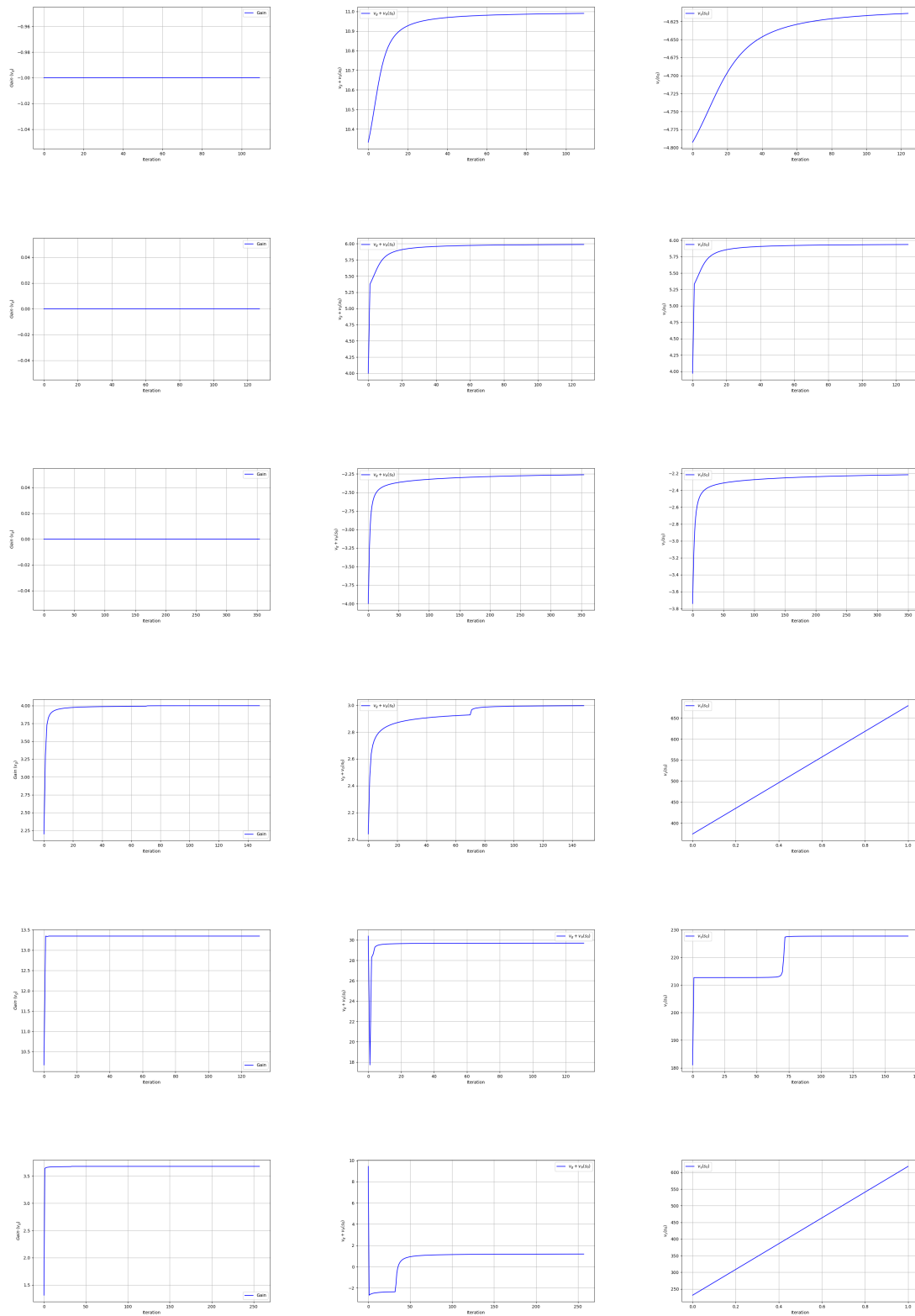


Figure 6.11: Policy Value Progression for Vanilla Policy Gradient (Exact). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain progression, gain + bias progression, discounted value progression

6.2.1.2 Natural Policy Gradient

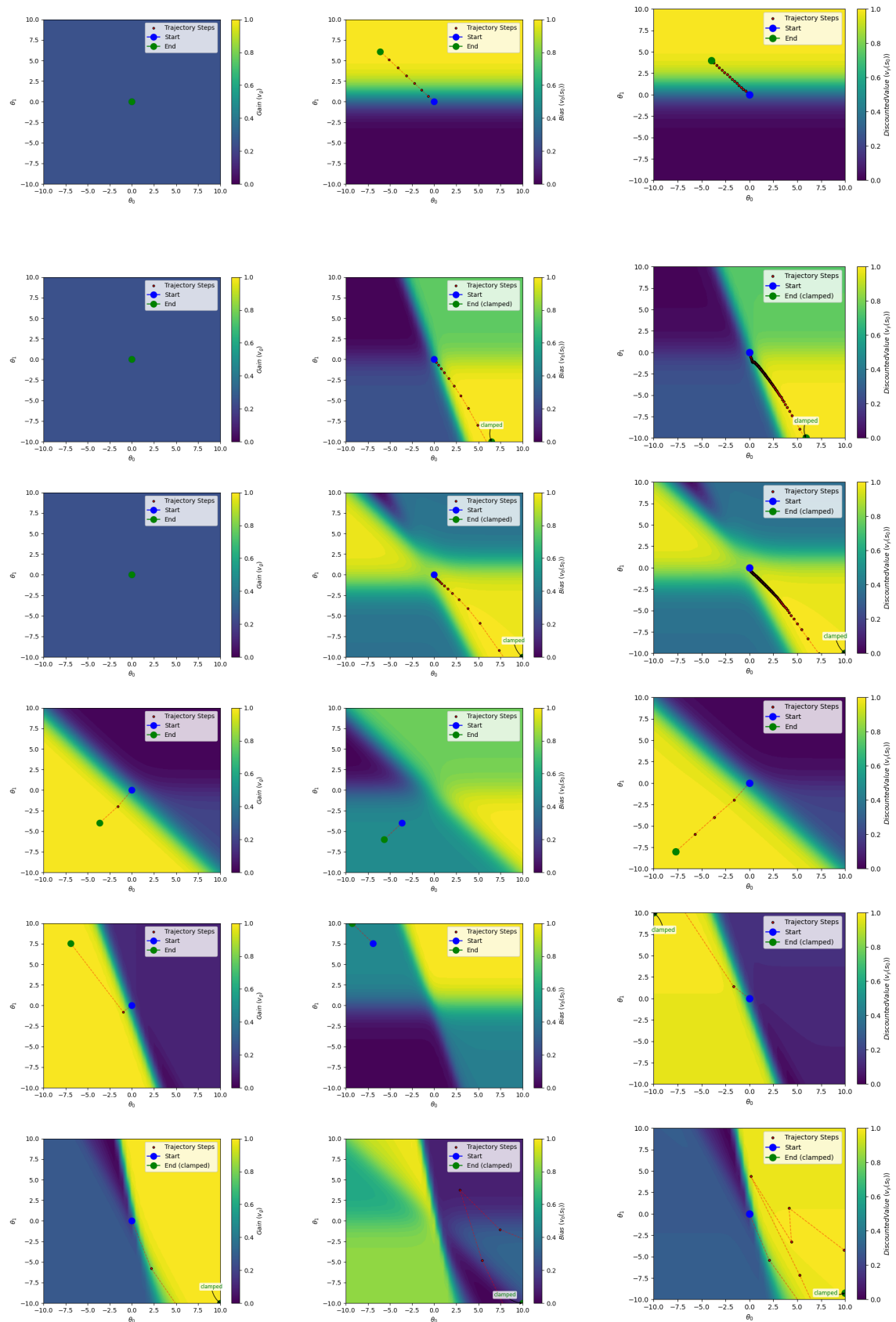


Figure 6.12: Trajectory for Natural Policy Gradients (Exact). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain trajectory, bias trajectory, discounted value trajectory (exact).

The exact natural policy gradient yields an effective progression towards the optimal region. Compared to exact vanilla policy gradient that can be observed at 6.10, the resulting trajectories across all the environments have bigger steps. We can also observe that under discounting-free method, the natural variant converged with much smaller iterations, i.e. less than 13 iterations compared to its vanilla counterpart, which needs bigger than 100 iterations across all environments. Meanwhile, for the discounted method, as the tuned gamma could be different across environments, the objective could be different between the vanilla and the natural variant. Coincidentally, in all environment except A3 and B2, the tuned gamma is the same. However, the natural policy variant only converged faster in environment A1. At both discounting-free and discounted scheme, the policy's value converged to similar values as in the vanilla variant as can be seen at 6.11 and 6.13.

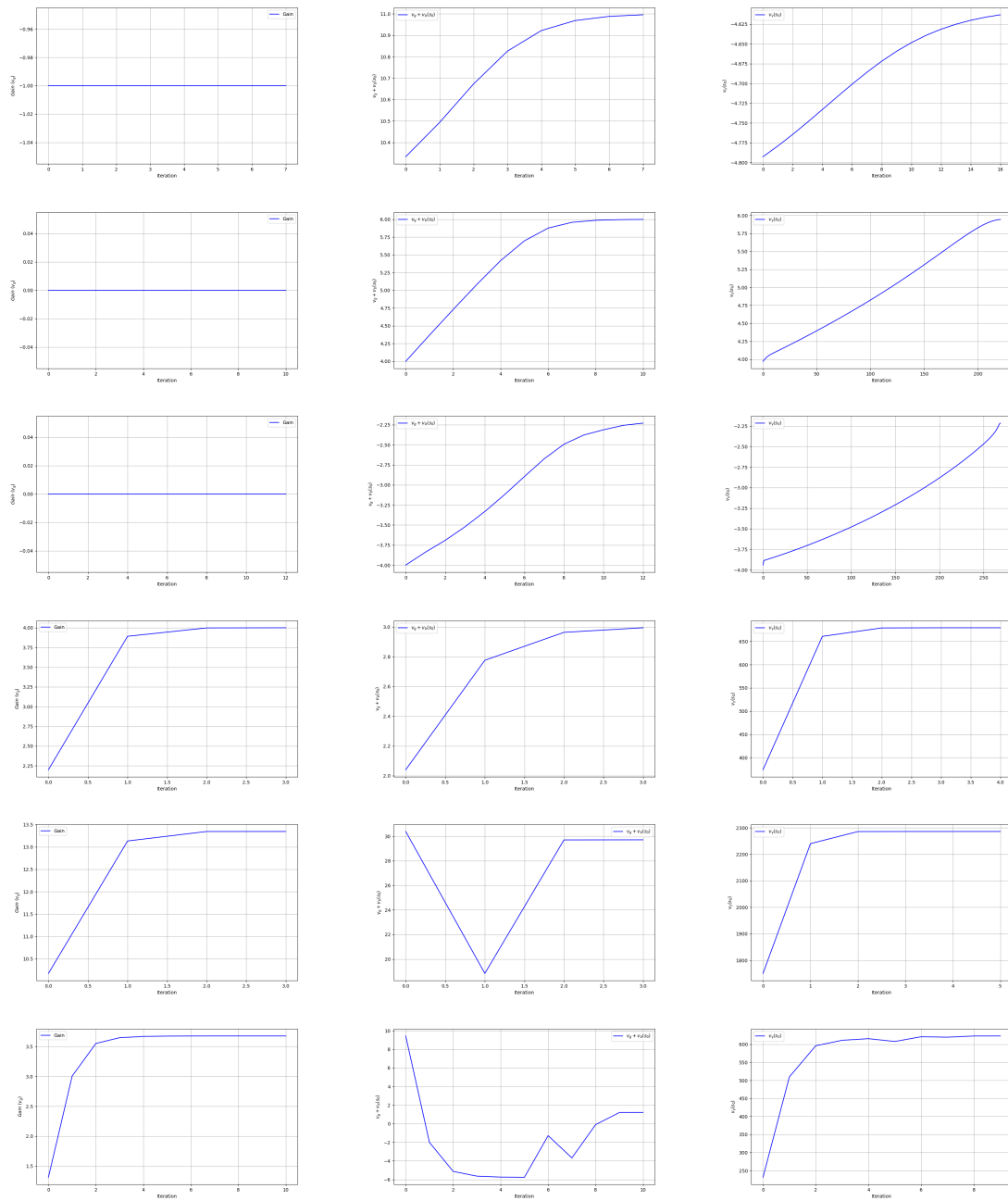


Figure 6.13: Policy Value Progression for Natural Policy Gradient (Exact). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain progression, gain + bias progression, discounted value progression

6.2.2 Policy Gradient

6.2.2.1 Vanilla Policy Gradient

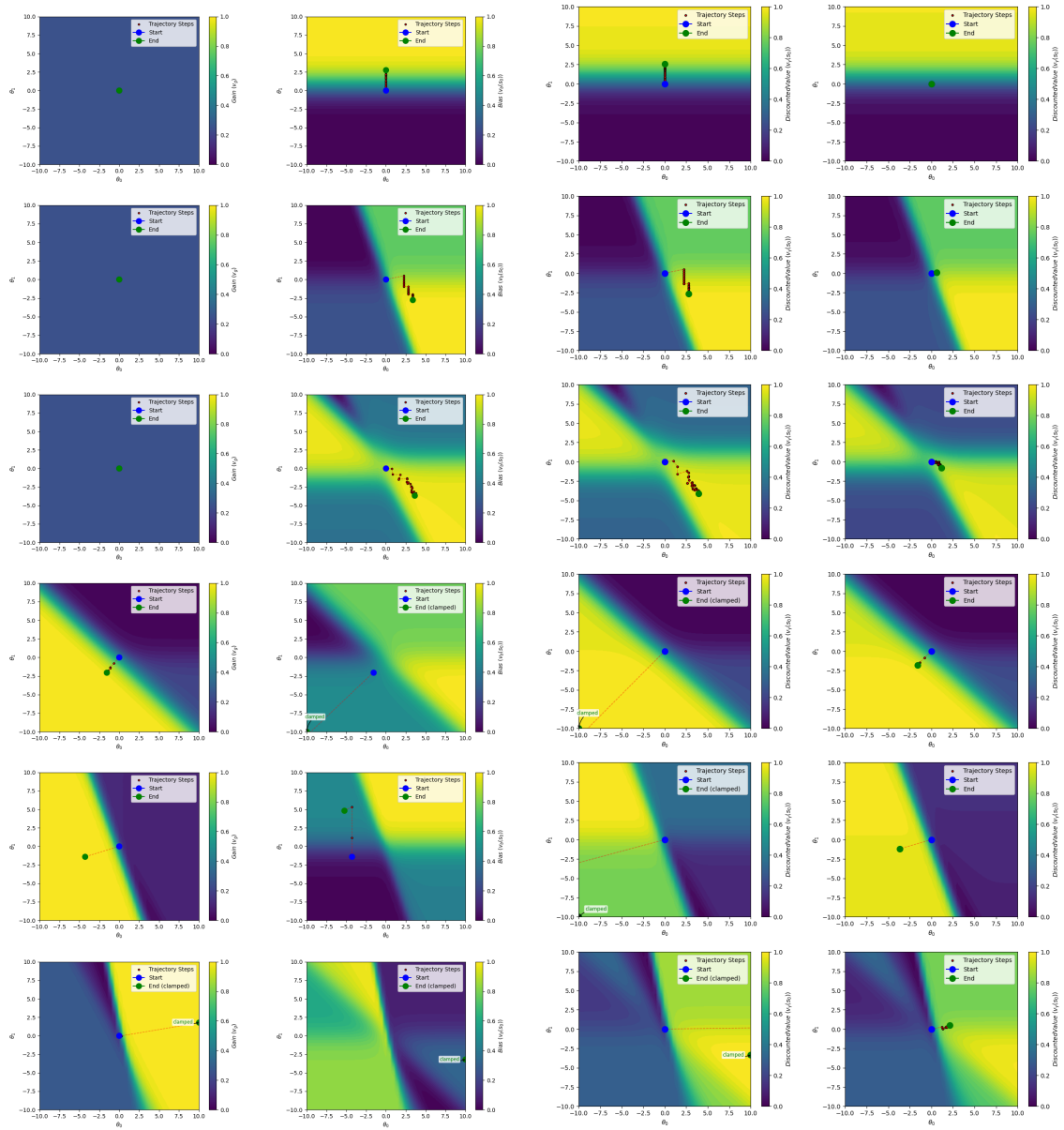


Figure 6.14: Trajectory for Vanilla Policy Gradient (Sampling). Environment (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain trajectory, bias trajectory, discounted value trajectory (popular), discounted value trajectory (proper). The resulting γ sequentially from A1 to B3:
0.99, 0.96, 0.97, 0.91, 0.99, 0.91, 0.94, 0.91, 0.94, 0.99, 0.94, 0.92

For discounting-free scheme, vanilla policy gradient with sampling seems working effectively. Even though the trajectory is not as smooth as the exact method, it eventually leads to the optimal region. We can verify this by looking at the converged policy's value at 6.15, which is comparable with its exact counterpart at 6.11. Not only that, the number

of iterations needed for convergence is much smaller than its exact counterpart, i.e., less than 30 for all environments, compared to at least bigger than 100 in the exact scheme.

However, for discounted scheme, the policy gradient seems not working as effective as in discounting-free. For example in environment A1, both popular or proper sampling yields a trajectory that seems doesn't move much from its origin position. In environment A2, the same thing happened with proper sampling. In environment B3, the proper sampling ends up in the green region instead of yellow region.

If we compare the resulting trajectories under popular sampling and proper sampling, the first one produces more trajectory steps and thus ends up in a farther position from its origin. We can also verify this by observing the figure at 6.15, where in popular sampling method, the policy value converged to a bigger value. But note that the tuned gamma for both methods might be different.

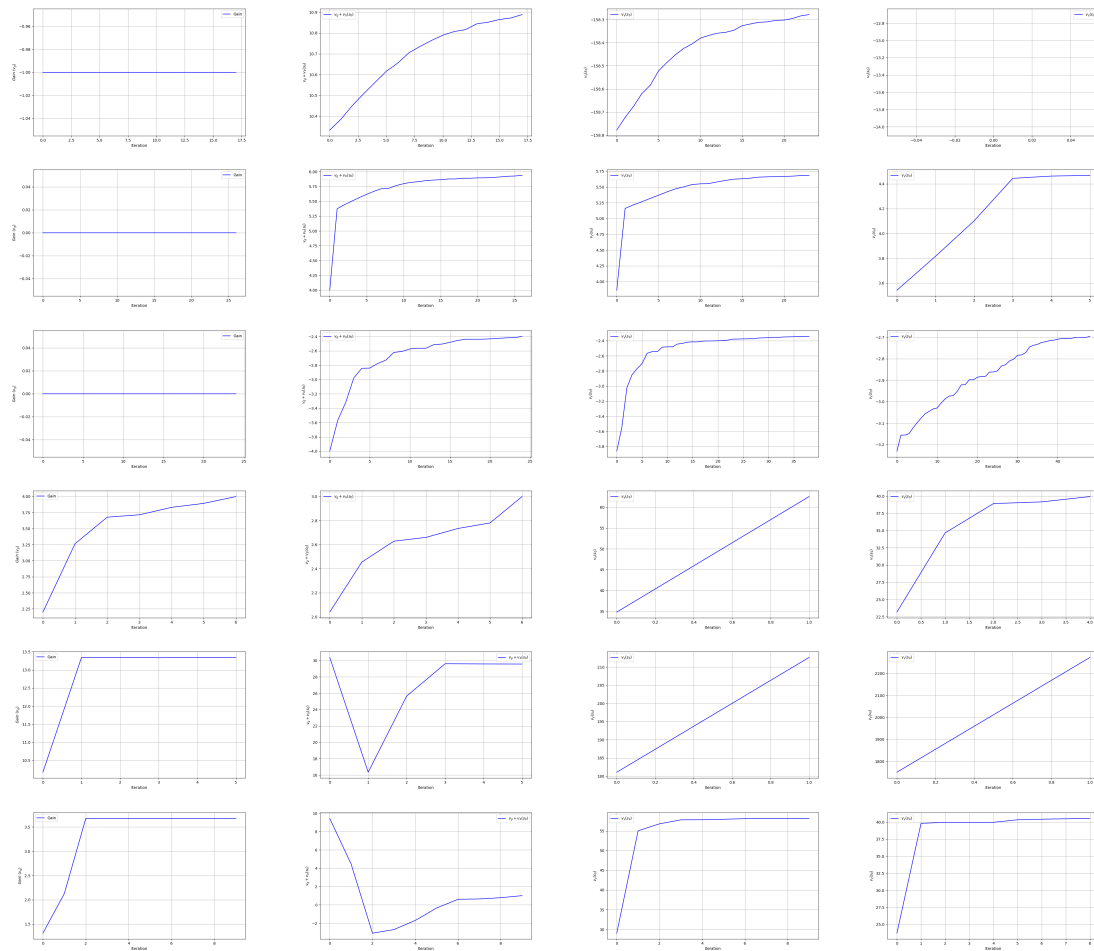


Figure 6.15: Policy Value Progression for Vanilla Policy Gradient (Sampling). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain progression, gain + bias progression, discounted value progression for popular sampling, discounted value progression for proper sampling. If there is no resulting line plotted, it means that the policy converged directly at the first iteration.

6.2.2.2 Natural Policy Gradient

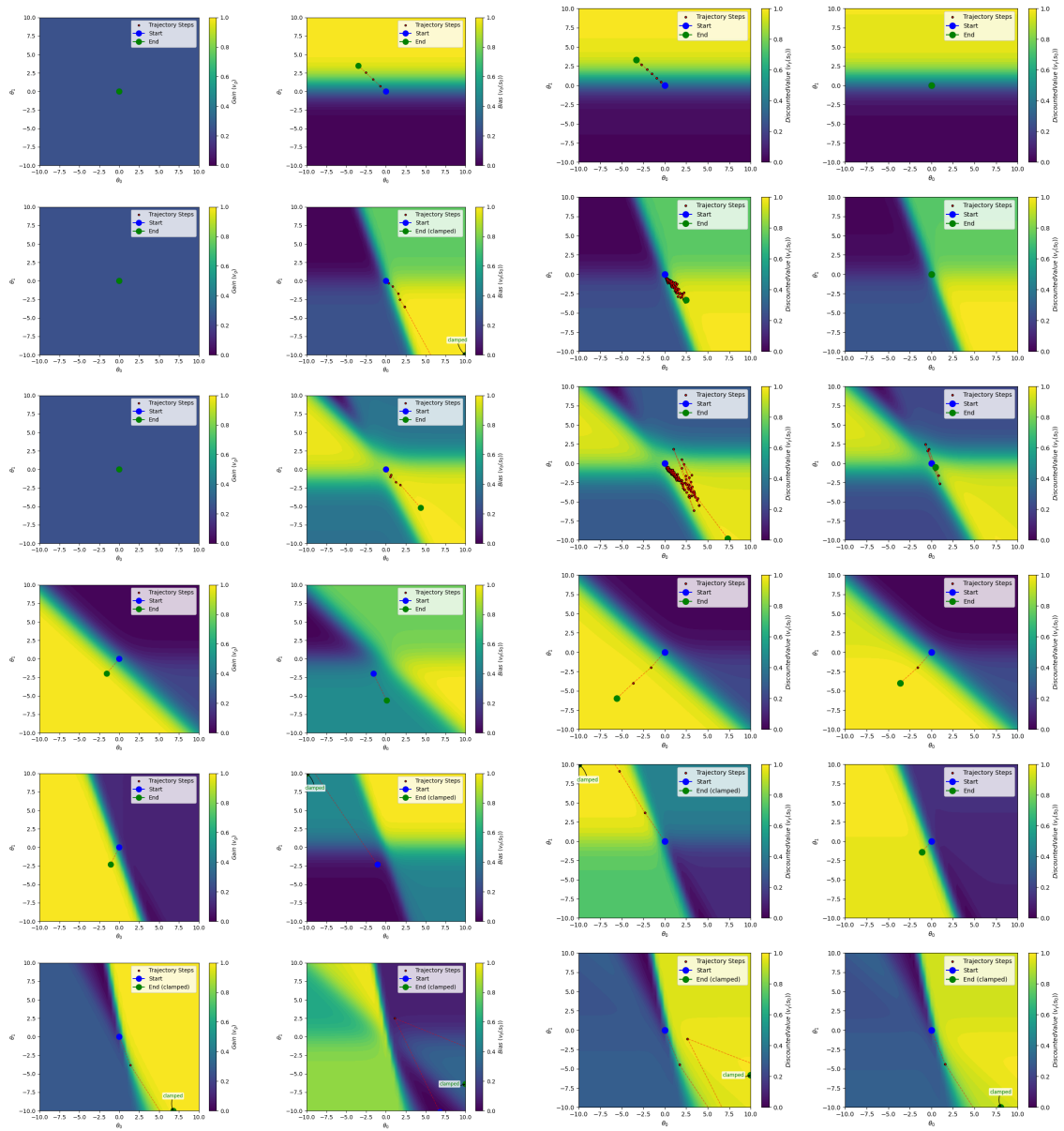


Figure 6.16: Trajectory for Natural Policy Gradient (Sampling). Environment (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain trajectory, bias trajectory, discounted value trajectory (popular), discounted value trajectory (proper). The resulting γ sequentially from A1 to B3: 0.97, 0.96, 0.94, 0.99, 0.96, 0.91, 0.99, 0.91, 0.92, 0.99, 0.99, 0.97

In discounting-free scheme, it produces similar result as its vanilla sampling method in terms of its effectiveness. The difference is that the natural policy gradient seems to have bigger steps, and thus it converged in fewer iterations. Both methods go to similar values in all the environments.

For discounted scheme, same as in vanilla sampling, the natural sampling is not as effective

as in the discounting-free scheme. The reason is also similar, i.e., at environment A1 at environment A2 with proper sampling, the parameter converged to a very close positions from its origin, which seems doesn't have the brighter color compared to its relatives.

Between vanilla and natural with proper sampling, both have the same tuned gamma is in environment A3, B1, and B2. In environment A3 and B2, the vanilla proper sampling yields a slightly higher value. However, it is the other way around for environment B1, where natural proper sampling yields a higher value compared to the vanilla proper sampling.

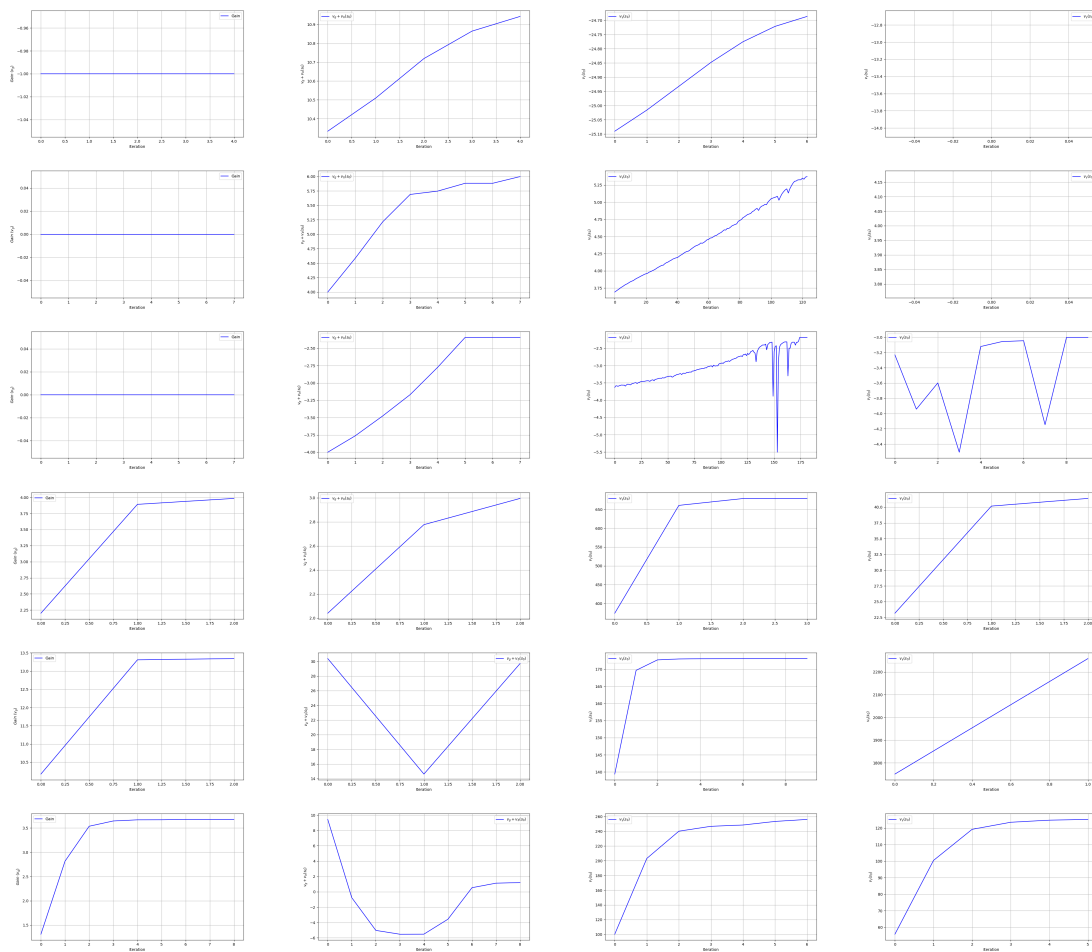


Figure 6.17: Policy Value Progression for Natural Policy Gradient (Sampling). Environments (from top to bottom): A1, A2, A3, B1, B2, B3. From left to right: gain progression, gain + bias progression, discounted value progression for popular sampling, discounted value progression for proper sampling. If there is no resulting line plotted, it means that the policy converged directly at the first iteration.

CHAPTER 7

CONCLUSIONS

7.1 Conclusions

1. Primary (Discounting-free vs Discounted scheme)

Discounting-free policy gradients works effectively in all 6 synthetic environments. In contrast, discounted policy gradients is not that effective compared to discounting-free. Across all environments tested, discounting-free scheme is more sample-efficient.

2. Secondary (Proper vs Popular Sampling)

In discounted scheme, proper sampling is more sample-efficient. However, in average the popular sampling is more effective. In most of the cases, popular sampling produces bigger trajectory steps and converged to a more optimal region.

Therefore, by the two conclusions above, to get better sample efficiency and thus less compute, in practice when dealing with problems in environments with transient states, we should consider to use discounting-free based policy gradients. Or, if using discounted based scheme, we should consider to use proper sampling method.

7.2 Limitations

1. Environments used are simple, synthetic environments
2. Policy gradient still involve exact computations for policy evaluations.
3. By its gradient type, this research only consider vanilla and natural policy gradients.
4. We are still using basic configurations for the hyperparameter optimization, e.g., no pruning yet.
5. The hyperparameter tuning evaluations for each tuning seed used window size of last 10 iterations.
6. We haven't derived the sample complexity in the form of upper bounds.

7.3 Future Works

1. Use realistic environments to get deeper insight in more practical scenarios.
2. Use approximations for policy improvements.
3. Consider adding more types of policy gradients.

4. Use more advanced hyperparameter optimization configurations, e.g., use pruning for early stopping unpromising trials.
5. Consider bigger window size for evaluations in hyperparameter tuning process to get a more accurate evaluation metrics.
6. Consider characteristics of the MDPs and experiment with different convergence thresholds to derive the sample complexity upper bounds.

REFERENCES

- Agarwal, A., Kakade, S. M., Lee, J. D., and Mahajan, G. (2019). Optimality and approximation with policy gradient methods in markov decision processes. *CoRR*, abs/1908.00261.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. *CoRR*, abs/1907.10902.
- Amit, R., Meir, R., and Ciosek, K. (2020). Discount factor as a regularizer in reinforcement learning. *CoRR*, abs/2007.02040.
- Bai, Q., Mondal, W. U., and Aggarwal, V. (2024). Regret analysis of policy gradient algorithm for infinite horizon average reward markov decision processes.
- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.
- Berner, C., Brockman, G., Chan, B., Cheung, V., Debiak, P., Dennison, C., Farhi, D., Fischer, Q., Hashme, S., Hesse, C., Józefowicz, R., Gray, S., Olsson, C., Pachocki, J., Petrov, M., de Oliveira Pinto, H. P., Raiman, J., Salimans, T., Schlatter, J., Schneider, J., Sidor, S., Sutskever, I., Tang, J., Wolski, F., and Zhang, S. (2019). Dota 2 with large scale deep reinforcement learning. *CoRR*, abs/1912.06680.
- Boyd, S. and Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.
- Briggs, A. and Sculpher, M. (1998). An introduction to markov modelling for economic evaluation. *Pharmacoeconomics*, 13(4):397–409.
- DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., Zhang, X., Yu, X., Wu, Y., Wu, Z. F., Gou, Z., Shao, Z., Li, Z., Gao, Z., Liu, A., Xue, B., Wang, B., Wu, B., Feng, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Ding, H., Xin, H., Gao, H., Qu, H., Li, H., Guo, J., Li, J., Wang, J., Chen, J., Yuan, J., Qiu, J., Li, J., Cai, J. L., Ni, J., Liang, J., Chen, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Zhao, L., Wang, L., Zhang, L., Xu, L., Xia, L., Zhang, M., Zhang, M., Tang, M., Li, M., Wang, M., Li, M., Tian, N., Huang, P., Zhang, P., Wang, Q., Chen, Q., Du, Q., Ge, R., Zhang, R., Pan, R., Wang, R., Chen, R. J., Jin, R. L., Chen, R., Lu, S., Zhou, S., Chen, S., Ye, S., Wang, S., Yu, S., Zhou, S., Pan, S., Li, S. S., Zhou, S., Wu, S., Ye, S., Yun, T., Pei, T., Sun, T., Wang, T., Zeng, W.,

- Zhao, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Xiao, W. L., An, W., Liu, X., Wang, X., Chen, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yang, X., Li, X., Su, X., Lin, X., Li, X. Q., Jin, X., Shen, X., Chen, X., Sun, X., Wang, X., Song, X., Zhou, X., Wang, X., Shan, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhang, Y., Xu, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Yu, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Ou, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Tang, Y., Zha, Y., Yan, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Ma, Z., Yan, Z., Wu, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Pan, Z., Huang, Z., Xu, Z., Zhang, Z., and Zhang, Z. (2025). Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.
- Dewanto, V., Dunn, G., Eshragh, A., Gallagher, M., and Roosta, F. (2020). Average-reward model-free reinforcement learning: a systematic review and literature mapping. *CoRR*, abs/2010.08920.
- Dewanto, V. and Gallagher, M. (2021a). Examining average and discounted reward optimality criteria in reinforcement learning. *CoRR*, abs/2107.01348.
- Dewanto, V. and Gallagher, M. (2021b). A nearly blackwell-optimal policy gradient method. *CoRR*, abs/2105.13609.
- Ganesh, S. and Aggarwal, V. (2025). Regret analysis of average-reward unichain mdps via an actor-critic approach.
- Grand-Clement, J. and Petrik, M. (2023). Reducing blackwell and average optimality to discounted mdps via the blackwell discount factor.
- Hordijk, A., Dekker, R., and Kallenberg, L. C. M. (1985). Sensitivity-analysis in discounted markovian decision problems. *OR Spektrum*, 7:143–151.
- Ilyas, A., Engstrom, L., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., and Madry, A. (2018). Are deep policy gradient algorithms truly policy gradient algorithms? *CoRR*, abs/1811.02553.
- Jaderberg, M., Dalibard, V., Osindero, S., Czarnecki, W. M., Donahue, J., Razavi, A., Vinyals, O., Green, T., Dunning, I., Simonyan, K., Fernando, C., and Kavukcuoglu, K. (2017). Population based training of neural networks. *CoRR*, abs/1711.09846.
- Kakade, M. S. (2003). *On the Sample Complexity of Reinforcement Learning*. PhD thesis.
- Lehmann, M. (2024). The definitive guide to policy gradients in deep reinforcement

learning: Theory, algorithms and implementations.

- Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N. M. O., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2015). Continuous control with deep reinforcement learning. *CoRR*, abs/1509.02971.
- McClean, S. (2021). Using markov models to characterize and predict process target compliance. *Mathematics*, 9(11).
- Mei, J., Chung, W., Thomas, V., Dai, B., Szepesvari, C., and Schuurmans, D. (2023). The role of baselines in policy gradient optimization.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., Silver, D., and Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. *CoRR*, abs/1602.01783.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. A. (2013). Playing atari with deep reinforcement learning. *CoRR*, abs/1312.5602.
- Moldovan, T. M. and Abbeel, P. (2012). Safe exploration in markov decision processes. *CoRR*, abs/1205.4810.
- Mondal, W. U. and Aggarwal, V. (2024). Improved sample complexity analysis of natural policy gradient algorithm with general parameterization for infinite horizon discounted reward markov decision processes.
- Naik, A., Shariff, R., Yasui, N., and Sutton, R. S. (2019). Discounted reinforcement learning is not an optimization problem. *CoRR*, abs/1910.02140.
- Naik, A., Wan, Y., Tomar, M., and Sutton, R. S. (2024). Reward centering.
- Nota, C. and Thomas, P. S. (2019). Is the policy gradient a gradient? *CoRR*, abs/1906.07073.
- OpenAI, Andrychowicz, M., Baker, B., Chociej, M., Józefowicz, R., McGrew, B., Pachocki, J., Petron, A., Plappert, M., Powell, G., Ray, A., Schneider, J., Sidor, S., Tobin, J., Welinder, P., Weng, L., and Zaremba, W. (2018). Learning dexterous in-hand manipulation. *CoRR*, abs/1808.00177.
- Ponse, K., Plaat, A., van Stein, N., and Moerland, T. M. (2025). Econojax: A fast & scalable economic simulation in jax.
- Puterman, M. L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, Inc., USA, 1st edition.
- Robbins, H. and Monroe, S. (1951). A Stochastic Approximation Method. *The Annals of*

Mathematical Statistics, 22(3):400 – 407.

- Schneckenreither, M. (2020). Average reward adjusted discounted reinforcement learning: Near-blackwell-optimal policies for real-world applications. *CoRR*, abs/2004.00857.
- Schulman, J., Levine, S., Moritz, P., Jordan, M. I., and Abbeel, P. (2015). Trust region policy optimization. *CoRR*, abs/1502.05477.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., and Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489.
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T. P., Simonyan, K., and Hassabis, D. (2017a). Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *CoRR*, abs/1712.01815.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., and Hassabis, D. (2017b). Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T. P., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., and Hassabis, D. (2017c). Mastering the game of go without human knowledge. *Nature*, 550:354–359.
- Silver, D., Singh, S., Precup, D., and Sutton, R. S. (2021). Reward is enough. *Artificial Intelligence*, 299:103535.
- Sonnenberg, F. A. and Beck, J. R. (1993). Markov models in medical decision making: A practical guide. *Medical Decision Making*, 13(4):322–338. PMID: 8246705.
- Strens, M. J. A. (2000). A bayesian framework for reinforcement learning. page 943950.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA.
- Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. In Solla, S., Leen, T., and

- Müller, K., editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press.
- Veinott, A. F. (1969). Discrete dynamic programming with sensitive discount optimality criteria. *The Annals of Mathematical Statistics*, 40(5):1635–1660.
- Wan, Y., Korenkevych, D., and Zhu, Z. (2025). An empirical study of deep reinforcement learning in continuing tasks.
- Waschneck, B., Reichstaller, A., Belzner, L., Altenmüller, T., Bauernhansl, T., Knapp, A., and Kyek, A. (2018). Deep reinforcement learning for semiconductor production scheduling. In *2018 29th Annual SEMI Advanced Semiconductor Manufacturing Conference (ASMC)*, pages 301–306.
- Weaver, L. and Tao, N. (2013). The optimal reward baseline for gradient-based reinforcement learning. *CoRR*, abs/1301.2315.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.*, 8(34):229256.
- Wu, Y., Mansimov, E., Liao, S., Grosse, R. B., and Ba, J. (2017). Scalable trust-region method for deep reinforcement learning using kronecker-factored approximation. *CoRR*, abs/1708.05144.
- Yuan, R., Gower, R. M., and Lazaric, A. (2021). A general sample complexity analysis of vanilla policy gradient. *CoRR*, abs/2107.11433.
- Zheng, S., Trott, A., Srinivasa, S., Parkes, D. C., and Socher, R. (2021). The AI economist: Optimal economic policy design via two-level deep reinforcement learning. *CoRR*, abs/2108.02755.

LAMPIRAN

